Question Prompts
Load Libraries
Read in Dataset
Cleaning the Dataset
  Address Missing Values
  Address Remaining Missing Values
  Drop ID Column
  Factor Conversion
  Outliers Analysis
  Visualize the distribution of the SalePrice
Penalized Regression
Support Vector Machines
XGBoost
Principal Component Analysis (PCA)
K-Means Clustering
Artificial Neural Networks
Random Forest
Linear Regression Comparison

Machine Learning Project

Ahsan Ahmad, Varun Selvam, Nikita Muddapati

2024-12-04

Question Prompts

1. Which machine learning methods did you implement?

We implemented the following machine learning methods:

1. Penalized Regression (Ridge, Lasso, Elastic Net)
2. Support Vector Machines (Linear and Radial Kernels)
3. XGBoost
4. Principal Component Analysis (PCA)
5. K-Means Clustering
6. Artificial Neural Networks (ANN)

7. Random Forest

2. Discuss the key contribution of each method to your analysis.

Penalized Regression: Ridge, Lasso, and Elastic Net effectively reduced multicollinearity and handled overfitting. Among them, Elastic Net performed best with the lowest RMSE and highest $R2$, making it the most reliable for regression tasks.

Support Vector Machines (SVM): The linear kernel SVM showed strong predictive performance, second only to Elastic Net in generalization. However, the radial kernel SVM underperformed, likely due to insufficient tuning or inappropriate assumptions about non-linearity in the data.

XGBoost: Although XGBoost is generally known for its performance, it underperformed here with a relatively high RMSE and lower $R2$, suggesting it might not suit this dataset or require further hyperparameter tuning.

Principal Component Analysis (PCA): PCA reduced dimensionality and helped identify the most influential predictors. While it did not directly improve predictive performance, it provided valuable insights into feature importance and guided feature selection for other models.

K-Means Clustering: K-Means offered an unsupervised view of the data, grouping houses into clusters. While clustering was not directly predictive, it could be useful for segmenting markets or identifying hidden patterns in the data.

Artificial Neural Networks (ANN): ANN demonstrated excellent generalization, achieving the second-best RMSE and $R2$ values. This method is robust for capturing complex, non-linear relationships in the data.

Random Forest: Random Forest effectively captured complex patterns and prediction accuracy but highlighted tendency of overfitting.

3. Did all methods support your conclusions or did some provide conflicting results? If so, how did you reconcile the differences?

Not all methods supported the same conclusions.

Elastic Net Regression provided the best predictive performance, with low RMSE and high $R2$, making it the most reliable model. SVM (Linear Kernel) and ANN both supported the conclusions of Elastic Net, performing consistently well. SVM (Radial Kernel) and XGBoost presented conflicting results, with lower $R2$ and higher RMSE, suggesting weaker fits.These results highlight potential issues with hyperparameter tuning or the inherent suitability of these methods for this dataset.

Load Libraries

```
pacman::p_load(DDoutlier,tidyverse,glmnet,car,psych,
               forecast,lmtest,caret,Metrics,rpart,
               rpart.plot,reshape2,rminer,matrixStats,
               knitr,e1071,lattice,tictoc,ISLR,dataPreparation,
               mlbench,ROCR,tidymodels,dbscan,parallel,factoextra,cluster,gridExtra,
doParallel,dials,
               readxl,dplyr,ranger,themis,ggplot2)

library(RWeka)
library(kernlab)
library(yardstick)
```

Read in Dataset

```
house_prices <- read.csv("C:/Users/nikit/Downloads/Machine Learning/train_house_pric
es.csv")
```

Cleaning the Dataset

This dataset has a few issues which must be addressed through cleaning. Outliers must also be addressed. Afterwards, the dataset can be utilized for modelling.

Address Missing Values

```
# Define function to check %age of NAs in each column

find_na <- function(x) sum(is.na(x))

# Apply function to each column with map()
missing_values <- map(.x = house_prices, .f = find_na) %>%
  unlist() %>%
  data.frame()

missing_values %>%
  filter(. != 0) %>%
  mutate(pctg = ./nrow(house_prices)) %>%
  arrange(desc(pctg)) %>%
  round(4)
```

	.	pctg
	<dbl>	<dbl>
PoolQC	1453	0.9952
MiscFeature	1406	0.9630

	. <dbl>	pctg <dbl>
Alley	1369	0.9377
Fence	1179	0.8075
FireplaceQu	690	0.4726
LotFrontage	259	0.1774
GarageType	81	0.0555
GarageYrBlt	81	0.0555
GarageFinish	81	0.0555
GarageQual	81	0.0555

Address Columns greater than 90% of NA's

Check Levels for Variables

The below code checks the levels for the variables that have more than 90% of their values missing.

```
unique(house_prices$PoolQC)
```

```
## [1] NA   "Ex" "Fa" "Gd"
```

```
unique(house_prices$MiscFeature)
```

```
## [1] NA     "Shed" "Gar2" "Othr" "TenC"
```

```
unique(house_prices$Alley)
```

```
## [1] NA     "Grvl" "Pave"
```

Per the Kaggle Dictionary, Na for `PoolQC` means that there is no pool. Thus, these NA values can be replaced with "No Pool"

Additionally, per the Kaggle Dictionary, NA for `MiscFeature` means that there is no Miscellaneous Feature in the house. These NA values can also be replaced with "No Misc Feature".

Finally NA for `Alley` means that there is no Alley Access for that house. Hence NA for this column can also be replaced with "No Alley".

Recode PoolQC

```
# Recoding cols

# Recode NA in FireplaceQu to "No Fireplace"
house_prices$PoolQC[is.na(house_prices$PoolQC)] <- "No Pool"

# Check that recoding the NA values worked
unique(house_prices$PoolQC)
```

```
## [1] "No Pool" "Ex"      "Fa"      "Gd"
```

Recode MiscFeature

```
# Recode NA in MiscFeature to "No Misc Feature"
house_prices$MiscFeature[is.na(house_prices$MiscFeature)] <- "No Misc Feature"

# Check that recoding the NA values worked
unique(house_prices$MiscFeature)
```

```
## [1] "No Misc Feature" "Shed"            "Gar2"            "Othr"
## [5] "TenC"
```

Recode Alley

```
# Recode NA in MiscFeature to "No Misc Feature"
house_prices$Alley[is.na(house_prices$Alley)] <- "No Alley"

# Check that recoding the NA values worked
unique(house_prices$Alley)
```

```
## [1] "No Alley" "Grvl"     "Pave"
```

Address Remaining Missing Values

```
# use above function again to check %age of remaining columns' NA's (<90%)
# recode cols again/impute numeric cols with median and categorical cols with mode

missing_values <- map(.x = house_prices, .f = find_na) %>%
  unlist() %>%
  data.frame()

missing_values %>%
  filter(. != 0) %>%
  mutate(pctg = ./nrow(house_prices)) %>%
  arrange(desc(pctg))
```

	.	pctg
	<int>	<dbl>
Fence	1179	0.8075342466
FireplaceQu	690	0.4726027397
LotFrontage	259	0.1773972603
GarageType	81	0.0554794521
GarageYrBlt	81	0.0554794521
GarageFinish	81	0.0554794521
GarageQual	81	0.0554794521
GarageCond	81	0.0554794521
BsmtExposure	38	0.0260273973
BsmtFinType2	38	0.0260273973

1-10 of 16 rows Previous **1** 2 Next

Analyze Fence Quality

```
unique(house_prices$Fence)
```

```
## [1] NA      "MnPrv" "GdWo"  "GdPrv" "MnWw"
```

The "Fence Quality" is a categorical variable that contains 4 categories: * MnPrv * GdWo * GdPrv * MnWw.

Per the Kaggle Competition Website, this column describes the quality of the fence for a property. The Kaggle Website also states that "NA" represents "No Fence".

Thus, NA can be recoded as "No Fence"

```
# Recode NA in Fence to "No Fence"
house_prices$Fence[is.na(house_prices$Fence)] <- "No Fence"

# Check that recoding the NA values worked
unique(house_prices$Fence)
```

```
## [1] "No Fence" "MnPrv"    "GdWo"     "GdPrv"    "MnWw"
```

Recoding the NA Values was successful.

Analyze Fireplace Qu

`Fireplace Qu` stands for Fireplace Quality and describes the quality of a fireplace. Additionally Kaggle states that NA means that the home does not have a fireplace.

Thus, any NA values will be recoded to represent “No Fireplace”.

```
# Recode NA in FireplaceQu to "No Fireplace"
house_prices$FireplaceQu[is.na(house_prices$FireplaceQu)] <- "No Fireplace"

# Check that recoding the NA values worked
unique(house_prices$FireplaceQu)
```

```
## [1] "No Fireplace" "TA"          "Gd"          "Fa"          "Ex"
## [6] "Po"
```

Analyze Lot Frontage

```
# Select only numeric columns
numeric_data <- house_prices %>%
  select(-Id,-OverallQual,-OverallCond,-MSSubClass) %>%
  select(where(is.numeric))

# Calculate the correlation matrix
cor_matrix <- cor(numeric_data, use = "complete.obs")

# Extract correlations for LotFrontage
lotfrontage_correlations <- cor_matrix["LotFrontage", ]

# Display the correlations
lotfrontage_correlations
```

```
##   LotFrontage       LotArea     YearBuilt  YearRemodAdd    MasVnrArea
##  1.0000000000  0.4211841021  0.1097255707  0.0864139680  0.1899685917
##    BsmtFinSF1    BsmtFinSF2     BsmtUnfSF   TotalBsmtSF     X1stFlrSF
##  0.2413522339  0.0493053240  0.1153058755  0.3876195129  0.4510850287
##     X2ndFlrSF  LowQualFinSF     GrLivArea  BsmtFullBath  BsmtHalfBath
##  0.0750038007  0.0111480855  0.3963060214  0.1180881480  0.0004335725
##      FullBath      HalfBath  BedroomAbvGr  KitchenAbvGr  TotRmsAbvGrd
##  0.1857853092  0.0456783485  0.2704038861 -0.0035464716  0.3484211056
##    Fireplaces   GarageYrBlt    GarageCars    GarageArea    WoodDeckSF
##  0.2603208280  0.0698781184  0.2865868103  0.3568509373  0.0821656268
##   OpenPorchSF EnclosedPorch    X3SsnPorch   ScreenPorch      PoolArea
##  0.1618151174  0.0142610142  0.0697157667  0.0359059844  0.2117461173
##       MiscVal        MoSold        YrSold     SalePrice
##  0.0014707377  0.0188145348  0.0132670710  0.3442697721
```

Lot Frontage does not seem to be strongly correlated with any of the numeric variables. Thus, using a package like Amelia might not be very appropriate. (Amelia would basically create a model that takes the other columns and uses them to predict potential values for the other columns.)

It should also be noted that although the correlation with the Target Variable `SalePrice` is quite low at .34, it could be still be useful in combination with other variables.

However in terms of dealing with the NA's, imputation with the median should be fairly reliable. The column is only missing 17.74% of its values. This means that utilizing the median should be reliable since most of the column's values (82.26%) are present.

```
# Compute the median
median_lot_frontage <- median(house_prices$LotFrontage,na.rm = TRUE)   # median= 69

median_lot_frontage
```

```
## [1] 69
```

```
# Impute with the median
house_prices$LotFrontage[is.na(house_prices$LotFrontage)] <- median_lot_frontage

# Check Missing Values
sum(is.na(house_prices$LotFrontage))
```

```
## [1] 0
```

Analyze all the Garage Variables

The Garage Variables all are missing 81 variables which comprise 5.55% of the dataset:

- `GarageType`
- `GarageYrBlt`
- `GarageFinish`
- `GarageQual`
- `GarageCond`

According to Kaggle, NA for `GarageType` , `GarageFinish` , `GarageQual` , `GarageCond` represents "No Garage". This means that the NA value can be recoded as "No Garage" for the above columns.

NA for "Garage Built" however will be a little tricky since including a number like 0 or -9999 or 1 to represent no Garage may confuse any ensemble models. This is because the model may believe that 1 or -9999 is a valid year.

Thus, to avoid any potential confusion or creating possible outliers like 9999, we will just leave NA in `GarageYrBlt` alone. (We can't replace it with a string like "No Garage" because the column type will change and it'll no longer be numeric.)

Instead, a new column will be created that more explicitly states whether a house has a garage or not. Thus, a new column will be created wtih "0" representing no garage and "1" representing a garage. (This info can be gleaned from the other Garage Columns but we believe that having an explicit column for

Garage will make it easier to assess Garage's influence on price and it can be used to compare houses that don't have garages vs having a garage for clustering as well. (We may do clustering and then run a model on that cluster, so having this column will be helpful for that as well.)

```
# create a vector to store all the columns
garage_col <- c("GarageType", "GarageFinish", "GarageQual", "GarageCond")

# Replace NA in specified columns with new value
house_prices <- house_prices %>%
  mutate(across(all_of(garage_col), ~ replace_na(., "No Garage")))
```

The new column `HasGarage` has been created to represent explicitly whether a house has a garage or not.

```
house_prices <- house_prices %>%
  mutate(HasGarage = ifelse(is.na(GarageYrBlt), "No", "Yes")  # Create a binary indi
cator for garage presence
  )
```

Analyze all the Basement Variables

According to Kaggle, NA for `BsmtExposure` , `BsmtFinType2` , `BsmtQual` , `BsmtCond` and `BsmtFinType1` represents "No Basement". This means that the NA value can be recoded as "No Basement" for the above columns.

They are also all missing roughly the same amount of values in all the columns.

```
# create a vector to store all the columns
basement_col <- c("BsmtExposure", "BsmtFinType2", "BsmtQual", "BsmtCond","BsmtFinTyp
e1")

# Replace NA in specified columns with new value
house_prices <- house_prices %>%
  mutate(across(all_of(basement_col), ~ replace_na(., "No Basement")))
```

Analyze all the MasVnr Variables

Kaggle states that if there is no Masonry Veneer for the house, then the value is "None" for the `MasVnrType` variable. This indicates that the variable is genuinely missing values. (i.e any NA values are geniunely missing values and cannot be imputed with "None")

Additionally, if a house does not have a Masonry Veneer, then the `MasVnrArea` is zero. The below code chunk shows this:

```
house_prices %>%
  select(MasVnrType,MasVnrArea) %>% # Select the following two columns
  filter(MasVnrType == "None") %>% # Only select rows where client marked "None".
  head()
```

	MasVnrType <chr>	MasVnrArea <int>
1	None	0
2	None	0
3	None	0
4	None	0
5	None	0
6	None	0

6 rows

The code shows that everytime `MasVnrType` is None, the `MasVnrArea` variable is Zero. Despite this, there are only 8 values missing in both columns which is less than 1% of the data at 0.54% This is a very small amount which means that imputing with median/mode is appropriate.

```
# Define function to find the mode of a character vector
find_mode <- function(x){
  table(x) %>%
    which.max() %>%
    names()
}

# Compute with the median
MasVnrType_mode <- find_mode(house_prices$MasVnrType)

MasVnrType_mode
```

```
## [1] "None"
```

```
# Impute with the median
house_prices$MasVnrType[is.na(house_prices$MasVnrType)] <- MasVnrType_mode  # mode =
"None"

# Check Missing Values
sum(is.na(house_prices$MasVnrType))
```

```
## [1] 0
```

```
# Compute with the median
median_MasVnrArea <- median(house_prices$MasVnrArea,na.rm = TRUE)  #median = 0
median_MasVnrArea
```

```
## [1] 0
```

```
# Impute with the median
house_prices$MasVnrArea[is.na(house_prices$MasVnrArea)] <- median_MasVnrArea

# Check Missing Values
sum(is.na(house_prices$MasVnrArea))
```

```
## [1] 0
```

MasVnrType has been successfully imputed with the mode while MasVnrArea has also been successfully imputed with the median.

Analyze Electrical Variable

The Electrical Variable is only missing one observation. This missing value can be replaced with the mode since it's a categorical variable.

```
# Get the mode for electrical
electrical_mode <- find_mode(house_prices$Electrical)

# Impute with the mode
house_prices$Electrical[is.na(house_prices$Electrical)] <- electrical_mode

# Check missing values
sum(is.na(house_prices$Electrical))
```

```
## [1] 0
```

The imputation was successful, there are no missing values, except for `GarageYrBlt` This however was done on purpose, please see "Analyze all the Garage Variables" for more information.

```
map(.x = house_prices, .f = find_na) %>%
  unlist() %>%
  data.frame()
```

	.
	<int>
Id	0
MSSubClass	0
MSZoning	0
LotFrontage	0
LotArea	0

	.
	<int>
Street	0
Alley	0
LotShape	0
LandContour	0
Utilities	0

1-10 of 82 rows Previous **1** 2 3 4 5 6 … 9 Next

Drop ID Column

```
# Drop Id column not needed
house_prices <- house_prices %>%
  select(-Id)
```

The Id Column is dropped since it is just used to identify each individual house. It does not say anything else or reveal any interesting characteristics about the house.

Factor Conversion

```
# Factoring all categorical and numeric variables

# Mutate any character variables into factors
house_prices <- house_prices %>% mutate_if(is.character, as.factor)

# Numeric Variables which should be factors stored as a vector
cat_values <- c("OverallQual","OverallCond","MSSubClass", "MoSold")

# Utilize mutate to transform all the columns in the cat_values vector to factors.
house_prices <- house_prices %>%
  mutate(across(all_of(cat_values), as.factor))

# Display the summary of train_clean
summary(house_prices)
```

```
##    MSSubClass     MSZoning     LotFrontage        LotArea        Street
##  20     :536   C (all):  10   Min.   : 21.00   Min.   :  1300   Grvl:   6
##  60     :299   FV     :  65   1st Qu.: 60.00   1st Qu.:  7554   Pave:1454
##  50     :144   RH     :  16   Median : 69.00   Median :  9478
##  120    : 87   RL     :1151   Mean   : 69.86   Mean   : 10517
##  30     : 69   RM     : 218   3rd Qu.: 79.00   3rd Qu.: 11602
##  160    : 63                  Max.   :313.00   Max.   :215245
##  (Other):262
##       Alley      LotShape  LandContour  Utilities      LotConfig    LandSlope
##  Grvl    :  50   IR1:484   Bnk:  63    AllPub:1459   Corner : 263   Gtl:1382
##  No Alley:1369   IR2: 41   HLS:  50    NoSeWa:   1   CulDSac:  94   Mod:  65
##  Pave    :  41   IR3: 10   Low:  36                  FR2    :  47   Sev:  13
##                  Reg:925   Lvl:1311                  FR3    :   4
##                                                      Inside :1052
##
##
##   Neighborhood   Condition1     Condition2     BldgType      HouseStyle
##  NAmes  :225   Norm   :1260   Norm   :1445   1Fam  :1220   1Story :726
##  CollgCr:150   Feedr  :  81   Feedr  :   6   2fmCon:  31   2Story :445
##  OldTown:113   Artery :  48   Artery :   2   Duplex:  52   1.5Fin :154
##  Edwards:100   RRAn   :  26   PosN   :   2   Twnhs :  43   SLvl   : 65
##  Somerst: 86   PosN   :  19   RRNn   :   2   TwnhsE: 114   SFoyer : 37
##  Gilbert: 79   RRAe   :  11   PosA   :   1                 1.5Unf : 14
##  (Other):707   (Other):  15   (Other):   2                 (Other): 19
##   OverallQual   OverallCond    YearBuilt     YearRemodAdd    RoofStyle
##  5      :397   5      :821   Min.   :1872   Min.   :1950   Flat   :  13
##  6      :374   6      :252   1st Qu.:1954   1st Qu.:1967   Gable  :1141
##  7      :319   7      :205   Median :1973   Median :1994   Gambrel:  11
##  8      :168   8      : 72   Mean   :1971   Mean   :1985   Hip    : 286
##  4      :116   4      : 57   3rd Qu.:2000   3rd Qu.:2004   Mansard:   7
##  9      : 43   3      : 25   Max.   :2010   Max.   :2010   Shed   :   2
##  (Other): 43   (Other): 28
##     RoofMatl     Exterior1st   Exterior2nd    MasVnrType    MasVnrArea
##  CompShg:1434   VinylSd:515   VinylSd:504   BrkCmn : 15   Min.   :   0.0
##  Tar&Grv:  11   HdBoard:222   MetalSd:214   BrkFace:445   1st Qu.:   0.0
##  WdShngl:   6   MetalSd:220   HdBoard:207   None   :872   Median :   0.0
##  WdShake:   5   Wd Sdng:206   Wd Sdng:197   Stone  :128   Mean   : 103.1
##  ClyTile:   1   Plywood:108   Plywood:142                 3rd Qu.: 164.2
##  Membran:   1   CemntBd: 61   CmentBd: 60                 Max.   :1600.0
##  (Other):   2   (Other):128   (Other):136
##  ExterQual ExterCond  Foundation         BsmtQual          BsmtCond
##  Ex: 52    Ex:   3   BrkTil:146   Ex         :121   Fa         :  45
##  Fa: 14    Fa:  28   CBlock:634   Fa         : 35   Gd         :  65
##  Gd:488    Gd: 146   PConc :647   Gd         :618   No Basement:  37
##  TA:906    Po:   1   Slab  : 24   No Basement: 37   Po         :   2
##            TA:1282   Stone :  6   TA         :649   TA         :1311
##                      Wood  :  3
##
```

```
##       BsmtExposure      BsmtFinType1   BsmtFinSF1          BsmtFinType2
##  Av         :221   ALQ        :220   Min.   :   0.0   ALQ        :  19
##  Gd         :134   BLQ        :148   1st Qu.:   0.0   BLQ        :  33
##  Mn         :114   GLQ        :418   Median : 383.5   GLQ        :  14
##  No         :953   LwQ        : 74   Mean   : 443.6   LwQ        :  46
##  No Basement: 38   No Basement: 37   3rd Qu.: 712.2   No Basement:  38
##                    Rec        :133   Max.   :5644.0   Rec        :  54
##                    Unf        :430                    Unf        :1256
##    BsmtFinSF2        BsmtUnfSF       TotalBsmtSF      Heating     HeatingQC
##  Min.   :   0.00   Min.   :   0.0   Min.   :   0.0   Floor:   1   Ex:741
##  1st Qu.:   0.00   1st Qu.: 223.0   1st Qu.: 795.8   GasA :1428   Fa: 49
##  Median :   0.00   Median : 477.5   Median : 991.5   GasW :  18   Gd:241
##  Mean   :  46.55   Mean   : 567.2   Mean   :1057.4   Grav :   7   Po:  1
##  3rd Qu.:   0.00   3rd Qu.: 808.0   3rd Qu.:1298.2   OthW :   2   TA:428
##  Max.   :1474.00   Max.   :2336.0   Max.   :6110.0   Wall :   4
##
##  CentralAir Electrical     X1stFlrSF      X2ndFlrSF     LowQualFinSF
##  N:  95     FuseA:  94   Min.   : 334   Min.   :   0   Min.   :  0.000
##  Y:1365     FuseF:  27   1st Qu.: 882   1st Qu.:   0   1st Qu.:  0.000
##             FuseP:   3   Median :1087   Median :   0   Median :  0.000
##             Mix  :   1   Mean   :1163   Mean   : 347   Mean   :  5.845
##             SBrkr:1335   3rd Qu.:1391   3rd Qu.: 728   3rd Qu.:  0.000
##                          Max.   :4692   Max.   :2065   Max.   :572.000
##
##    GrLivArea     BsmtFullBath     BsmtHalfBath        FullBath
##  Min.   : 334   Min.   :0.0000   Min.   :0.00000   Min.   :0.000
##  1st Qu.:1130   1st Qu.:0.0000   1st Qu.:0.00000   1st Qu.:1.000
##  Median :1464   Median :0.0000   Median :0.00000   Median :2.000
##  Mean   :1515   Mean   :0.4253   Mean   :0.05753   Mean   :1.565
##  3rd Qu.:1777   3rd Qu.:1.0000   3rd Qu.:0.00000   3rd Qu.:2.000
##  Max.   :5642   Max.   :3.0000   Max.   :2.00000   Max.   :3.000
##
##     HalfBath       BedroomAbvGr    KitchenAbvGr   KitchenQual  TotRmsAbvGrd
##  Min.   :0.0000   Min.   :0.000   Min.   :0.000   Ex:100      Min.   : 2.000
##  1st Qu.:0.0000   1st Qu.:2.000   1st Qu.:1.000   Fa: 39      1st Qu.: 5.000
##  Median :0.0000   Median :3.000   Median :1.000   Gd:586      Median : 6.000
##  Mean   :0.3829   Mean   :2.866   Mean   :1.047   TA:735      Mean   : 6.518
##  3rd Qu.:1.0000   3rd Qu.:3.000   3rd Qu.:1.000               3rd Qu.: 7.000
##  Max.   :2.0000   Max.   :8.000   Max.   :3.000               Max.   :14.000
##
##  Functional    Fireplaces          FireplaceQu      GarageType   GarageYrBlt
##  Maj1:  14   Min.   :0.000   Ex          : 24   2Types   :  6   Min.   :1900
##  Maj2:   5   1st Qu.:0.000   Fa          : 33   Attchd   :870   1st Qu.:1961
##  Min1:  31   Median :1.000   Gd          :380   Basment  : 19   Median :1980
##  Min2:  34   Mean   :0.613   No Fireplace:690   BuiltIn  : 88   Mean   :1979
##  Mod :  15   3rd Qu.:1.000   Po          : 20   CarPort  :  9   3rd Qu.:2002
##  Sev :   1   Max.   :3.000   TA          :313   Detchd   :387   Max.   :2010
##  Typ :1360                                      No Garage: 81   NA's   :81
##     GarageFinish   GarageCars      GarageArea         GarageQual
```

```
##  Fin      :352   Min.   :0.000   Min.   :   0.0   Ex       :   3
##  No Garage: 81   1st Qu.:1.000   1st Qu.: 334.5   Fa       :  48
##  RFn      :422   Median :2.000   Median : 480.0   Gd       :  14
##  Unf      :605   Mean   :1.767   Mean   : 473.0   No Garage:  81
##                  3rd Qu.:2.000   3rd Qu.: 576.0   Po       :   3
##                  Max.   :4.000   Max.   :1418.0   TA       :1311
##
##      GarageCond   PavedDrive   WoodDeckSF      OpenPorchSF     EnclosedPorch
##  Ex       :   2   N:  90     Min.   :  0.00   Min.   :  0.00   Min.   :  0.00
##  Fa       :  35   P:  30     1st Qu.:  0.00   1st Qu.:  0.00   1st Qu.:  0.00
##  Gd       :   9   Y:1340     Median :  0.00   Median : 25.00   Median :  0.00
##  No Garage:  81              Mean   : 94.24   Mean   : 46.66   Mean   : 21.95
##  Po       :   7              3rd Qu.:168.00   3rd Qu.: 68.00   3rd Qu.:  0.00
##  TA       :1326              Max.   :857.00   Max.   :547.00   Max.   :552.00
##
##    X3SsnPorch      ScreenPorch        PoolArea           PoolQC
##  Min.   :  0.00   Min.   :  0.00   Min.   :  0.000   Ex     :   2
##  1st Qu.:  0.00   1st Qu.:  0.00   1st Qu.:  0.000   Fa     :   2
##  Median :  0.00   Median :  0.00   Median :  0.000   Gd     :   3
##  Mean   :  3.41   Mean   : 15.06   Mean   :  2.759   No Pool:1453
##  3rd Qu.:  0.00   3rd Qu.:  0.00   3rd Qu.:  0.000
##  Max.   :508.00   Max.   :480.00   Max.   :738.000
##
##       Fence               MiscFeature      MiscVal             MoSold
##  GdPrv   :  59   Gar2           :   2   Min.   :    0.00   6      :253
##  GdWo    :  54   No Misc Feature:1406   1st Qu.:    0.00   7      :234
##  MnPrv   : 157   Othr           :   2   Median :    0.00   5      :204
##  MnWw    :  11   Shed           :  49   Mean   :   43.49   4      :141
##  No Fence:1179   TenC           :   1   3rd Qu.:    0.00   8      :122
##                                          Max.   :15500.00   3      :106
##                                                             (Other):400
##      YrSold        SaleType    SaleCondition    SalePrice      HasGarage
##  Min.   :2006   WD     :1267   Abnorml: 101   Min.   : 34900   No :  81
##  1st Qu.:2007   New    : 122   AdjLand:   4   1st Qu.:129975   Yes:1379
##  Median :2008   COD    :  43   Alloca :  12   Median :163000
##  Mean   :2008   ConLD  :   9   Family :  20   Mean   :180921
##  3rd Qu.:2009   ConLI  :   5   Normal :1198   3rd Qu.:214000
##  Max.   :2010   ConLw  :   5   Partial: 125   Max.   :755000
##                 (Other):   9
```

This code will all the character variables along with some numeric variables to factors.

- `OverallQual` is being converted because the numeric ratings have a categorical values with them. i.e 10 is “very excellent” while 9 is “excellent”. Thus to capture these categorical values, `OverallQual` is being converted to a factor. (This info is from the data dictionary on Kaggle.)
    - Overall Quality rates the overall material and finish of the house.

- `OverallCond` also reflect this with the numeric rating containing categorical information. For instance, a 9 is “excellent” while an 8 is “very good”. (This info is from the data dictionary on Kaggle.)
    - Overall Cond rates the overall condition of the house.
- `MSSubclass` also contains categorical information which has been represented as a numeric variable. Thus, to capture this information, this variable will also be converted to a factor. For instance, “20” represents a “1-STORY 1946 & NEWER ALL STYLES”
    - Identifies the type of dwelling involved in a sale.
- `MoSold` just represents the month that a house was sold. This should be a categorical variable, it makes no sense to calculate the “median” month, etc. There additionally is no meaningful numeric diference between “January” and “February” for instance. These are just names of the months which this variable represents. It has just been created as a numeric column.

Outliers Analysis

To remove the outliers, the LOF (Local Outlier Factor) algorithm will be utilized. This approach utilizes LOOP (Local Outlier Probability) where it utilizes the LOF Scores to calculate the probability of an observation being an outlier.

Afterwards any observations that have a greater than 75% probability of being an outlier will be removed from the dataset.

```
numeric_columns <- sapply(house_prices, is.numeric)

house_clean_numeric <- house_prices[, numeric_columns] # subset train_clean using on
ly numeric cols  (33 cols)

house_clean_numeric <- house_clean_numeric %>% select(-GarageYrBlt)
# Can't have missing values

lof<- sapply(5:10, function(k) LOF(house_clean_numeric,k))
```

This above code calculates 6 LOF Values for each row in the dataset. For instance, for row 1, the chunk will calculate the LOF for 5 neighbors, 6 neighbors, etc. up to 10 neighbors.

```
#calcuate median lof of different neighborhood sizes.

lof<-apply(lof,1,median)
```

The above code then calculates the median LOF score for each row from the 6 LOF scores previously generated for each row.

```
loop<- sapply(5:10, function(k) LOOP(house_clean_numeric,k))
loop<-apply(loop,1,median)

# Remove observation with LOOP more than .75
# hpc = house_prices_clean
hpc <- house_prices[loop <= 0.75, ]
```

A potential issue however that was present in the last approach with utilizing the LOF Algorithm was that the threshold was a little arbitrarily decided. A potential work around to this issue is to utilize LOOP. LOOP utilizes LOF Scores to calculate the probability of an observation being an outlier. Afterwards, any LOOP values that are greater than .75 would be filtered out from the dataset.

This method is more robust then other LOF approaches like arbitrarily selecting threshold based on some guidelines. Additionally, this method calculates the median LOF Score across different number of neighbors which gives a more robust view of an observation's Local Reachabilty Density and LOF score. (This is because we are taking the median of several LOF Scores as opposed to just one LOF score like in the previous method.)

There is the issue of selecting which probabilty threshold should be selected for determining if something is an outlier. However, the median LOF Scores which are used to calculate the LOOP should provide a better view than just using LOF Scores from a fixed neighbor number. (i.e calculating the LOF Scores from just 6 neighbors for every observation.)

```
row_reduction_02 <- ((1460 - nrow(hpc))/1460) * 100

round(row_reduction_02,2)
```

```
## [1] 1.44
```

```
nrow(hpc)
```

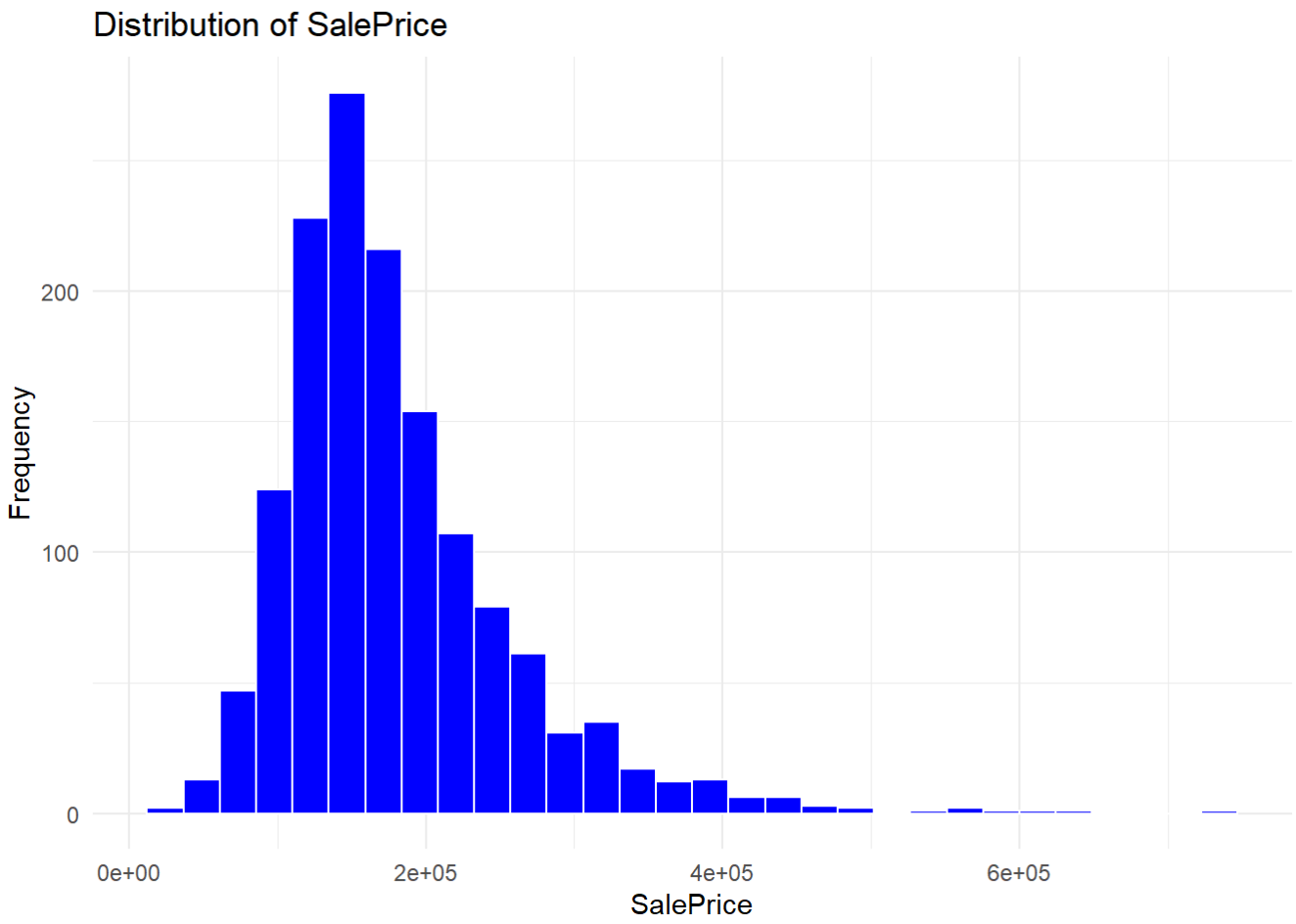
```
## [1] 1439
```

```
ncol(hpc)
```

```
## [1] 81
```

Visualize the distribution of the SalePrice

```
ggplot(hpc, aes(SalePrice)) +
  geom_histogram(bins = 30, fill = "blue", color = "white") +
  theme_minimal() +
  labs(title = "Distribution of SalePrice", x = "SalePrice", y = "Frequency")
```



Penalized Regression

Partition the Dataset

```
#Ridge Regression cannot handle missing values, there is one column yr built which h
as missing values, so this colulmn will be removed.
hpc_01 <- hpc %>%
  select(-GarageYrBlt)

set.seed(100) # Set Seed for reproducibility
index_row_number_split <- createDataPartition(hpc_01$SalePrice,p=.8,list = FALSE) #S
plit into 80/20 with 80 in split and 20 in train
```

```
# Subset train using the index and assign it to train_fold.
train_fold <- hpc_01[index_row_number_split, ]

# Subset all remaining rows into the validation fold.
test <- hpc_01[-index_row_number_split, ]
```

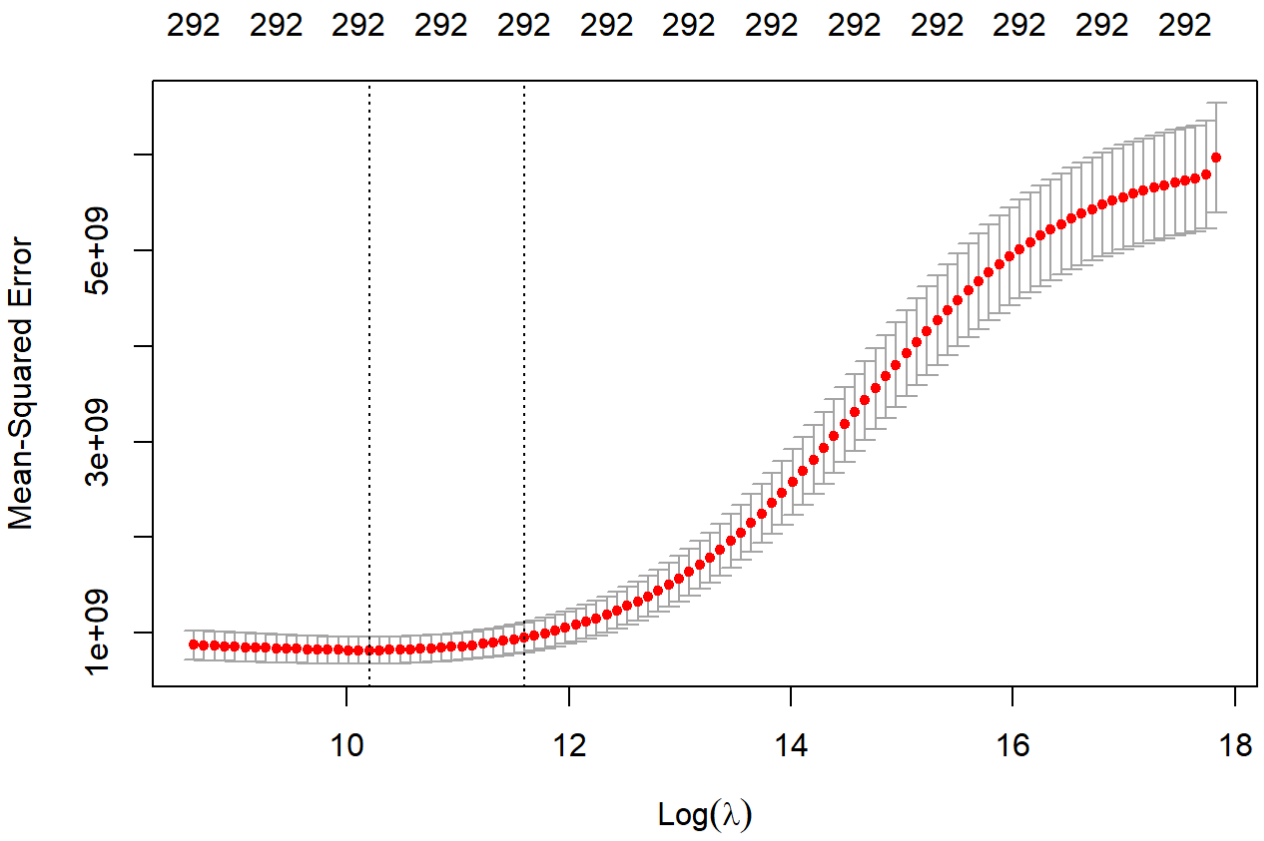
K-Fold Validation Penalized Regression Function

```
perform_kfold_cv <- function(X, y, alpha, lambda, k, seed) {
  # Ensure reproducibility
  set.seed(seed)

  # Create k folds
  folds <- sample(1:k, size = nrow(X), replace = TRUE)

  # Initialize vectors to store RMSE and R^2 for each fold
  cv_rmse <- numeric(k)
  cv_r2 <- numeric(k)

  for (i in 1:k) {
    # Split data into training and validation sets
    train_idx <- which(folds != i)
    test_idx <- which(folds == i)

    X_train <- X[train_idx, , drop = FALSE]  # Subset predictors for training
    y_train <- y[train_idx]                  # Subset target for training

    X_test <- X[test_idx, , drop = FALSE]    # Subset predictors for testing
    y_test <- y[test_idx]                    # Subset target for testing

    # Train the model on the training fold
    model <- glmnet(X_train, y_train, alpha = alpha, lambda = lambda)

    # Predict on the validation fold
    predictions <- predict(model, newx = X_test)

    # Calculate RMSE for this fold
    cv_rmse[i] <- sqrt(mean((y_test - predictions)^2))

    # Calculate R^2 for this fold
    ss_total <- sum((y_test - mean(y_test))^2)
    ss_residual <- sum((y_test - predictions)^2)
    cv_r2[i] <- 1 - (ss_residual / ss_total)
  }

  # Calculate average RMSE and R^2 across all folds
  mean_rmse <- mean(cv_rmse)
  mean_r2 <- mean(cv_r2)

  # Print messages with the metrics
  cat("Average RMSE across ", k, "-fold cross-validation: ", round(mean_rmse, 2),
"\n")

  cat("Average R^2 across ", k, "-fold cross-validation: ", round(mean_r2, 2), "\n")
```

```
}
```

Ridge Regression

```
X <- model.matrix(SalePrice ~ ., train_fold)
y <- train_fold$SalePrice

cv_ridge <- cv.glmnet(X, y, alpha = 0)  # alpha = 0 for Ridge
plot(cv_ridge)
```



Introduced L2 regularization to shrink coefficients of less important features.

The Optimal λ (left vertical dashed line) is the first vertical dashed line (around log(λ) ≈ 10). This marks the λ value that minimizes the Mean Squared Error. This value of λ balances the trade-off between bias and variance, giving the best model performance.

High MSE after log(λ) ≈ 12: As λ increases beyond the optimal point, the MSE starts increasing, indicating that the model is becoming overly regularized, leading to underfitting.

The plot shows that the model is performing well in finding an optimal λ that minimizes the prediction error. The region around log(λ) ≈ 10 is the best choice for λ, and choosing a value in this region should provide a good balance between bias and variance.

```
ridge_best_lambda <- cv_ridge$lambda.min
ridge_model <- glmnet(X, y, alpha = 0, lambda = ridge_best_lambda)

# Displaying ridge model
coef(ridge_model)
```

```
## 299 x 1 sparse Matrix of class "dgCMatrix"
##                                       s0
## (Intercept)                -7.048801e+05
## (Intercept)                 .
## MSSubClass30               -4.524746e+03
## MSSubClass40                4.758746e+03
## MSSubClass45                5.291890e+02
## MSSubClass50                1.305033e+03
## MSSubClass60                2.927752e+03
## MSSubClass70                2.014644e+03
## MSSubClass75                7.023079e+03
## MSSubClass80               -2.323323e+03
## MSSubClass85               -5.388790e+03
## MSSubClass90               -5.246218e+03
## MSSubClass120              -6.774750e+03
## MSSubClass160              -7.137595e+03
## MSSubClass180              -1.779061e+03
## MSSubClass190              -4.336505e+03
## MSZoningFV                  6.717653e+03
## MSZoningRH                  1.673252e+03
## MSZoningRL                  2.841563e+03
## MSZoningRM                 -1.838751e+03
## LotFrontage                 8.365240e+01
## LotArea                     3.665320e-01
## StreetPave                  1.878945e+04
## AlleyNo Alley               2.012051e+03
## AlleyPave                   2.596838e+03
## LotShapeIR2                 1.259002e+04
## LotShapeIR3                -5.757907e+02
## LotShapeReg                 1.057638e+02
## LandContourHLS              8.380324e+03
## LandContourLow             -3.846058e+03
## LandContourLvl              2.181130e+03
## UtilitiesNoSeWa             .
## LotConfigCulDSac            6.917146e+03
## LotConfigFR2               -5.858812e+03
## LotConfigFR3               -2.203019e+03
## LotConfigInside            -1.456972e+03
## LandSlopeMod                3.769061e+03
## LandSlopeSev               -1.022605e+04
## NeighborhoodBlueste         4.845746e+03
## NeighborhoodBrDale          4.776460e+03
## NeighborhoodBrkSide         4.511228e+03
## NeighborhoodClearCr        -3.028329e+03
## NeighborhoodCollgCr        -1.472498e+03
## NeighborhoodCrawfor         1.729338e+04
## NeighborhoodEdwards        -9.309970e+03
## NeighborhoodGilbert        -7.210193e+03
```

```
## NeighborhoodIDOTRR         -7.214735e+03
## NeighborhoodMeadowV        -1.402649e+04
## NeighborhoodMitchel        -8.944023e+03
## NeighborhoodNAmes          -4.685132e+03
## NeighborhoodNoRidge         2.146917e+04
## NeighborhoodNPkVill         6.784600e+03
## NeighborhoodNridgHt         2.122020e+04
## NeighborhoodNWAmes         -4.643583e+03
## NeighborhoodOldTown        -4.760744e+03
## NeighborhoodSawyer         -3.334898e+03
## NeighborhoodSawyerW         2.432851e+03
## NeighborhoodSomerst         4.318726e+03
## NeighborhoodStoneBr         2.586454e+04
## NeighborhoodSWISU          -4.548392e+03
## NeighborhoodTimber         -2.206424e+03
## NeighborhoodVeenker         1.127830e+04
## Condition1Feedr            -4.672129e+03
## Condition1Norm              4.260865e+03
## Condition1PosA              6.866551e+03
## Condition1PosN             -2.415803e+03
## Condition1RRAe             -9.685107e+03
## Condition1RRAn              2.911938e+03
## Condition1RRNe             -1.326068e+04
## Condition1RRNn             -1.038097e+03
## Condition2Feedr             5.250530e+03
## Condition2Norm              1.652566e+04
## Condition2PosA              1.612042e+04
## Condition2PosN             -1.246026e+05
## Condition2RRAe                      .
## Condition2RRAn                      .
## Condition2RRNn              2.075150e+04
## BldgType2fmCon             -2.681340e+03
## BldgTypeDuplex             -5.327972e+03
## BldgTypeTwnhs              -9.521201e+03
## BldgTypeTwnhsE             -5.588924e+03
## HouseStyle1.5Unf            4.456456e+03
## HouseStyle1Story           -1.668030e+03
## HouseStyle2.5Fin            2.336911e+03
## HouseStyle2.5Unf           -2.271666e+03
## HouseStyle2Story            8.643858e+02
## HouseStyleSFoyer            2.998676e+02
## HouseStyleSLvl             -8.315148e+02
## OverallQual2               -9.428927e+03
## OverallQual3               -1.091875e+04
## OverallQual4               -6.990950e+03
## OverallQual5               -6.272664e+03
## OverallQual6               -6.242941e+03
## OverallQual7               -9.054359e+02
## OverallQual8                1.404145e+04
```

```
## OverallQual9               4.286016e+04
## OverallQual10              6.062094e+04
## OverallCond2              -2.526999e+03
## OverallCond3              -1.288505e+04
## OverallCond4              -7.700983e+03
## OverallCond5              -1.986099e+03
## OverallCond6              -9.704480e+02
## OverallCond7               4.879921e+03
## OverallCond8               5.571620e+03
## OverallCond9               1.680736e+04
## YearBuilt                  9.229484e+01
## YearRemodAdd               1.522038e+02
## RoofStyleGable            -2.303654e+03
## RoofStyleGambrel          -2.601688e+03
## RoofStyleHip               2.210218e+03
## RoofStyleMansard           9.406002e+03
## RoofStyleShed              .
## RoofMatlCompShg            9.758286e+02
## RoofMatlMembran            3.245818e+04
## RoofMatlMetal              1.756806e+04
## RoofMatlRoll              -7.645965e+02
## RoofMatlTar&Grv           -2.156397e+04
## RoofMatlWdShake           -1.129245e+03
## RoofMatlWdShngl            2.291054e+04
## Exterior1stAsphShn        -2.805312e+03
## Exterior1stBrkComm         2.909378e+04
## Exterior1stBrkFace         1.334368e+04
## Exterior1stCBlock         -7.021307e+03
## Exterior1stCemntBd         4.640552e+03
## Exterior1stHdBoard        -2.197510e+03
## Exterior1stImStucc        -1.261989e+04
## Exterior1stMetalSd        -7.329715e+02
## Exterior1stPlywood        -2.583790e+03
## Exterior1stStone           4.144398e+03
## Exterior1stStucco          6.176661e+03
## Exterior1stVinylSd         5.956036e+02
## Exterior1stWd Sdng        -1.614967e+03
## Exterior1stWdShing        -1.515008e+03
## Exterior2ndAsphShn         3.256353e+03
## Exterior2ndBrk Cmn         4.060339e+03
## Exterior2ndBrkFace        -9.715601e+02
## Exterior2ndCBlock         -6.951108e+03
## Exterior2ndCmentBd         2.220537e+03
## Exterior2ndHdBoard        -1.818883e+03
## Exterior2ndImStucc         1.592718e+04
## Exterior2ndMetalSd        -1.161963e+03
## Exterior2ndOther          -5.974209e+03
## Exterior2ndPlywood        -8.722759e+02
## Exterior2ndStone           1.947828e+03
```

```
## Exterior2ndStucco          -2.364242e+02
## Exterior2ndVinylSd          1.535974e+03
## Exterior2ndWd Sdng          8.425703e+01
## Exterior2ndWd Shng         -2.764284e+03
## MasVnrTypeBrkFace          -5.189594e+02
## MasVnrTypeNone             -1.156135e+02
## MasVnrTypeStone             5.734858e+03
## MasVnrArea                  1.460760e+01
## ExterQualFa                -4.833428e+03
## ExterQualGd                -1.882457e+03
## ExterQualTA                -5.947652e+03
## ExterCondFa                -3.463139e+03
## ExterCondGd                -2.431772e+02
## ExterCondPo                -2.233228e+04
## ExterCondTA                 6.447731e+02
## FoundationCBlock           -6.680551e+02
## FoundationPConc             2.804193e+03
## FoundationSlab             -1.016013e+03
## FoundationStone             8.868944e+02
## FoundationWood             -1.348656e+04
## BsmtQualFa                 -3.755276e+03
## BsmtQualGd                 -5.340616e+03
## BsmtQualNo Basement        -7.619773e+02
## BsmtQualTA                 -5.049731e+03
## BsmtCondGd                  3.096676e+03
## BsmtCondNo Basement        -7.408046e+02
## BsmtCondPo                 -8.730560e+03
## BsmtCondTA                  1.744647e+03
## BsmtExposureGd              1.060402e+04
## BsmtExposureMn             -6.249868e+02
## BsmtExposureNo             -4.117151e+03
## BsmtExposureNo Basement    -1.282016e+03
## BsmtFinType1BLQ             2.934751e+02
## BsmtFinType1GLQ             5.682455e+03
## BsmtFinType1LwQ            -2.096754e+03
## BsmtFinType1No Basement    -6.412477e+02
## BsmtFinType1Rec            -1.716493e+03
## BsmtFinType1Unf            -1.738915e+03
## BsmtFinSF1                  1.182911e+01
## BsmtFinType2BLQ            -3.394467e+03
## BsmtFinType2GLQ             5.473120e+03
## BsmtFinType2LwQ            -3.412251e+02
## BsmtFinType2No Basement    -1.814111e+03
## BsmtFinType2Rec            -4.926058e+03
## BsmtFinType2Unf            -1.102880e+03
## BsmtFinSF2                  5.302920e+00
## BsmtUnfSF                   9.208446e-01
## TotalBsmtSF                 1.424318e+01
## HeatingGasA                 2.301734e+03
```

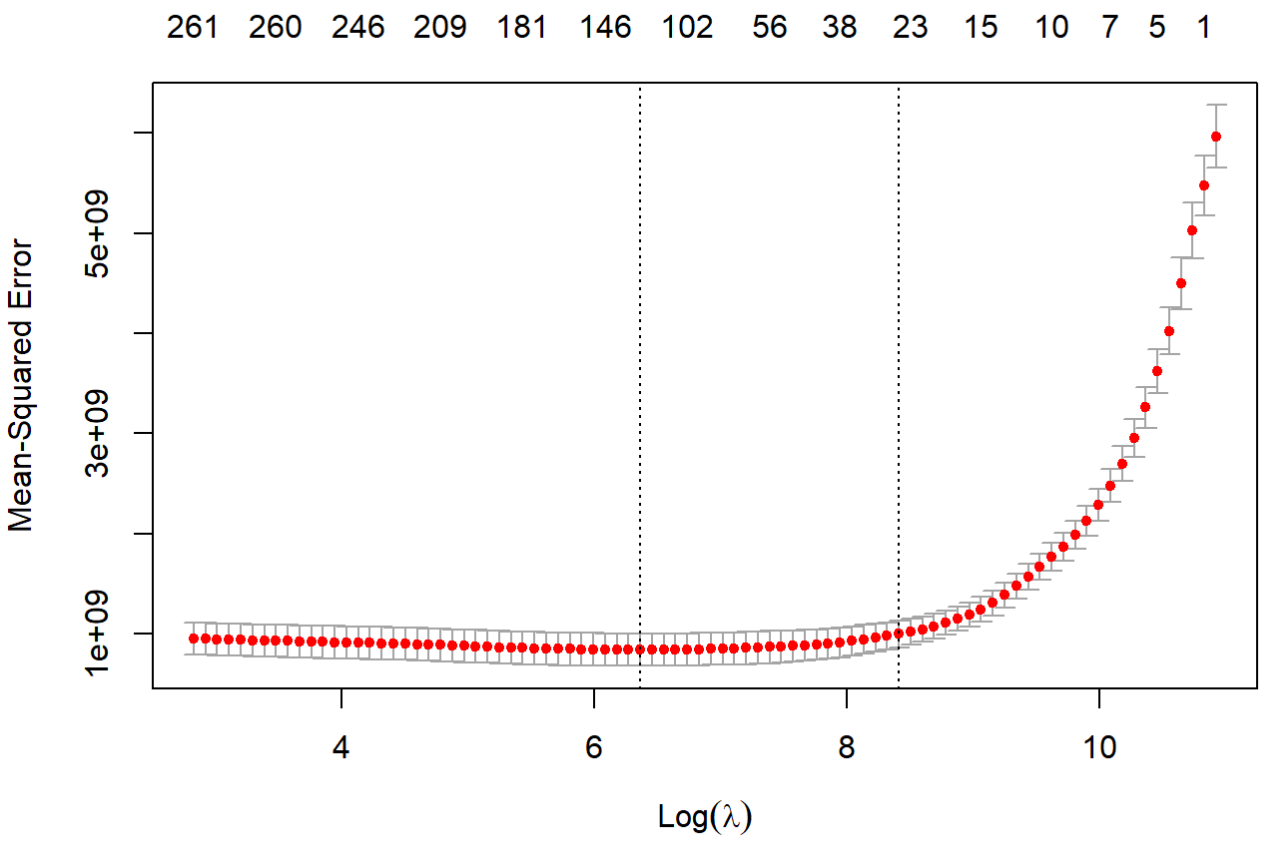
```
## HeatingGasW                -1.929519e+03
## HeatingGrav                -2.194533e+03
## HeatingOthW                -1.107315e+04
## HeatingWall                 1.560434e+03
## HeatingQCFa                -3.750119e+03
## HeatingQCGd                -2.024854e+03
## HeatingQCPo                -9.234660e+01
## HeatingQCTA                -3.190513e+03
## CentralAirY                 2.985714e+03
## ElectricalFuseF            -5.040398e+02
## ElectricalFuseP             2.365897e+03
## ElectricalMix               7.493275e+03
## ElectricalSBrkr            -1.276957e+03
## X1stFlrSF                   1.658370e+01
## X2ndFlrSF                   9.576879e+00
## LowQualFinSF                1.184253e+00
## GrLivArea                   1.581809e+01
## BsmtFullBath                3.187538e+03
## BsmtHalfBath               -5.360983e+02
## FullBath                    6.653688e+03
## HalfBath                    4.572860e+03
## BedroomAbvGr               -1.850297e+02
## KitchenAbvGr               -1.191390e+04
## KitchenQualFa              -2.852901e+03
## KitchenQualGd              -5.350522e+03
## KitchenQualTA              -6.270400e+03
## TotRmsAbvGrd                2.556220e+03
## FunctionalMaj2             -9.719947e+03
## FunctionalMin1             -6.303992e+03
## FunctionalMin2             -5.563107e+03
## FunctionalMod               4.756768e+03
## FunctionalSev              -1.721760e+04
## FunctionalTyp               5.800306e+03
## Fireplaces                  4.571451e+03
## FireplaceQuFa              -5.559167e+03
## FireplaceQuGd               3.799035e+02
## FireplaceQuNo Fireplace    -2.096567e+03
## FireplaceQuPo              -4.611682e+03
## FireplaceQuTA              -1.229670e+02
## GarageTypeAttchd            5.106904e+02
## GarageTypeBasment          -1.981661e+03
## GarageTypeBuiltIn           6.367586e+03
## GarageTypeCarPort          -6.293017e+03
## GarageTypeDetchd           -1.178465e+03
## GarageTypeNo Garage        -8.292684e+01
## GarageFinishNo Garage      -1.704701e+02
## GarageFinishRFn            -2.704367e+03
## GarageFinishUnf            -2.215247e+03
## GarageCars                  4.189752e+03
```

```
## GarageArea                   2.094414e+01
## GarageQualFa                -3.176678e+03
## GarageQualGd                 5.848220e+03
## GarageQualNo Garage         -4.874632e+02
## GarageQualPo                -2.917072e+02
## GarageQualTA                -2.081592e+03
## GarageCondFa                -3.730582e+03
## GarageCondGd                -7.769072e+03
## GarageCondNo Garage         -5.297617e+02
## GarageCondPo                -3.259192e+03
## GarageCondTA                 1.121989e+03
## PavedDriveP                 -2.911298e+03
## PavedDriveY                  2.529456e+03
## WoodDeckSF                   1.355827e+01
## OpenPorchSF                  2.024327e+01
## EnclosedPorch                1.629462e+01
## X3SsnPorch                   3.356117e+01
## ScreenPorch                  4.853787e+01
## PoolArea                     1.035428e+02
## PoolQCFa                    -8.623943e+04
## PoolQCGd                    -6.951304e+04
## PoolQCNo Pool               -7.157835e+04
## FenceGdWo                   -2.103947e+03
## FenceMnPrv                   1.062507e+03
## FenceMnWw                   -5.369729e+03
## FenceNo Fence                6.502624e+02
## MiscFeatureNo Misc Feature   1.459262e+02
## MiscFeatureOthr             -5.045983e+03
## MiscFeatureShed              1.097320e+02
## MiscFeatureTenC              .
## MiscVal                      1.452252e+00
## MoSold2                     -3.037364e+03
## MoSold3                     -8.471885e+02
## MoSold4                     -9.089364e+01
## MoSold5                      1.494940e+03
## MoSold6                      6.293581e+02
## MoSold7                      2.713965e+02
## MoSold8                     -1.176798e+03
## MoSold9                      7.858866e+02
## MoSold10                    -3.327987e+03
## MoSold11                    -3.611187e+02
## MoSold12                    -2.856278e+03
## YrSold                       1.577045e+02
## SaleTypeCon                  3.001278e+04
## SaleTypeConLD                1.126695e+03
## SaleTypeConLI                4.491595e+02
## SaleTypeConLw               -4.998485e+03
## SaleTypeCWD                  2.638532e+03
## SaleTypeNew                  8.869680e+03
```

```
## SaleTypeOth                2.995639e+03
## SaleTypeWD                -2.546049e+03
## SaleConditionAdjLand       1.812957e+04
## SaleConditionAlloca        2.166379e+03
## SaleConditionFamily       -4.057335e+03
## SaleConditionNormal        2.900012e+03
## SaleConditionPartial       7.897406e+03
## HasGarageYes               4.073642e+02
```

Lasso Regression

```
# Lasso Regression
cv_lasso <- cv.glmnet(X, y, alpha = 1)  # alpha = 1 for Lasso
plot(cv_lasso)
```



Applied L1 regularization to perform both coefficient shrinkage and variable selection.

The Lasso Regression performs best with a λ value around log(λ) ≈ 6. Beyond this, as λ increases, the model complexity decreases, but at the cost of predictive performance. (Housing price predictions are less accurate). Beyond that, the plot suggests the model is functioning well in terms of balancing sparsity (feature selection) and prediction accuracy.

```
lasso_best_lambda <- cv_lasso$lambda.min
lasso_model <- glmnet(X, y, alpha = 1, lambda = lasso_best_lambda)

# Displaying Lasso Model
coef(lasso_model)
```

```
## 299 x 1 sparse Matrix of class "dgCMatrix"
##                                       s0
## (Intercept)                -8.162912e+05
## (Intercept)                 .
## MSSubClass30                .
## MSSubClass40                .
## MSSubClass45                .
## MSSubClass50                .
## MSSubClass60                .
## MSSubClass70                .
## MSSubClass75                .
## MSSubClass80                .
## MSSubClass85                .
## MSSubClass90               -1.534618e+03
## MSSubClass120              -6.232845e+03
## MSSubClass160              -7.002863e+03
## MSSubClass180               .
## MSSubClass190               .
## MSZoningFV                  2.516814e+03
## MSZoningRH                  .
## MSZoningRL                  7.050291e+02
## MSZoningRM                  .
## LotFrontage                 4.650837e+01
## LotArea                     3.238795e-01
## StreetPave                  1.547816e+04
## AlleyNo Alley               .
## AlleyPave                   .
## LotShapeIR2                 1.225821e+04
## LotShapeIR3                 .
## LotShapeReg                 .
## LandContourHLS              2.487686e+03
## LandContourLow              .
## LandContourLvl              1.142279e+03
## UtilitiesNoSeWa             .
## LotConfigCulDSac            6.038254e+03
## LotConfigFR2               -2.556308e+03
## LotConfigFR3                .
## LotConfigInside             .
## LandSlopeMod                .
## LandSlopeSev               -3.540413e+03
## NeighborhoodBlueste         .
## NeighborhoodBrDale          .
## NeighborhoodBrkSide         6.782453e+03
## NeighborhoodClearCr         .
## NeighborhoodCollgCr         .
## NeighborhoodCrawfor         2.544409e+04
## NeighborhoodEdwards        -4.844478e+03
## NeighborhoodGilbert         .
```

```
## NeighborhoodIDOTRR         -2.553461e+03
## NeighborhoodMeadowV        -9.047150e+03
## NeighborhoodMitchel        -3.989729e+03
## NeighborhoodNAmes           .
## NeighborhoodNoRidge         1.740271e+04
## NeighborhoodNPkVill         4.311497e+03
## NeighborhoodNridgHt         2.164700e+04
## NeighborhoodNWAmes          .
## NeighborhoodOldTown        -1.306708e+03
## NeighborhoodSawyer          .
## NeighborhoodSawyerW         4.629833e+02
## NeighborhoodSomerst         5.095351e+03
## NeighborhoodStoneBr         2.513495e+04
## NeighborhoodSWISU           .
## NeighborhoodTimber          .
## NeighborhoodVeenker         8.178232e+03
## Condition1Feedr            -1.735682e+03
## Condition1Norm              4.587358e+03
## Condition1PosA              .
## Condition1PosN              .
## Condition1RRAe              .
## Condition1RRAn              .
## Condition1RRNe              .
## Condition1RRNn              .
## Condition2Feedr             .
## Condition2Norm              4.862816e+02
## Condition2PosA              .
## Condition2PosN             -2.138551e+05
## Condition2RRAe              .
## Condition2RRAn              .
## Condition2RRNn              .
## BldgType2fmCon              .
## BldgTypeDuplex             -7.112194e+02
## BldgTypeTwnhs              -8.548312e+03
## BldgTypeTwnhsE             -4.615257e+03
## HouseStyle1.5Unf            5.206698e+03
## HouseStyle1Story            .
## HouseStyle2.5Fin            .
## HouseStyle2.5Unf            .
## HouseStyle2Story            .
## HouseStyleSFoyer            .
## HouseStyleSLvl              .
## OverallQual2                .
## OverallQual3                .
## OverallQual4               -1.257041e+02
## OverallQual5                .
## OverallQual6                .
## OverallQual7                6.332025e+03
## OverallQual8                2.900630e+04
```

```
## OverallQual9                7.404587e+04
## OverallQual10               1.037461e+05
## OverallCond2               -5.450440e+03
## OverallCond3               -1.052167e+04
## OverallCond4               -6.996038e+03
## OverallCond5               -7.698144e+02
## OverallCond6                .
## OverallCond7                7.753605e+03
## OverallCond8                7.865081e+03
## OverallCond9                1.960165e+04
## YearBuilt                   3.226486e+02
## YearRemodAdd                1.692868e+02
## RoofStyleGable             -9.502739e+02
## RoofStyleGambrel            .
## RoofStyleHip                .
## RoofStyleMansard            .
## RoofStyleShed               .
## RoofMatlCompShg             .
## RoofMatlMembran             2.664912e+04
## RoofMatlMetal               .
## RoofMatlRoll                .
## RoofMatlTar&Grv            -2.099018e+04
## RoofMatlWdShake             .
## RoofMatlWdShngl             .
## Exterior1stAsphShn          .
## Exterior1stBrkComm          1.049081e+04
## Exterior1stBrkFace          1.605142e+04
## Exterior1stCBlock           .
## Exterior1stCemntBd          2.477437e+03
## Exterior1stHdBoard         -2.209259e+02
## Exterior1stImStucc          .
## Exterior1stMetalSd          .
## Exterior1stPlywood         -1.132072e+02
## Exterior1stStone            .
## Exterior1stStucco           3.192597e+03
## Exterior1stVinylSd          .
## Exterior1stWd Sdng          .
## Exterior1stWdShing          .
## Exterior2ndAsphShn          .
## Exterior2ndBrk Cmn          .
## Exterior2ndBrkFace          .
## Exterior2ndCBlock           .
## Exterior2ndCmentBd          .
## Exterior2ndHdBoard         -8.468380e+02
## Exterior2ndImStucc          2.623387e+03
## Exterior2ndMetalSd          .
## Exterior2ndOther            .
## Exterior2ndPlywood          .
## Exterior2ndStone            .
```

```
## Exterior2ndStucco           .
## Exterior2ndVinylSd          1.863761e+03
## Exterior2ndWd Sdng          .
## Exterior2ndWd Shng          .
## MasVnrTypeBrkFace           .
## MasVnrTypeNone              .
## MasVnrTypeStone             3.200150e+03
## MasVnrArea                  6.647359e+00
## ExterQualFa                -1.515629e+03
## ExterQualGd                 .
## ExterQualTA                -2.418676e+03
## ExterCondFa                -5.495837e+02
## ExterCondGd                 .
## ExterCondPo                -5.084080e+03
## ExterCondTA                 6.471448e+01
## FoundationCBlock            .
## FoundationPConc             1.912659e+03
## FoundationSlab              .
## FoundationStone             .
## FoundationWood             -1.983039e+04
## BsmtQualFa                  .
## BsmtQualGd                 -3.921595e+03
## BsmtQualNo Basement         .
## BsmtQualTA                 -2.742296e+03
## BsmtCondGd                  .
## BsmtCondNo Basement         .
## BsmtCondPo                  .
## BsmtCondTA                  .
## BsmtExposureGd              1.032441e+04
## BsmtExposureMn              .
## BsmtExposureNo             -2.784990e+03
## BsmtExposureNo Basement     .
## BsmtFinType1BLQ             .
## BsmtFinType1GLQ             3.466948e+03
## BsmtFinType1LwQ             .
## BsmtFinType1No Basement     .
## BsmtFinType1Rec            -9.196847e+02
## BsmtFinType1Unf             .
## BsmtFinSF1                  1.635661e+01
## BsmtFinType2BLQ             .
## BsmtFinType2GLQ             5.178967e+03
## BsmtFinType2LwQ             .
## BsmtFinType2No Basement     .
## BsmtFinType2Rec             .
## BsmtFinType2Unf             .
## BsmtFinSF2                  3.108778e+00
## BsmtUnfSF                   .
## TotalBsmtSF                 1.622501e+01
## HeatingGasA                 1.557778e+03
```

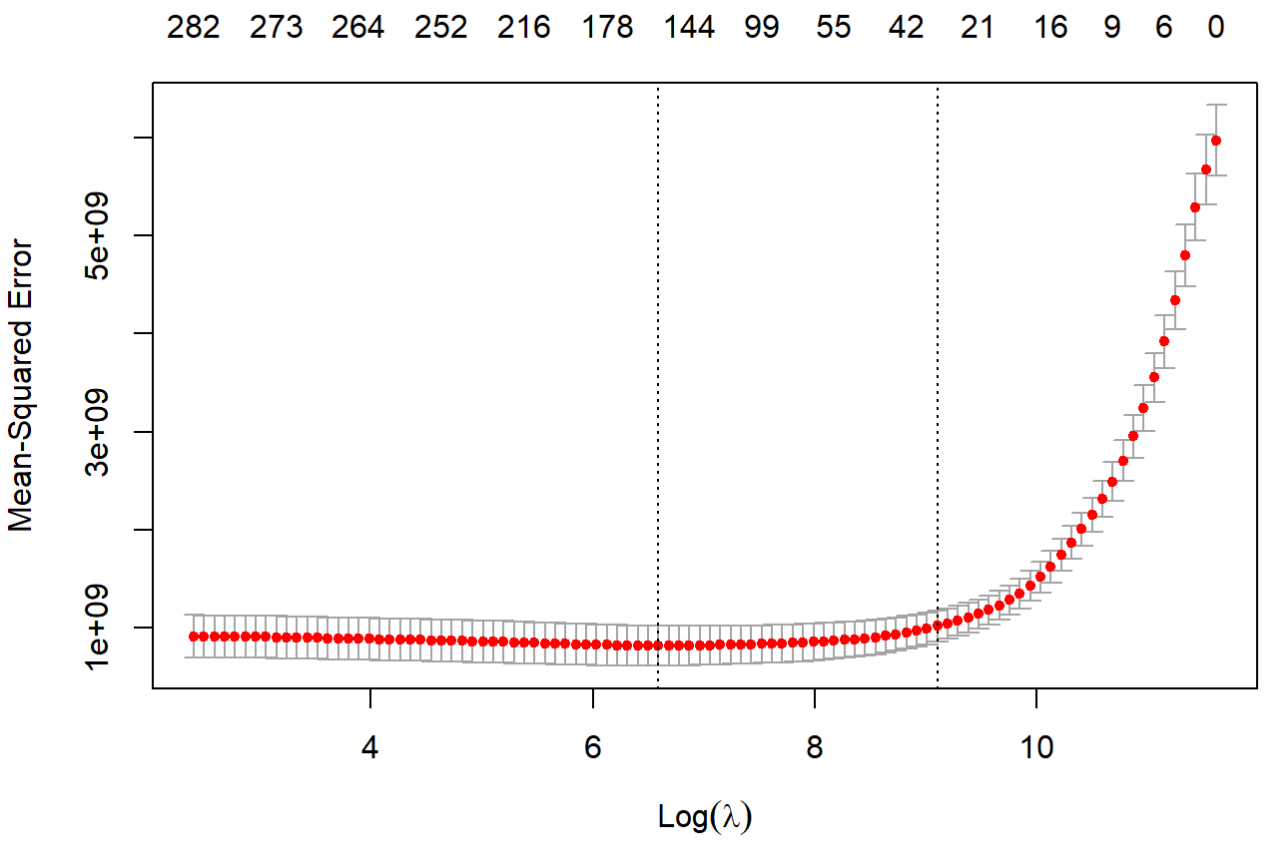
```
## HeatingGasW                .
## HeatingGrav                .
## HeatingOthW               -1.859936e+04
## HeatingWall                .
## HeatingQCFa                .
## HeatingQCGd                .
## HeatingQCPo                .
## HeatingQCTA               -1.626257e+03
## CentralAirY                .
## ElectricalFuseF            .
## ElectricalFuseP            .
## ElectricalMix              .
## ElectricalSBrkr            .
## X1stFlrSF                  .
## X2ndFlrSF                  .
## LowQualFinSF              -1.022142e+01
## GrLivArea                  5.093158e+01
## BsmtFullBath               1.219093e+03
## BsmtHalfBath               .
## FullBath                   2.773232e+03
## HalfBath                   1.690636e+03
## BedroomAbvGr              -3.356765e+02
## KitchenAbvGr              -2.119084e+04
## KitchenQualFa             -1.297457e+03
## KitchenQualGd             -3.134602e+03
## KitchenQualTA             -4.267901e+03
## TotRmsAbvGrd               3.828595e+02
## FunctionalMaj2             .
## FunctionalMin1             .
## FunctionalMin2             .
## FunctionalMod              3.142295e+03
## FunctionalSev             -1.976934e+04
## FunctionalTyp              1.181417e+04
## Fireplaces                 3.927879e+03
## FireplaceQuFa             -7.516624e+02
## FireplaceQuGd              .
## FireplaceQuNo Fireplace    .
## FireplaceQuPo              .
## FireplaceQuTA              .
## GarageTypeAttchd           .
## GarageTypeBasment          .
## GarageTypeBuiltIn          1.184709e+03
## GarageTypeCarPort         -2.435473e+03
## GarageTypeDetchd           .
## GarageTypeNo Garage        .
## GarageFinishNo Garage      .
## GarageFinishRFn            .
## GarageFinishUnf            .
## GarageCars                 1.088068e+03
```

```
## GarageArea                   2.786240e+01
## GarageQualFa                 .
## GarageQualGd                 .
## GarageQualNo Garage          .
## GarageQualPo                 .
## GarageQualTA                 .
## GarageCondFa                -8.253475e+02
## GarageCondGd                 .
## GarageCondNo Garage          .
## GarageCondPo                 .
## GarageCondTA                 .
## PavedDriveP                 -2.159400e+03
## PavedDriveY                  .
## WoodDeckSF                   8.854390e+00
## OpenPorchSF                  1.105063e+00
## EnclosedPorch                1.313898e+01
## X3SsnPorch                   2.130312e+01
## ScreenPorch                  5.100779e+01
## PoolArea                     .
## PoolQCFa                    -9.583883e+04
## PoolQCGd                    -8.028581e+04
## PoolQCNo Pool               -1.320529e+05
## FenceGdWo                   -6.441820e+02
## FenceMnPrv                   .
## FenceMnWw                   -1.669259e+03
## FenceNo Fence                .
## MiscFeatureNo Misc Feature   .
## MiscFeatureOthr              .
## MiscFeatureShed              .
## MiscFeatureTenC              .
## MiscVal                      .
## MoSold2                     -5.652320e+01
## MoSold3                      .
## MoSold4                      .
## MoSold5                      1.256979e+03
## MoSold6                      2.066227e+02
## MoSold7                      .
## MoSold8                     -5.971790e+02
## MoSold9                      .
## MoSold10                    -1.498792e+03
## MoSold11                     .
## MoSold12                    -2.171365e+03
## YrSold                       .
## SaleTypeCon                  1.666190e+04
## SaleTypeConLD                .
## SaleTypeConLI                .
## SaleTypeConLw                .
## SaleTypeCWD                  .
## SaleTypeNew                  2.224779e+04
```

```
## SaleTypeOth                .
## SaleTypeWD                 .
## SaleConditionAdjLand        8.423880e+03
## SaleConditionAlloca         .
## SaleConditionFamily        -1.612182e+02
## SaleConditionNormal         4.187940e+03
## SaleConditionPartial        .
## HasGarageYes                .
```

Elastic Net Regression

```
# Elastic Net Regression
cv_elastic_net <- cv.glmnet(X, y, alpha = 0.5)  # alpha = 0.5 for Elastic Net
plot(cv_elastic_net)
```



Combined both L1 and L2 penalties to balance the strengths of Ridge and Lasso. Cross-validation was used to select the optimal lambda (penalty) for each model, which helps control overfitting and improves generalization.

The MSE decreases initially as λincreases (moving from left to right) because the model becomes regularized, reducing overfitting.

However, asλincreases further, the MSE starts to rise again because the model becomes too simplistic, shrinking the coefficients too much (underfitting).

- The left vertical line (λ_min) represents the value of where the MSE is minimized. This is the most accurate model in terms of cross-validation error.
- The right vertical line (λ_1se) provides a more parsimonious model (fewer predictors) that still performs within one standard error of the minimum MSE. The minimum MSE occurs around Log(λ) ≈ 6.

```
elastic_net_best_lambda <- cv_elastic_net$lambda.min
elastic_net_model <- glmnet(X, y, alpha = 0.5, lambda = elastic_net_best_lambda)

# Displaying Elastic Net Model
coef(elastic_net_model)
```

```
## 299 x 1 sparse Matrix of class "dgCMatrix"
##                                       s0
## (Intercept)                -7.437394e+05
## (Intercept)                 .
## MSSubClass30                .
## MSSubClass40                .
## MSSubClass45                .
## MSSubClass50                .
## MSSubClass60                .
## MSSubClass70                .
## MSSubClass75                .
## MSSubClass80                .
## MSSubClass85                .
## MSSubClass90               -9.165137e+02
## MSSubClass120              -7.291464e+03
## MSSubClass160              -7.304550e+03
## MSSubClass180               .
## MSSubClass190               .
## MSZoningFV                  5.338380e+03
## MSZoningRH                  .
## MSZoningRL                  8.386724e+02
## MSZoningRM                  .
## LotFrontage                 3.932546e+01
## LotArea                     3.901446e-01
## StreetPave                  2.012216e+04
## AlleyNo Alley               .
## AlleyPave                   .
## LotShapeIR2                 1.374234e+04
## LotShapeIR3                 .
## LotShapeReg                 2.450765e+02
## LandContourHLS              4.860670e+03
## LandContourLow             -2.093553e+03
## LandContourLvl              2.912554e+03
## UtilitiesNoSeWa             .
## LotConfigCulDSac            6.189874e+03
## LotConfigFR2               -4.383920e+03
## LotConfigFR3                .
## LotConfigInside             .
## LandSlopeMod                2.586965e+03
## LandSlopeSev               -9.235758e+03
## NeighborhoodBlueste         .
## NeighborhoodBrDale          1.187995e+03
## NeighborhoodBrkSide         7.635836e+03
## NeighborhoodClearCr         .
## NeighborhoodCollgCr         .
## NeighborhoodCrawfor         2.616389e+04
## NeighborhoodEdwards        -6.165356e+03
## NeighborhoodGilbert        -6.640387e+02
```

```
## NeighborhoodIDOTRR         -4.590970e+03
## NeighborhoodMeadowV        -1.064478e+04
## NeighborhoodMitchel        -5.064538e+03
## NeighborhoodNAmes          -1.945933e+02
## NeighborhoodNoRidge         2.056534e+04
## NeighborhoodNPkVill         1.070664e+04
## NeighborhoodNridgHt         2.301445e+04
## NeighborhoodNWAmes         -3.798957e+02
## NeighborhoodOldTown        -2.315373e+03
## NeighborhoodSawyer          .
## NeighborhoodSawyerW         3.413764e+03
## NeighborhoodSomerst         5.328525e+03
## NeighborhoodStoneBr         2.787389e+04
## NeighborhoodSWISU           .
## NeighborhoodTimber          .
## NeighborhoodVeenker         9.654808e+03
## Condition1Feedr            -2.204814e+03
## Condition1Norm              4.860059e+03
## Condition1PosA              1.395204e+03
## Condition1PosN              .
## Condition1RRAe             -3.929362e+03
## Condition1RRAn              9.156354e+02
## Condition1RRNe             -6.122896e+03
## Condition1RRNn              .
## Condition2Feedr             .
## Condition2Norm              9.924782e+02
## Condition2PosA              .
## Condition2PosN             -2.132356e+05
## Condition2RRAe              .
## Condition2RRAn              .
## Condition2RRNn              .
## BldgType2fmCon              .
## BldgTypeDuplex             -1.531615e+03
## BldgTypeTwnhs              -1.127430e+04
## BldgTypeTwnhsE             -6.326598e+03
## HouseStyle1.5Unf            6.047048e+03
## HouseStyle1Story            .
## HouseStyle2.5Fin            .
## HouseStyle2.5Unf            .
## HouseStyle2Story            .
## HouseStyleSFoyer            .
## HouseStyleSLvl              .
## OverallQual2                .
## OverallQual3               -2.041047e+03
## OverallQual4               -1.144886e+03
## OverallQual5               -8.585703e+02
## OverallQual6                .
## OverallQual7                6.918949e+03
## OverallQual8                2.815601e+04
```

```
## OverallQual9                6.920910e+04
## OverallQual10               9.641499e+04
## OverallCond2               -4.356046e+03
## OverallCond3               -1.319948e+04
## OverallCond4               -7.417916e+03
## OverallCond5               -1.838229e+03
## OverallCond6                .
## OverallCond7                8.218489e+03
## OverallCond8                8.955176e+03
## OverallCond9                2.206665e+04
## YearBuilt                   3.195397e+02
## YearRemodAdd                1.510784e+02
## RoofStyleGable             -1.136901e+03
## RoofStyleGambrel           -6.187022e+00
## RoofStyleHip                .
## RoofStyleMansard            4.337313e+03
## RoofStyleShed               .
## RoofMatlCompShg             .
## RoofMatlMembran             3.620096e+04
## RoofMatlMetal               1.194655e+04
## RoofMatlRoll                .
## RoofMatlTar&Grv            -2.353075e+04
## RoofMatlWdShake             .
## RoofMatlWdShngl             5.518128e+03
## Exterior1stAsphShn          .
## Exterior1stBrkComm          2.843589e+04
## Exterior1stBrkFace          1.674696e+04
## Exterior1stCBlock           .
## Exterior1stCemntBd          4.001360e+03
## Exterior1stHdBoard         -1.995778e+02
## Exterior1stImStucc          .
## Exterior1stMetalSd          .
## Exterior1stPlywood         -1.172382e+03
## Exterior1stStone            .
## Exterior1stStucco           4.751751e+03
## Exterior1stVinylSd          .
## Exterior1stWd Sdng          .
## Exterior1stWdShing          .
## Exterior2ndAsphShn          .
## Exterior2ndBrk Cmn          .
## Exterior2ndBrkFace         -3.025886e+02
## Exterior2ndCBlock           .
## Exterior2ndCmentBd          .
## Exterior2ndHdBoard         -1.162012e+03
## Exterior2ndImStucc          1.264123e+03
## Exterior2ndMetalSd         -2.631628e+01
## Exterior2ndOther           -1.703850e+03
## Exterior2ndPlywood          .
## Exterior2ndStone            .
```

```
## Exterior2ndStucco           .
## Exterior2ndVinylSd          2.875471e+03
## Exterior2ndWd Sdng          .
## Exterior2ndWd Shng          .
## MasVnrTypeBrkFace           .
## MasVnrTypeNone              .
## MasVnrTypeStone             3.742786e+03
## MasVnrArea                  7.588671e+00
## ExterQualFa                -1.264761e+03
## ExterQualGd                 .
## ExterQualTA                -2.001460e+03
## ExterCondFa                -1.256163e+03
## ExterCondGd                 .
## ExterCondPo                -1.510387e+04
## ExterCondTA                 7.030358e+02
## FoundationCBlock            .
## FoundationPConc             1.833457e+03
## FoundationSlab              .
## FoundationStone             1.349989e+03
## FoundationWood             -2.293714e+04
## BsmtQualFa                 -8.209919e+02
## BsmtQualGd                 -6.274566e+03
## BsmtQualNo Basement         .
## BsmtQualTA                 -4.400787e+03
## BsmtCondGd                  .
## BsmtCondNo Basement         .
## BsmtCondPo                  .
## BsmtCondTA                  5.012391e+02
## BsmtExposureGd              1.109765e+04
## BsmtExposureMn              .
## BsmtExposureNo             -3.171278e+03
## BsmtExposureNo Basement     .
## BsmtFinType1BLQ             .
## BsmtFinType1GLQ             3.695862e+03
## BsmtFinType1LwQ            -5.664915e+02
## BsmtFinType1No Basement     .
## BsmtFinType1Rec            -1.193139e+03
## BsmtFinType1Unf             .
## BsmtFinSF1                  1.609032e+01
## BsmtFinType2BLQ             .
## BsmtFinType2GLQ             6.436826e+03
## BsmtFinType2LwQ             .
## BsmtFinType2No Basement     .
## BsmtFinType2Rec            -1.113696e+02
## BsmtFinType2Unf             .
## BsmtFinSF2                  4.832113e+00
## BsmtUnfSF                   .
## TotalBsmtSF                 1.665776e+01
## HeatingGasA                 4.263056e+03
```

```
## HeatingGasW                .
## HeatingGrav                .
## HeatingOthW               -1.503524e+04
## HeatingWall                .
## HeatingQCFa                .
## HeatingQCGd               -6.680402e+02
## HeatingQCPo                .
## HeatingQCTA               -1.575673e+03
## CentralAirY                .
## ElectricalFuseF            .
## ElectricalFuseP            .
## ElectricalMix              .
## ElectricalSBrkr           -9.702561e+02
## X1stFlrSF                  .
## X2ndFlrSF                  .
## LowQualFinSF              -1.058161e+01
## GrLivArea                  4.694861e+01
## BsmtFullBath               1.623389e+03
## BsmtHalfBath               .
## FullBath                   4.174877e+03
## HalfBath                   2.596478e+03
## BedroomAbvGr              -1.018044e+03
## KitchenAbvGr              -2.154182e+04
## KitchenQualFa             -5.252531e+03
## KitchenQualGd             -7.373259e+03
## KitchenQualTA             -7.934814e+03
## TotRmsAbvGrd               1.023985e+03
## FunctionalMaj2            -1.425459e+03
## FunctionalMin1            -5.492066e+02
## FunctionalMin2             .
## FunctionalMod              5.582584e+03
## FunctionalSev             -1.834388e+04
## FunctionalTyp              1.194496e+04
## Fireplaces                 3.689614e+03
## FireplaceQuFa             -2.399275e+03
## FireplaceQuGd              .
## FireplaceQuNo Fireplace    .
## FireplaceQuPo              .
## FireplaceQuTA              .
## GarageTypeAttchd           .
## GarageTypeBasment          .
## GarageTypeBuiltIn          2.020913e+03
## GarageTypeCarPort         -3.139626e+03
## GarageTypeDetchd           .
## GarageTypeNo Garage        .
## GarageFinishNo Garage      .
## GarageFinishRFn           -4.540892e+02
## GarageFinishUnf            .
## GarageCars                 9.348360e+02
```

```
## GarageArea                   2.674158e+01
## GarageQualFa                 .
## GarageQualGd                 .
## GarageQualNo Garage          .
## GarageQualPo                -2.761645e+02
## GarageQualTA                -1.926697e+02
## GarageCondFa                -3.156844e+03
## GarageCondGd                -1.847038e+03
## GarageCondNo Garage          .
## GarageCondPo                 .
## GarageCondTA                 .
## PavedDriveP                 -3.646253e+03
## PavedDriveY                  .
## WoodDeckSF                   1.154446e+01
## OpenPorchSF                  4.576572e+00
## EnclosedPorch                1.996056e+01
## X3SsnPorch                   3.096202e+01
## ScreenPorch                  5.370747e+01
## PoolArea                     .
## PoolQCFa                    -1.347394e+05
## PoolQCGd                    -1.210452e+05
## PoolQCNo Pool               -1.658194e+05
## FenceGdWo                   -1.459905e+03
## FenceMnPrv                   .
## FenceMnWw                   -4.795975e+03
## FenceNo Fence                .
## MiscFeatureNo Misc Feature   .
## MiscFeatureOthr              .
## MiscFeatureShed              .
## MiscFeatureTenC              .
## MiscVal                      .
## MoSold2                     -1.211213e+03
## MoSold3                     -2.140638e+01
## MoSold4                      .
## MoSold5                      1.780441e+03
## MoSold6                      7.451616e+02
## MoSold7                      .
## MoSold8                     -1.544681e+03
## MoSold9                      .
## MoSold10                    -2.468063e+03
## MoSold11                     .
## MoSold12                    -3.200571e+03
## YrSold                       .
## SaleTypeCon                  2.511955e+04
## SaleTypeConLD                .
## SaleTypeConLI                .
## SaleTypeConLw                .
## SaleTypeCWD                  .
## SaleTypeNew                  2.254071e+04
```

```
## SaleTypeOth                5.792041e+02
## SaleTypeWD                 .
## SaleConditionAdjLand       1.603107e+04
## SaleConditionAlloca        .
## SaleConditionFamily        .
## SaleConditionNormal        5.494060e+03
## SaleConditionPartial       .
## HasGarageYes               .
```

Model Evaluation (Forecasting)

```
round_metrics <- function(x) {
  return(lapply(x, function(y) {
    if (is.numeric(y)) {
      return(round(y, 2))
    } else {
      return(y)
    }
  }))
}
```

Ridge Regression Model

Train Predictions

```
# Create the same train matrix to utilize for making predictions
newx <- model.matrix(SalePrice ~ ., train_fold)

# Predict
predictions_ridge_train <- predict(ridge_model, newx = newx)

# Create List of Model Metrics for Regression
metrics_list <- c("MAE", "RMSE", "R2", "COR","MAPE","RMSPE","RAE", "RRSE")

# Generate Model Metrics
ridge_train <- mmetric(train_fold$SalePrice,predictions_ridge_train,metrics_list)

round_metrics(ridge_train)
```

```
## $MAE
## [1] 13558.55
##
## $RMSE
## [1] 21043.08
##
## $R2
## [1] 0.93
##
## $COR
## [1] 0.96
##
## $MAPE
## [1] 8
##
## $RMSPE
## [1] 1.28
##
## $RAE
## [1] 23.95
##
## $RRSE
## [1] 27.23
```

Test Predictions

```
# Create matrix for the test data, then make predictions
newx_test <- model.matrix(SalePrice ~ ., test)

# Make Predictions
predictions_ridge_test <- predict(ridge_model, newx = newx_test)

ridge_test <- mmetric(test$SalePrice,predictions_ridge_test,metrics_list)

round_metrics(ridge_test)
```

```
## $MAE
## [1] 15371.32
##
## $RMSE
## [1] 26923.69
##
## $R2
## [1] 0.89
##
## $COR
## [1] 0.94
##
## $MAPE
## [1] 8.58
##
## $RMSPE
## [1] 1.35
##
## $RAE
## [1] 27.52
##
## $RRSE
## [1] 34.27
```

The model has comparable performance against the train and test sets. RMSE increases from 21,043.08 to 26,923 dollars which is a 5,880.01 increase. R^2 decreases from .96 (train) to .89 (test) which is a 7% decrease. This is farily modes and not too large.

5-fold validation

```
perform_kfold_cv(X = X, y = y, alpha = 0,lambda = ridge_best_lambda, k = 5, seed = 100)
```

```
## Average RMSE across  5 -fold cross-validation:  28639.64
## Average R^2 across  5 -fold cross-validation:  0.86
```

Implement cross validation for more rigorous evaluation since, model performance can be dependent on a particular split. Not ah huge decrease from the test set R^2 which was .89. RMSE also only increased modestly as well.

Lasso Regression Model

Train Predictions

```
# Generate Predictions
predictions_lasso_train <- predict(lasso_model, newx = newx)

# Generate Train Metrics
lasso_train <- mmetric(train_fold$SalePrice,predictions_lasso_train,metrics_list)
round_metrics(lasso_train)
```

```
## $MAE
## [1] 13406.77
##
## $RMSE
## [1] 20597.62
##
## $R2
## [1] 0.93
##
## $COR
## [1] 0.96
##
## $MAPE
## [1] 8.01
##
## $RMSPE
## [1] 1.3
##
## $RAE
## [1] 23.68
##
## $RRSE
## [1] 26.66
```

Test Predictions

```
# Generate Predictions
predictions_lasso_test <- predict(lasso_model, newx = newx_test)

# Generate Test Metrics
lasso_test <- mmetric(test$SalePrice,predictions_lasso_test,metrics_list)
round_metrics(lasso_test)
```

```
## $MAE
## [1] 15154.86
##
## $RMSE
## [1] 26227.82
##
## $R2
## [1] 0.89
##
## $COR
## [1] 0.94
##
## $MAPE
## [1] 8.39
##
## $RMSPE
## [1] 1.25
##
## $RAE
## [1] 27.13
##
## $RRSE
## [1] 33.39
```

5-Fold Validation

```
perform_kfold_cv(X = X, y = y, alpha = 0,lambda = lasso_best_lambda, k = 5, seed = 1
00)
```

```
## Average RMSE across  5 -fold cross-validation:  30836.7
## Average R^2 across  5 -fold cross-validation:  0.84
```

Elastic Net Regression Model

Train Predictions

```
# Generate Predictions
predictions_elastic_train <- predict(elastic_net_model, newx = newx)

# Generate Train Metrics
elastic_train <- mmetric(train_fold$SalePrice,predictions_elastic_train,metrics_lis
t)
round_metrics(elastic_train)
```

```
## $MAE
## [1] 12838.99
##
## $RMSE
## [1] 19783.31
##
## $R2
## [1] 0.93
##
## $COR
## [1] 0.97
##
## $MAPE
## [1] 7.63
##
## $RMSPE
## [1] 1.23
##
## $RAE
## [1] 22.68
##
## $RRSE
## [1] 25.6
```

Test Predictions

```
# Generate Predictions
predictions_elastic_test <- predict(elastic_net_model, newx = newx_test)

# Generate Test Metrics
elastic_test <- mmetric(test$SalePrice,predictions_elastic_test,metrics_list)
round_metrics(elastic_train)
```

```
## $MAE
## [1] 12838.99
##
## $RMSE
## [1] 19783.31
##
## $R2
## [1] 0.93
##
## $COR
## [1] 0.97
##
## $MAPE
## [1] 7.63
##
## $RMSPE
## [1] 1.23
##
## $RAE
## [1] 22.68
##
## $RRSE
## [1] 25.6
```

5 Fold Validation

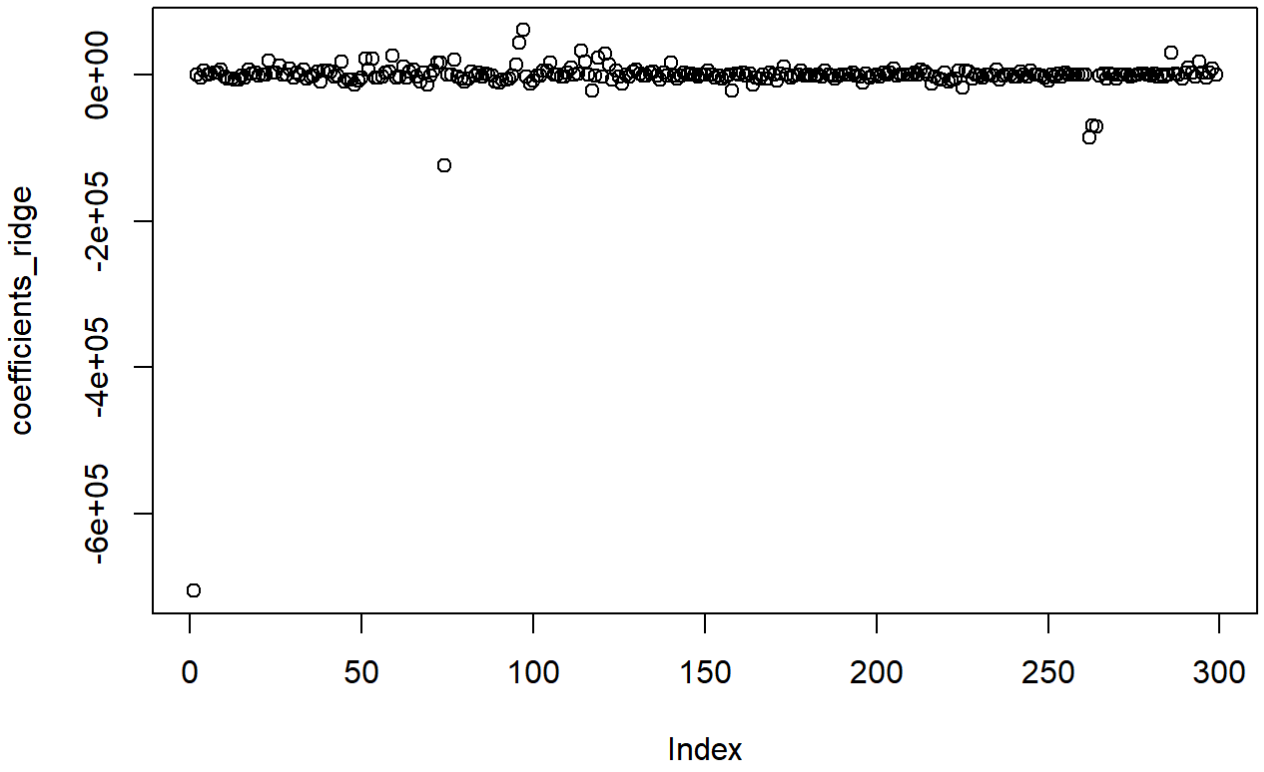
```
perform_kfold_cv(X = X, y = y, alpha = 0.5,lambda = elastic_net_best_lambda, k = 5,
seed = 100)
```

```
## Average RMSE across  5 -fold cross-validation:  29316.87
## Average R^2 across  5 -fold cross-validation:  0.85
```

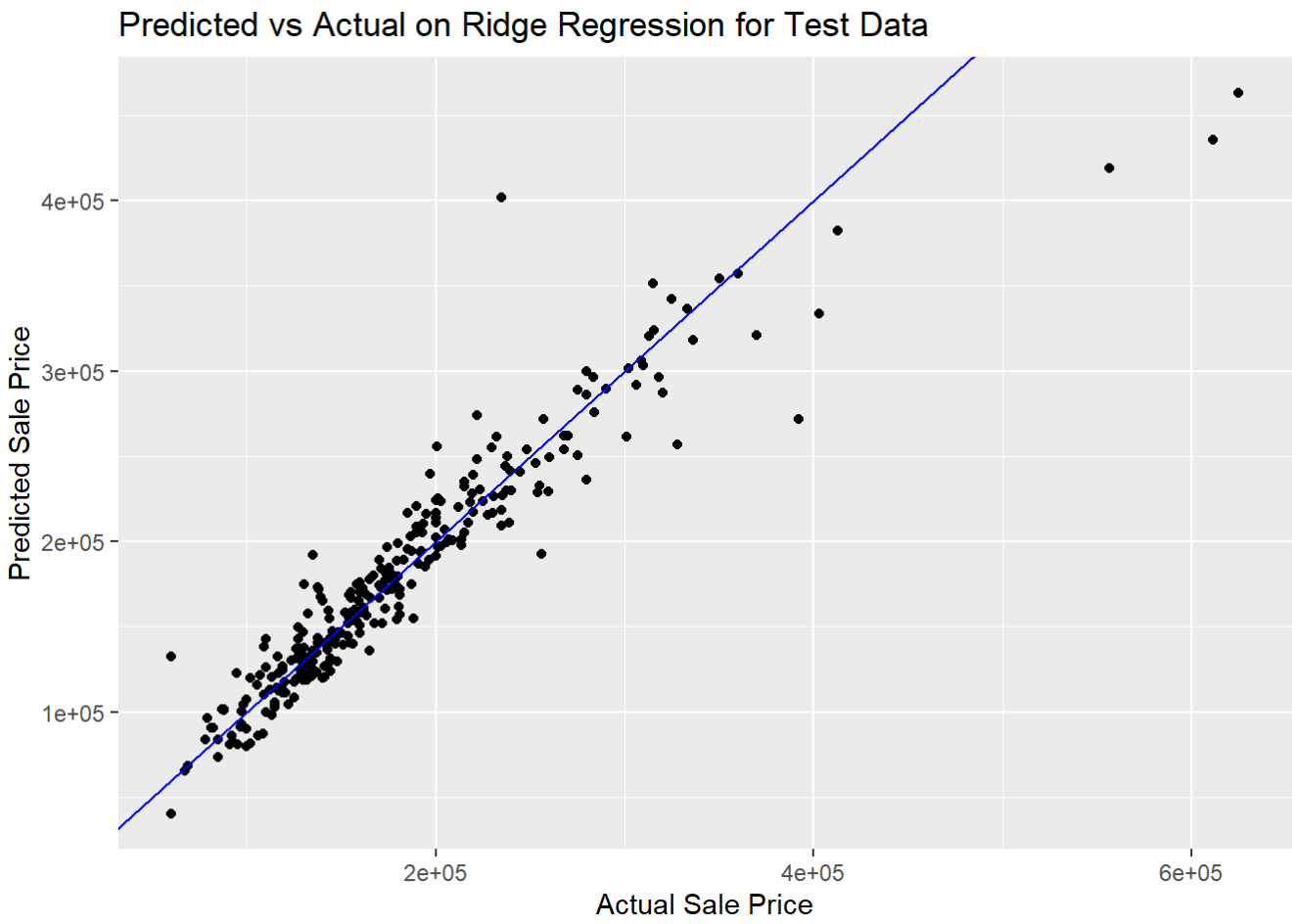
Visualizations

Ridge Regression Model

```
# Create a data frame for plotting
results_df_1 <- data.frame(Predicted = as.vector(predictions_ridge_test), Actual = t
est$SalePrice)

# Coefficients plot for Ridge model
coefficients_ridge <- coef(ridge_model)
plot(coefficients_ridge, main = "Ridge Coefficients")
```

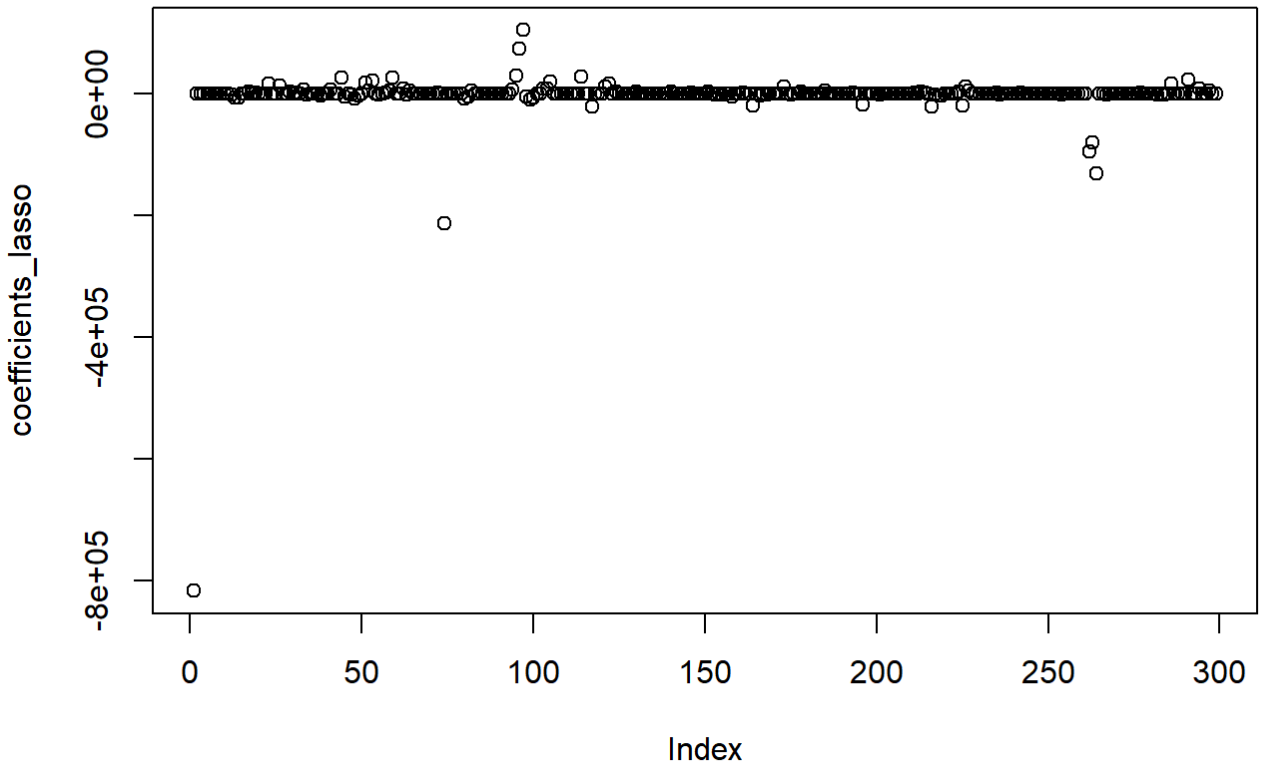



```
# Predicted vs Actual plot for Ridge model
ggplot(results_df_1, aes(x = Actual, y = Predicted)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, col = "blue") +
  ggtitle("Predicted vs Actual on Ridge Regression for Test Data") +
  xlab("Actual Sale Price") +
  ylab("Predicted Sale Price")
```

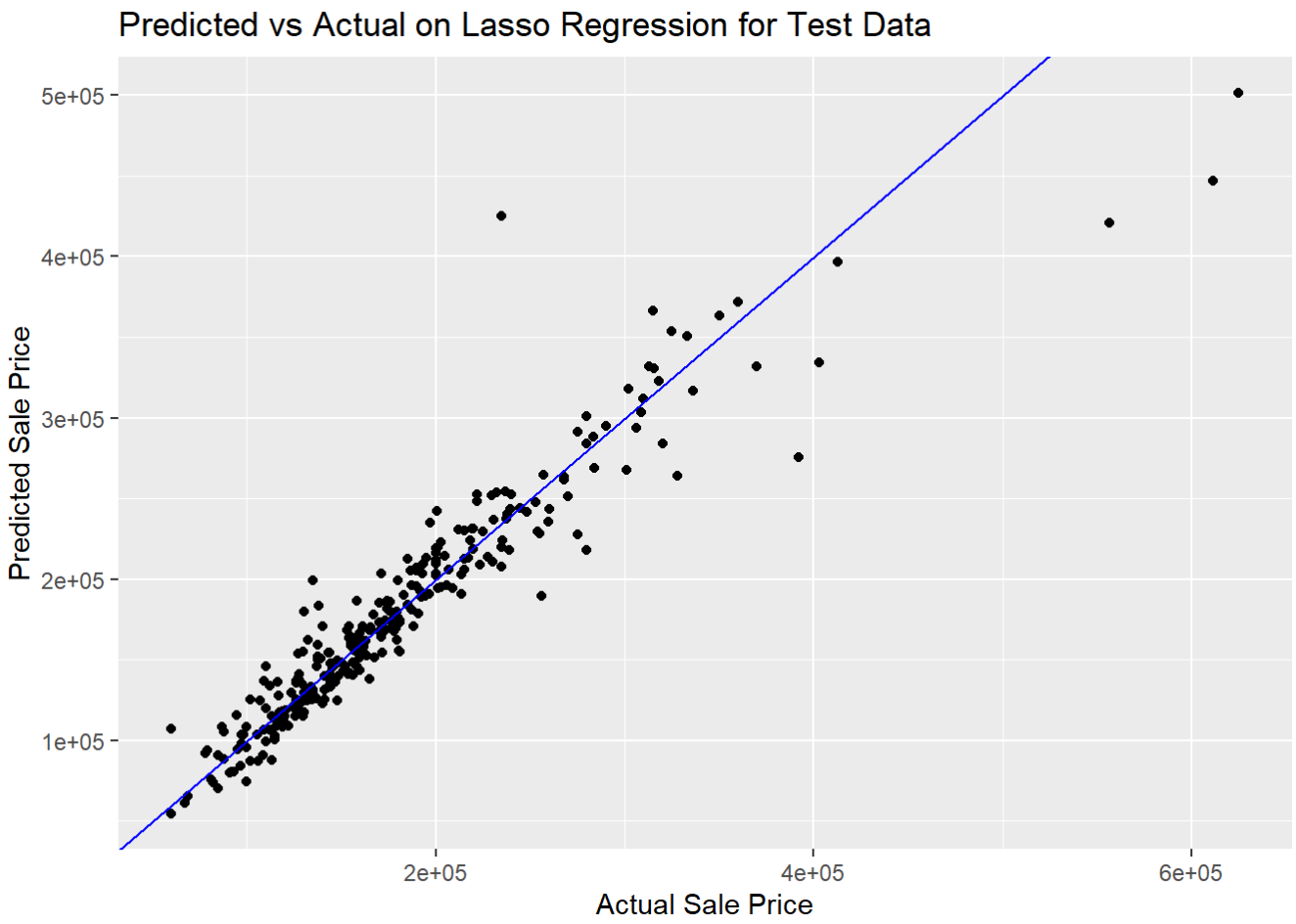


Lasso Regression Model

```
# Create a data frame for plotting
results_df_2 <- data.frame(Predicted = as.vector(predictions_lasso_test), Actual = t
est$SalePrice)

# Coefficients plot for Ridge model
coefficients_lasso <- coef(lasso_model)
plot(coefficients_lasso, main = "Lasso Coefficients")
```

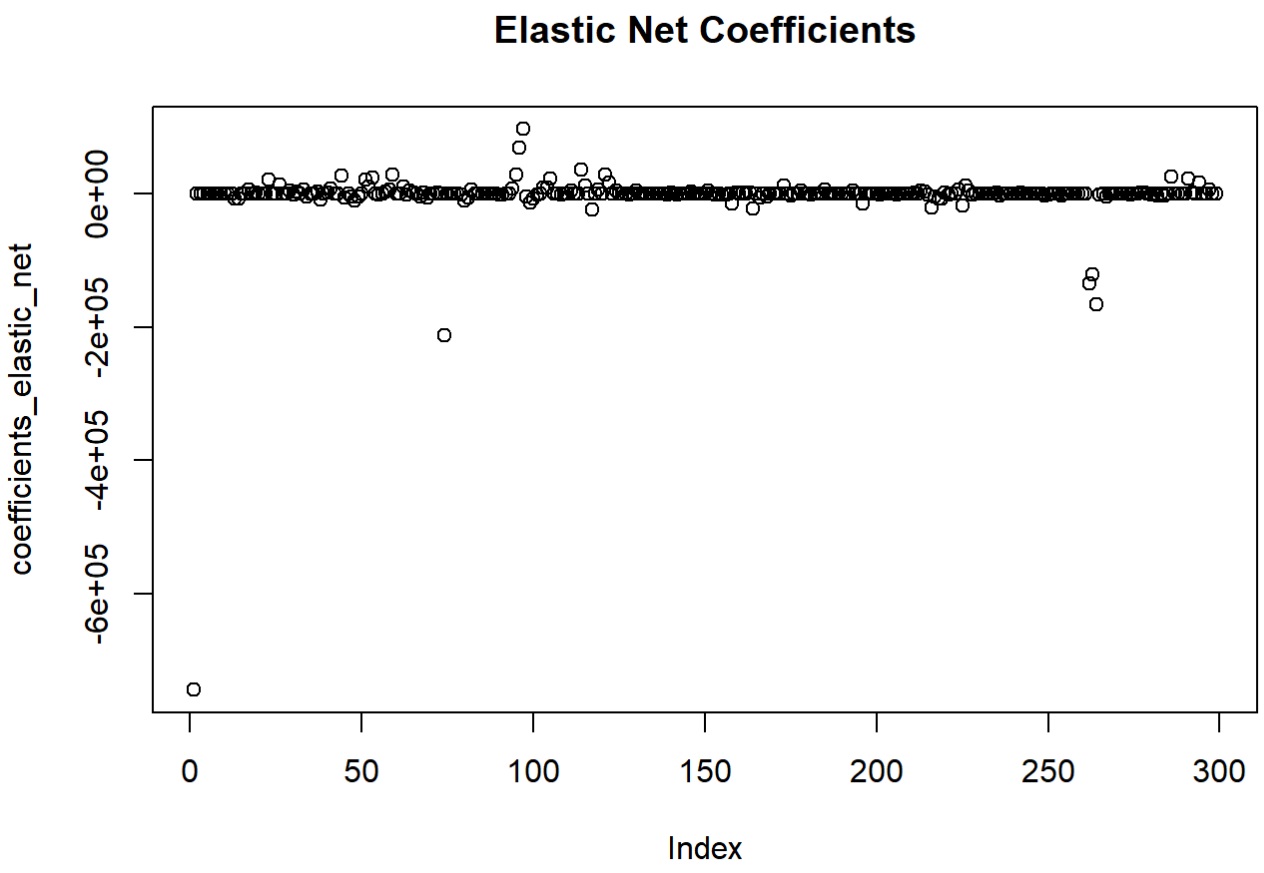



```
# Predicted vs Actual plot for Lasso model
ggplot(results_df_2, aes(x = Actual, y = Predicted)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, col = "blue") +
  ggtitle("Predicted vs Actual on Lasso Regression for Test Data") +
  xlab("Actual Sale Price") +
  ylab("Predicted Sale Price")
```



Elastic Net Regression

```
# Create a data frame for plotting
results_df_3 <- data.frame(Predicted = as.vector(predictions_elastic_test), Actual =
test$SalePrice)

# Coefficients plot for Ridge model
coefficients_elastic_net <- coef(elastic_net_model)
plot(coefficients_elastic_net, main = "Elastic Net Coefficients")
```




```
# Predicted vs Actual plot for Lasso model
ggplot(results_df_3, aes(x = Actual, y = Predicted)) +
  geom_point() +
  geom_abline(intercept = 0, slope = 1, col = "blue") +
  ggtitle("Predicted vs Actual on Elastic Net Regression for Test Data") +
  xlab("Actual Sale Price") +
  ylab("Predicted Sale Price")
```

Support Vector Machines

Create Data Partition

```
# Set seed for reproducibility
set.seed(100)

# Split the data

trainIndex <- createDataPartition(hpc_01$SalePrice, p = .8, list = FALSE)

trainData <- hpc_01[trainIndex, ]
testData <- hpc_01[-trainIndex, ]

row_count_train <- nrow(trainData)
row_count_test <- nrow(testData)

row_count_train
```

```
## [1] 1153
```

```
row_count_test
```

```
## [1] 286
```

```
round(row_count_train/nrow(hpc_01),2)
```

```
## [1] 0.8
```

```
round(row_count_test/nrow(hpc_01),2)
```

```
## [1] 0.2
```

SVM using Linear Kernel

The model below will utilize simple-hold out evaluation to first gauge this model's performance.

```
# Train SVM model with a linear kernel
tic()
svm_model_linear <- svm(SalePrice ~ ., data = trainData, kernel = "linear", cost =
1, scale = TRUE)
toc()
```

```
## 3.63 sec elapsed
```

```
svm_model_linear
```

```
##
## Call:
## svm(formula = SalePrice ~ ., data = trainData, kernel = "linear",
##     cost = 1, scale = TRUE)
##
##
## Parameters:
##    SVM-Type:  eps-regression
##  SVM-Kernel:  linear
##        cost:  1
##       gamma:  0.003355705
##     epsilon:  0.1
##
##
## Number of Support Vectors:  695
```

```
# Make prediction the train data
predictions_linear_train <- predict(svm_model_linear, newdata = trainData)

# Make predictions on the test data
predictions_linear_test <- predict(svm_model_linear, newdata = testData)

# Evaluating performance on Train Data
mmetric(trainData$SalePrice,predictions_linear_train,metrics_list)
```

```
##          MAE         RMSE           R2          COR         MAPE        RMSPE
## 1.182115e+04 2.113616e+04 9.252390e-01 9.618934e-01 7.013177e+00 1.254664e+00
##          RAE         RRSE
## 2.088153e+01 2.735193e+01
```

```
# Evaluating performance on Test Data
mmetric(testData$SalePrice,predictions_linear_test, metrics_list)
```

```
##          MAE         RMSE           R2          COR         MAPE        RMSPE
## 1.545378e+04 2.549662e+04 8.957120e-01 9.464206e-01 8.769010e+00 1.302462e+00
##          RAE         RRSE
## 2.766628e+01 3.245450e+01
```

The comparison of performance metrics between the training and test datasets for the linear kernel SVM model reveals several insights. On the training data, the model achieves a Mean Absolute Error (MAE) of 12,026.91 and a Root Mean Squared Error (RMSE) of 21,342.13, with a correlation (COR) of 0.9637 and an R^2 of 0.9287. These metrics indicate a strong predictive performance on the training data, with low error values and a high degree of explained variance. However, when evaluated on the test data, the model's performance slightly declines, evidenced by higher error values (MAE of 15,291.70 and RMSE of 22,458.20) and a slightly reduced correlation (0.9519) and R^2 (0.9061). This increase in error and decrease in correlation and R^2 suggest that the model experiences a modest reduction in predictive accuracy on unseen data, which could point to some degree of overfitting. Nevertheless, the relatively small decline in performance indicates that the linear kernel SVM generalizes reasonably well, but further refinement, such as regularization or feature engineering, could enhance its test set performance and reduce this gap.

Hyper parameter Tuning using Cross Validation

There are numerous hyperparamters that could be utilized to improve the model. It however would be inefficient to go through all the hyperparamters. Thus, grid search will be used to help find the best hyperparameters. It should be noted that the kernel will be kept the same.

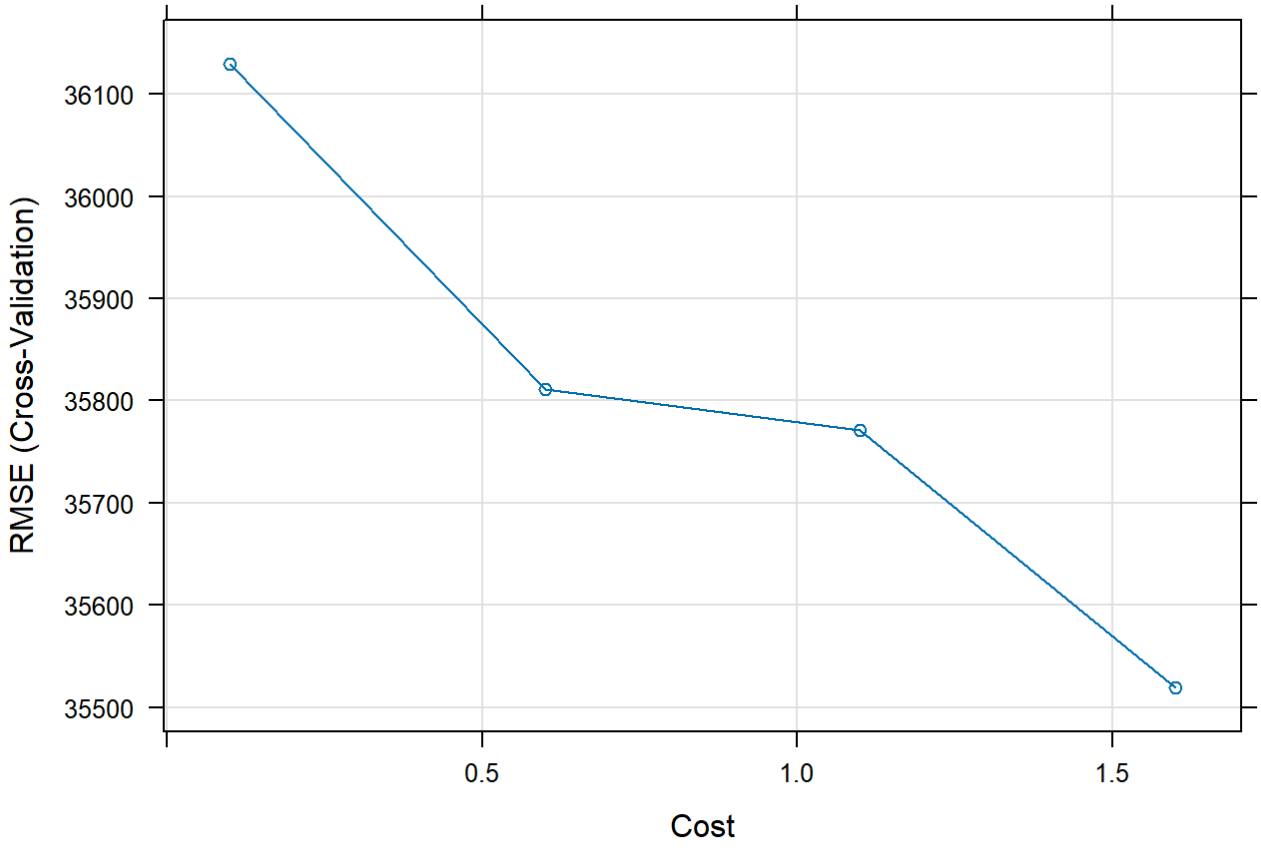
```
# Define tuning grid for hyperparameter tuning - different combinations of C
tune_grid <- expand.grid(C = seq(0.1, 2, by = 0.5))

# Perform Cross-Validation for regression
svm_tuned_linear <- train(SalePrice ~ .,
                          data = trainData,
                          method = "svmLinear",
                          tuneGrid = tune_grid,
                          metric = "RMSE", # Use RMSE for regression
                          trControl = trainControl(method = "cv", number = 5))

# Print the best model and parameters
print(svm_tuned_linear)
```

```
## Support Vector Machines with Linear Kernel
##
## 1153 samples
##   79 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 922, 923, 924, 921, 922
## Resampling results across tuning parameters:
##
##   C    RMSE      Rsquared   MAE
##   0.1  36130.08  0.7879621  23007.60
##   0.6  35811.69  0.7925817  22589.53
##   1.1  35770.96  0.7920089  22563.47
##   1.6  35518.72  0.7952105  22551.46
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was C = 1.6.
```

The tuned model employs a systematic grid search over a range of hyperparameters (values of C from 0.1 to 2.0) and selects the best model based on the smallest RMSE value. The optimal C value found through this process is 1.6, resulting in an RMSE of 38,424.82, R^2 of 0.7783, and MAE of 23,539.75. This indicates a balance between bias and variance, where the model effectively captures the underlying patterns in the data while avoiding overfitting.

Model Plot

```
# Plot model
plot(svm_tuned_linear)
```



The plot shows the RMSE score for the model across different C values, where C = 1.6 achieves the lowest RMSE, indicating the best model performance.

SVM using RBF Kernel

```
# Train SVM model with RBF kernel
svm_model_rbf <- svm(SalePrice ~ ., data = trainData, kernel = "radial", cost = 1, g
amma = 0.1)

svm_model_rbf
```

```
##
## Call:
## svm(formula = SalePrice ~ ., data = trainData, kernel = "radial",
##     cost = 1, gamma = 0.1)
##
##
## Parameters:
##    SVM-Type:  eps-regression
##  SVM-Kernel:  radial
##        cost:  1
##       gamma:  0.1
##     epsilon:  0.1
##
##
## Number of Support Vectors:  857
```

```
# Make prediction the train data
predictions_rbf_train <- predict(svm_model_rbf, newdata = trainData)

# Make predictions on the test data
predictions_rbf_test <- predict(svm_model_rbf, newdata = testData)

# Evaluating performance on Train Data
mmetric(trainData$SalePrice,predictions_rbf_train,metrics_list)
```

```
##          MAE         RMSE           R2          COR         MAPE        RMSPE
## 1.278839e+04 3.300890e+04 8.571956e-01 9.258486e-01 6.461886e+00 1.205117e+00
##          RAE         RRSE
## 2.259011e+01 4.271622e+01
```

```
# Evaluating performance on Test Data
mmetric(testData$SalePrice,predictions_rbf_test, metrics_list)
```

```
##          MAE         RMSE           R2          COR         MAPE        RMSPE
## 3.430367e+04 6.104687e+04 4.749815e-01 6.891891e-01 1.965925e+01 3.022800e+00
##          RAE         RRSE
## 6.141248e+01 7.770621e+01
```

Let's compare the result with the simple linear kernal SVM we trained before.

The comparison between the performance of SVM models with linear and radial kernels on both the training and test datasets highlights the underperformance of the radial kernel in this specific regression task.

Linear Kernel: For the train dataset, the linear kernel achieves a low MAE of 12,026.91 and RMSE of 21,342.13, with a high R^2 value of 0.9287 and a strong correlation (COR) of 0.9637. On the test dataset, while performance slightly declines, the model still maintains respectable accuracy, with an MAE of 15,291.70, RMSE of 22,458.20, R^2 of 0.9061, and COR of 0.9519. These metrics suggest that the linear kernel SVM generalizes well and provides robust predictions on unseen data.

Radial Kernel: The radial kernel, on the other hand, performs significantly worse. On the train dataset, it has a much higher MAE (13,993.60) and RMSE (37,485.72) compared to the linear kernel, along with a lower R^2 (0.8266) and COR (0.9092). This indicates that the model struggles to fit the training data effectively, likely due to the increased complexity of the radial basis function. The performance on the test dataset deteriorates further, with an MAE of 33,418.74 and RMSE of 51,593.48, accompanied by a very low R^2 (0.5547) and COR (0.7448). These results suggest that the radial kernel SVM not only overfits the training data but also generalizes poorly, as evidenced by its significant error and reduced correlation on the test data.

Conclusion: The linear kernel outperforms the radial kernel in this regression task, providing better accuracy, robustness, and generalization. While radial kernels are powerful for capturing non-linear relationships, their complexity needs careful tuning and justification based on the dataset's characteristics. In this case, the linear kernel is the more appropriate choice.

Hyper Parameter Tuning using Cross Validation

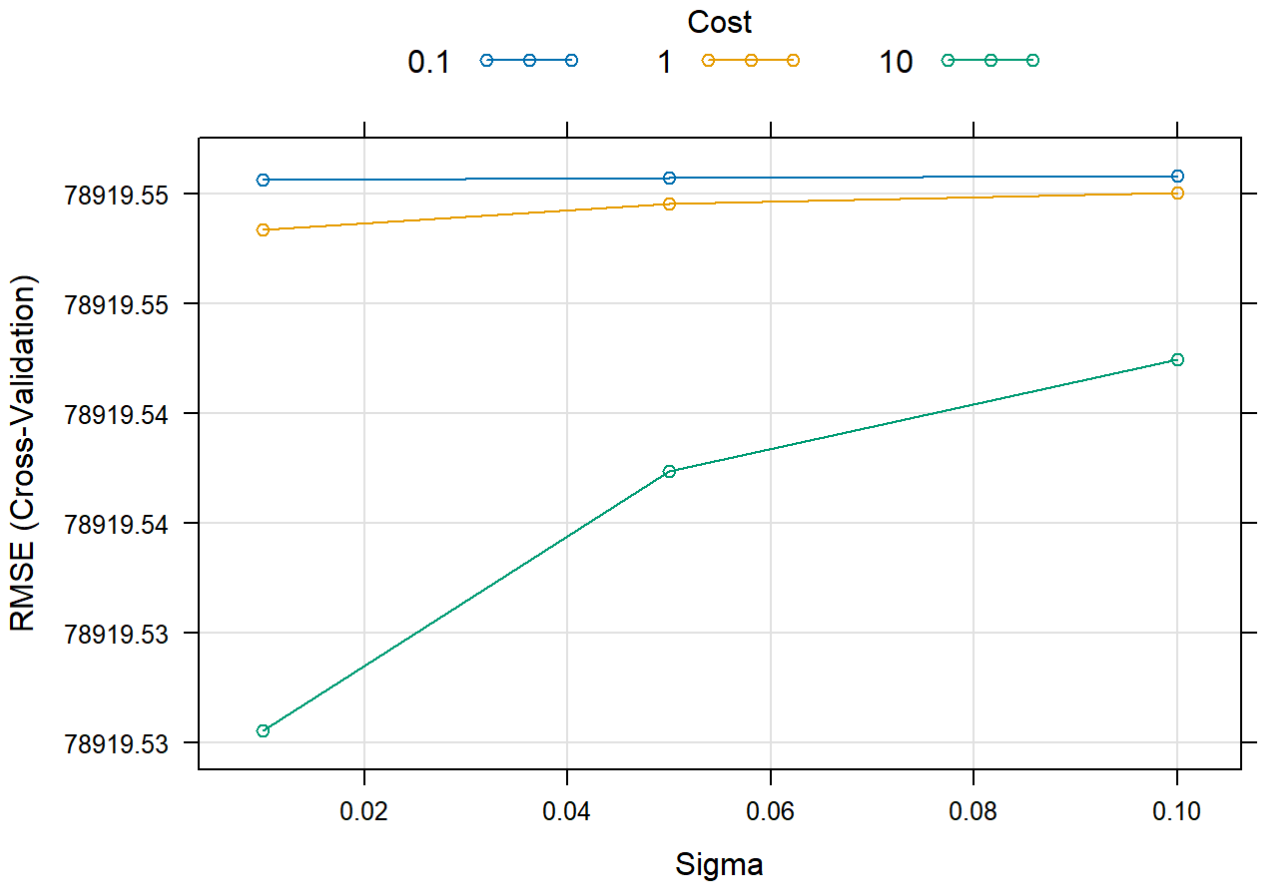
```
# Define tuning grid for tuning hyper parameters - different combinations of C and s
igma

tune_grid_rbf <- expand.grid(sigma = c(0.01, 0.05, 0.1), C = c(0.1, 1, 10))

# Perform Cross-Validation for regression
svm_tuned_rbf <- train(SalePrice ~ .,
                       data = trainData,
                       method = "svmRadial",
                       tuneGrid = tune_grid_rbf,
                       metric = ("RMSE"), # Use RMSE for regression
                       trControl = trainControl(method = "cv", number = 5))

# Print the best model and parameters
print(svm_tuned_rbf)
```

```
## Support Vector Machines with Radial Basis Function Kernel
##
## 1153 samples
##   79 predictor
##
## No pre-processing
## Resampling: Cross-Validated (5 fold)
## Summary of sample sizes: 922, 922, 922, 924, 922
## Resampling results across tuning parameters:
##
##   sigma  C     RMSE      Rsquared     MAE
##   0.01    0.1  78919.55  0.003950504  54862.85
##   0.01    1.0  78919.55  0.003950502  54862.84
##   0.01   10.0  78919.53  0.003950502  54862.80
##   0.05    0.1  78919.55  0.003378199  54862.85
##   0.05    1.0  78919.55  0.003378199  54862.84
##   0.05   10.0  78919.54  0.003378199  54862.81
##   0.10    0.1  78919.55  0.002883524  54862.85
##   0.10    1.0  78919.55  0.002883524  54862.84
##   0.10   10.0  78919.54  0.002883524  54862.82
##
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were sigma = 0.01 and C = 10.
```

As predicted from the comparison in between the simple Linear and simple Radial kernal SV Models, the radial kernel is not getting the variance at all as can be seen from the extremely low R^2 values above.

Model Plot

```
# Plot model
plot(svm_tuned_rbf)
```



K-fold validation

Since the linear kernel did the best, this model will be evaluated on the entire dataset to further assess performance. The model did use cross-validation to tune the hyperparmaters, however this was only with the train set.

Since, we have now found the best model, we are going to assess it's performance on the entire dataset. There will be no hyper-paramter tuning of the SVM model. This is just to assess its performance as previously stated.

Create k-fold function that accepts custom parameter settings

```
cv_function_ksvm <- function(df, target, nFolds, seedVal, metrics_list, kern, c)
{
# create folds using the assigned values

set.seed(seedVal)
folds = createFolds(df[,target],nFolds)

# The lapply loop

cv_results <- lapply(folds, function(x)
{
# data preparation:

  test_target <- df[x,target]
  test_input <- df[x,-target]

  train_target <- df[-x,target]
  train_input <- df[-x,-target]
   pred_model <- ksvm(train_target ~ .,data = train_input,kernel=kern,C=c)
  pred <- predict(pred_model, test_input)
  return(mmetric(test_target,pred,metrics_list))
})

cv_results_m <- as.matrix(as.data.frame(cv_results))
cv_mean<- as.matrix(rowMeans(cv_results_m))
cv_sd <- as.matrix(rowSds(cv_results_m))
colnames(cv_mean) <- "Mean"
colnames(cv_sd) <- "Sd"
cv_all <- cbind(cv_results_m, cv_mean, cv_sd)

kable(t(cbind(cv_all)),digits=2)
}
```

Run k-fold function

```
cv_function_ksvm(hpc_01,79,5,100,metrics_list = metrics_list,"vanilladot",1.6)
```

```
##  Setting default kernel parameters
##  Setting default kernel parameters
##  Setting default kernel parameters
##  Setting default kernel parameters
##  Setting default kernel parameters
```

	MAE	RMSE	R2	COR	MAPE	RMSPE	RAE	RRSE
Fold1	15032.30	24439.43	0.90	0.95	8.84	1.31	26.77	31.75
Fold2	16338.68	22639.10	0.92	0.96	10.26	1.63	27.94	28.68
Fold3	15766.52	32763.59	0.83	0.91	9.13	1.75	27.76	42.05
Fold4	15075.34	22317.70	0.91	0.95	9.83	1.65	27.48	30.16
Fold5	17837.92	30638.85	0.85	0.92	10.08	1.70	31.79	38.51
Mean	16010.15	26559.73	0.88	0.94	9.63	1.61	28.35	34.23
Sd	1155.22	4821.54	0.04	0.02	0.62	0.17	1.98	5.77

Function used a slightly different ksvm algorithm, however the results are still the same and the algorithm still uses the linear kernel.

Simple hold out evaluation for reference

We'll also do simple hold out evaluation for reference, however the k-fold validation still remains the most robust method of validation since it does not depend on a particular split.

```
#Create the best model

best_model_ksvm <- ksvm(SalePrice ~ . , data = trainData,kernel = "vanilladot", C =
1.6)
```

```
##  Setting default kernel parameters
```

```
best_model_ksvm_trainpred <- predict(best_model_ksvm,newdata = trainData)

best_model_ksvm_testpred <- predict(best_model_ksvm, newdata = testData)

mmetric(trainData$SalePrice,best_model_ksvm_trainpred,metrics_list)
```

```
##          MAE         RMSE           R2          COR         MAPE        RMSPE
## 1.180465e+04 2.101941e+04 9.260505e-01 9.623152e-01 7.011930e+00 1.249490e+00
##          RAE         RRSE
## 2.085237e+01 2.720085e+01
```

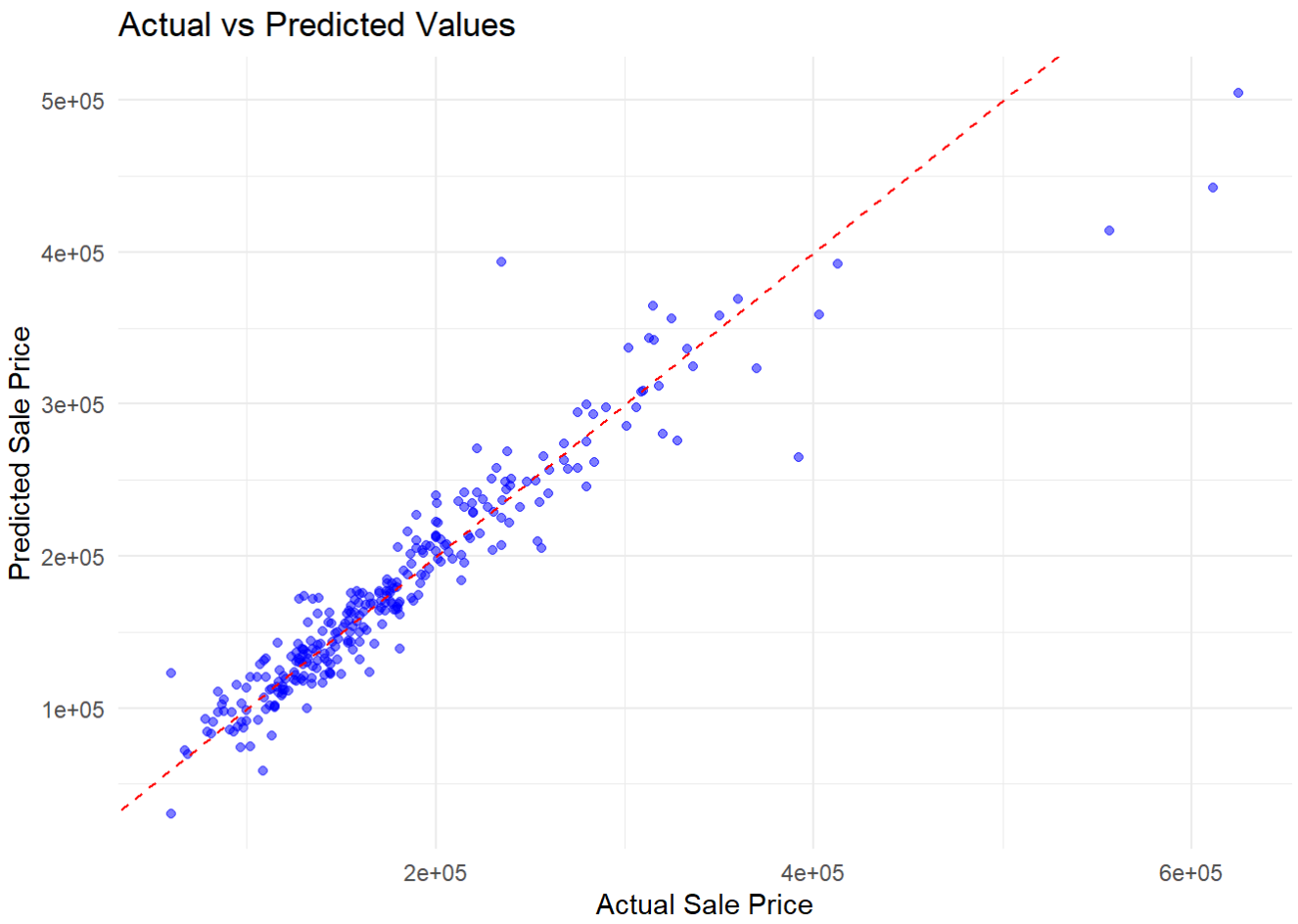
```
mmetric(testData$SalePrice, best_model_ksvm_testpred,metrics_list)
```

```
##          MAE         RMSE           R2          COR         MAPE         RMSPE
## 1.562495e+04 2.593427e+04 8.918009e-01 9.443521e-01 8.904998e+00 1.343278e+00
##          RAE         RRSE
## 2.797271e+01 3.301158e+01
```

Visualization of Best Model's Performance

Shows how well the model performs on the test data.

```
# Create a dataframe with actual and predicted values
results_df <- data.frame(
  Actual = testData$SalePrice,
  Predicted = best_model_ksvm_testpred
)

# Plot Actual vs Predicted
ggplot(results_df, aes(x = Actual, y = Predicted)) +
  geom_point(color = "blue", alpha = 0.5) +  # Scatter plot points
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +  # Per
fect fit line
  labs(
    title = "Actual vs Predicted Values",
    x = "Actual Sale Price",
    y = "Predicted Sale Price"
  ) +
  theme_minimal()
```



This model is doing a decent job with most of the points fairly close to the line. There are only a few points which deviate a lot from the line.

Limitations and Robustness Checks for SVM

Limitations:

Model interpretability: While SVMs, especially with an RBF kernel, can be powerful classifiers, they are often harder to interpret than simpler models like logistic regression. The decision boundary in a higher-dimensional space is not easily interpretable, making it harder to provide actionable insights into why certain customers are flagged as likely to default.

Overfitting risk: Even though cross-validation was applied, the risk of overfitting still exists, particularly in models with a high number of support vectors (as seen in the RBF kernel). This overfitting could lead to poor generalization when applied to new data.

Robustness Checks:

Cross-validation (5-fold): To ensure model stability, 5-fold cross-validation was applied during hyperparameter tuning for both the linear and RBF SVM models. This provided more reliable estimates of model performance by reducing the variance that can arise from a single train-test split.

Grid search for hyperparameter tuning: Extensive grid search was conducted to identify the best values for C (regularization parameter) in both models and sigma (kernel coefficient) for the RBF kernel. This ensures that the models were fine-tuned and optimized for the specific dataset.

XGBoost

Modeling

Split Data into Train and Test Groups

```
set.seed(123)
house_prices_rn <- initial_split(hpc_01, prop = 0.8,
strata = SalePrice)

HousePrice_train <- training(house_prices_rn)
HousePrice_test <- testing(house_prices_rn)
```

Grid Search

```
# Preprocessing with a Recipe
rec_HousePrice <- recipe(SalePrice ~ ., data = HousePrice_train) %>%
  step_dummy(all_nominal_predictors())     #dummify all numeric cols for modelling
```

Define the Boosted Tree Model with Adjustable Hyperparameters

```
model_HousePrice <- boost_tree(
  trees = tune(),         # Number of trees in the ensemble
  tree_depth = tune(),    # Maximum depth of each tree
  learn_rate = tune(),    # Learning rate
  mtry = tune(),          # Number of predictors to sample at each split
  min_n = tune(),         # Minimum number of data points in a node
  loss_reduction = tune(),# Minimum loss reduction for a split
  sample_size = tune()    # Proportion of data for each tree
) %>%
set_engine("xgboost", verbosity = 0) %>%
set_mode("regression")


model_HousePrice
```

```
## Boosted Tree Model Specification (regression)
##
## Main Arguments:
##   mtry = tune()
##   trees = tune()
##   min_n = tune()
##   tree_depth = tune()
##   learn_rate = tune()
##   loss_reduction = tune()
##   sample_size = tune()
##
## Engine-Specific Arguments:
##   verbosity = 0
##
## Computational engine: xgboost
```

Create Hyperparameter Grid for Tuning

```
# Define the hyperparameter grid
hyper_grid <- grid_random(
  trees(range = c(2, 500)),             # Number of trees in the ensemble
  tree_depth(range = c(2, 10)),          # Maximum depth of each tree
  learn_rate(range = c(0.01, 0.3)),      # Learning rate
  finalize(mtry(), HousePrice_train),    # Number of predictors to sample at each sp
lit
  min_n(range = c(2, 10)),               # Minimum number of data points in a node
  loss_reduction(range = c(0, 3)),       # Minimum loss reduction for a split
  sample_size = sample_prop(c(0.6, 0.9)),  # Proportion of data for each tree
  size = 200
)
```

Perform 3-fold Cross Validation

```
HousePrice_folds <- vfold_cv(HousePrice_train, v = 3)
```

Define Workflow

```
# Combine model specification and recipe into a single workflow for streamlined tuni
ng and evaluation

HousePrice_wf <- workflow() %>%
add_model(model_HousePrice) %>%
add_recipe(rec_HousePrice)

# See how many cores are available on local machine
num_cores <- detectCores()
num_cores
```

```
## [1] 8
```

Hyperparameter Tuning with Grid Search

```
# Hyperparameter Tuning with Grid Search
doParallel::registerDoParallel(cores = num_cores)

# Run the tuning process with both rmse and rsq metrics
set.seed(1234)
HousePrice_tune <- HousePrice_wf %>%
  tune_grid(
    resamples = HousePrice_folds,
    grid = hyper_grid,
    metrics = metric_set(rmse, rsq)  # Include both rmse and rsq
  )

# Stop parallel processing
doParallel::stopImplicitCluster()
# Collect metrics from the tuning results
HousePrice_metrics <- HousePrice_tune %>%
  collect_metrics() %>%
  pivot_wider(names_from = .metric, values_from = mean)

# Filter rows with non-missing values for rmse and rsq separately
HousePrice_rmse <- HousePrice_metrics %>%
  filter(!is.na(rmse)) %>%
  arrange(rmse) %>%
  head(1) # Select the model with the lowest RMSE

HousePrice_rsq <- HousePrice_metrics %>%
  filter(!is.na(rsq)) %>%
  arrange(desc(rsq)) %>%
  head(1) # Select the model with the highest R²

# Print the metrics and the best models for rmse and rsq

print(HousePrice_rmse)
```

```
## # A tibble: 1 × 13
##    mtry trees min_n tree_depth learn_rate loss_reduction sample_size .estimator
##   <int> <int> <int>      <int>      <dbl>          <dbl>       <dbl> <chr>
## 1     5   306     6          2       1.15           24.6       0.892 standard
## # ℹ 5 more variables: n <int>, std_err <dbl>, .config <chr>, rmse <dbl>,
## #   rsq <dbl>
```

```
print(HousePrice_rsq)
```

```
## # A tibble: 1 × 13
##    mtry trees min_n tree_depth learn_rate loss_reduction sample_size .estimator
##   <int> <int> <int>      <int>      <dbl>          <dbl>       <dbl> <chr>
## 1    10    34     4          2       1.12           282.       0.789 standard
## # ℹ 5 more variables: n <int>, std_err <dbl>, .config <chr>, rmse <dbl>,
## #   rsq <dbl>
```

Aggregate results and select the best-performing model based on low RMSE and high R^2, balancing accuracy and predictive power.

```
best_model_rmse <- HousePrice_tune %>%
  select_best(metric = "rmse")

best_model_rmse
```

mtry	trees	min_n	tree_depth	learn_rate	loss_reduction	sample_size
<int>	<int>	<int>	<int>	<dbl>	<dbl>	<dbl>
5	306	6	2	1.145587	24.63023	0.8919679

1 row | 1-7 of 8 columns

```
best_model_rsq <- HousePrice_tune %>%
  select_best(metric = "rsq")

best_model_rsq
```

mtry	trees	min_n	tree_depth	learn_rate	loss_reduction	sample_size
<int>	<int>	<int>	<int>	<dbl>	<dbl>	<dbl>
10	34	4	2	1.123282	281.8862	0.789059

1 row | 1-7 of 8 columns

Evaluate on Test Dataset

```
final_workflow_rmse <-
HousePrice_wf %>%
finalize_workflow(best_model_rmse)

final_workflow_rsq <-
HousePrice_wf %>%
finalize_workflow(best_model_rsq)


final_fit_rmse <-
final_workflow_rmse %>%
last_fit(split = house_prices_rn)

final_fit_rmse %>%
collect_metrics() %>%
    mutate_if(is.numeric, round, 2)
```

.metric	.estimator	.estimate	.config
<chr>	<chr>	<dbl>	<chr>
rmse	standard	42764.77	Preprocessor1_Model1
rsq	standard	0.73	Preprocessor1_Model1

2 rows

```
final_fit_rsq <-
final_workflow_rsq %>%
last_fit(split = house_prices_rn)

final_fit_rsq %>%
collect_metrics() %>%
    mutate_if(is.numeric, round, 2)
```

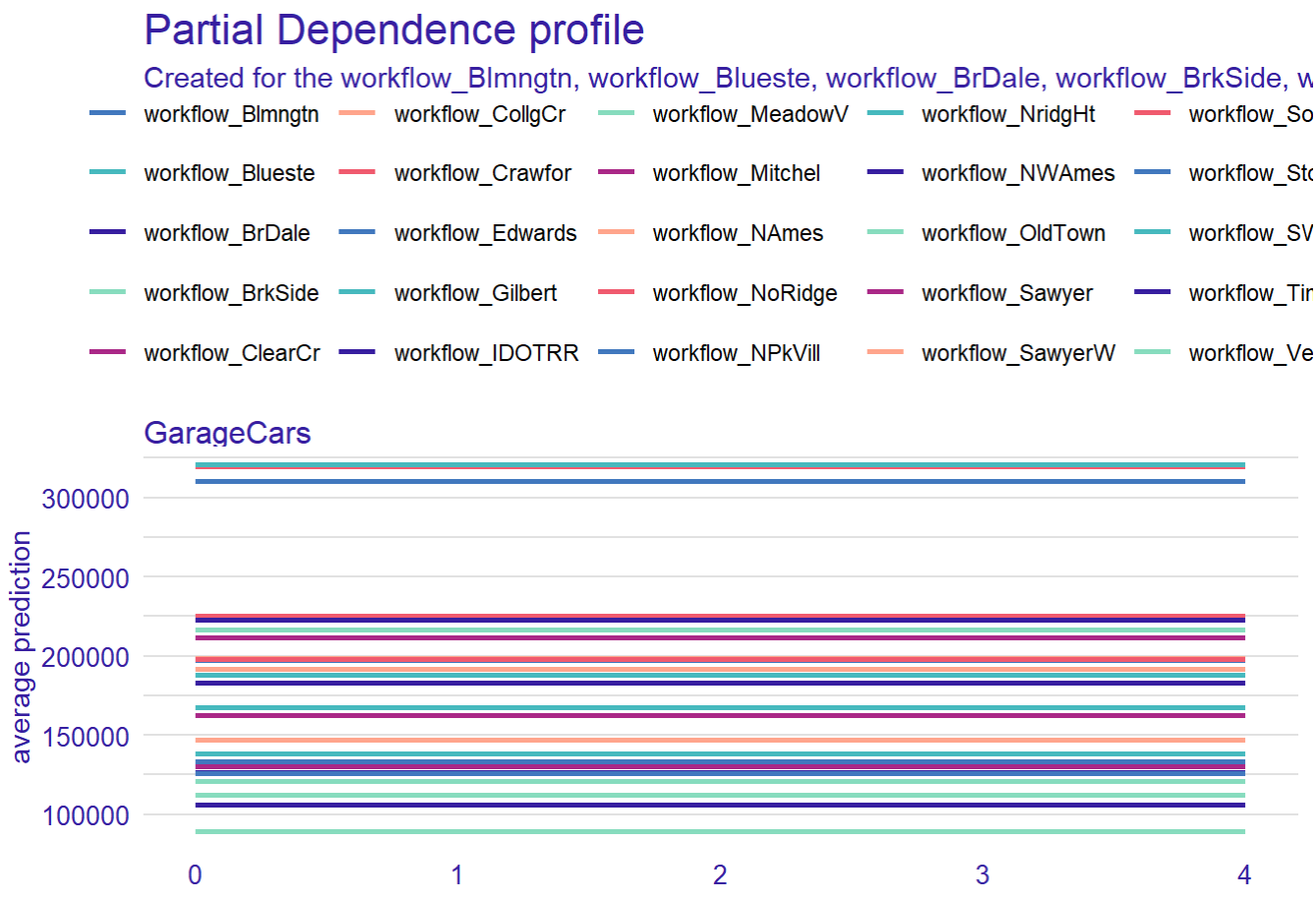
.metric	.estimator	.estimate	.config
<chr>	<chr>	<dbl>	<chr>
rmse	standard	43754.37	Preprocessor1_Model1
rsq	standard	0.73	Preprocessor1_Model1

2 rows

This is a good model, the model is not overfitting and generalizing almost perfectly to new data. R^2 slightly increased from 0.71% to 0.73%. The RMSE has also decreased from 44660 dollars to 43754 dollars. Training metrics (from cross-validation) show high R^2 and low RMSE, which can also be seen in the performance on the test.

Graphs

```
library(DALEXtra)

model_fitted <- final_workflow_rsq %>%
fit(data = HousePrice_train)

explainer_rf <- explain_tidymodels(model_fitted,
data = HousePrice_train[,-79],
y = HousePrice_train$SalePrice,
type = "pdp",verbose = FALSE)

garagecars_comp <- model_profile(explainer_rf,
variables = "GarageCars",
groups = "Neighborhood",N=NULL)

# Plotting the variable profile
plot(garagecars_comp)
```



This Partial Dependence Profile illustrates the effect of the GarageCars variable (number of garage spaces) on the average predictions of the XGboost model across multiple defined workflows. The x-axis represents the GarageCars values (0 to 4), and the y-axis shows the corresponding average predictions,

ranging from 100,000 to 300,000. Each workflow highlights how predictions change as GarageCars increase.The overall trend suggests that predictions increase as the GarageCars increase.

Assumptions, Limitations and Robustness Checks for XGBoost (Boosting)

Assumptions

The primary assumption was that the relationship between the predictors and the target variable is non-linear. We also assumed that there is no significant multicollinearity among the predictors or with the target variable. The data was expected to be unnormalized and potentially skewed, as XGBoost can handle skewed data effectively by focusing on regions where errors are high. Certain missing values were assumed to be managed automatically by XGBoost.

Limitations

The dataset used for this analysis is limited to one location, so generalization to other regions might not be straightforward.

Xgboost being a complex and robust model, is less interpretable than simpler linear models, making it harder to directly understand the impact of individual features on predictions.

As observed in few results, the model might show signs of overfitting, which indicates that it may not generalize well to new data without additional tuning or regularization.

The dataset is limited and may not capture external causal features affecting house prices, such as economic conditions or neighborhood developments, which may limit the model's predictive scope.

Robustness checks:

A grid search was conducted to identify optimal parameters, followed by a 3-fold cross-validation. The xgboost itself is robust, but further checks were performed by re-evaluating the best parameters and refining the model through a finalized workflow.

3-fold cross validation was performed on the best model by collecting additional metrics such as R-squared, MAE (Mean Absolute Error), and MAPE (Mean Absolute Percentage Error) apart from RMSE (Root Mean Squared Error). Finally, model performance was compared and assessed with the test set to check for generalizability, although the additional cross-validation check did not show proving results.

Our outcome has been affected by the cleaning decisions that we have used when cleaning the data. There are numerous ways and approaches to cleaning the data. For instance, we did not consider low-variance columns and imputed missing values for a with median/mode for a few columns which may introduce biases if the missing values are not random. (Based on our analysis for some of the columns like Lot Frontage, they appeared random, but this could not be the case. There was no strong evidence or documention to suggest that missing values conveyed something else like with the other variables like Fireplace Qu where missing values mean that there is no fireaplce.)

Additionally, we utilized simple imputation as opposed to utilizing a more complicated package like MICE. It is possible that our results would change if we had decided to take another approach.

Likewise, we adopted a very conservative approach with outliers since some extreme observations may be valid observations. This however will cause the residuals to increase and consequently any additional regression tree will work harder to correct for these larger residuals. This can consequently increase in sightly more overfitting. This issue however was avoided by hyperparameter tuning like reducing the tree depth for instance to 1.

We also created another column that was originally not apart of the dataset to explicitly record whether a house had a garage or not. Although this column ultimately didn't end up being important, it's still another column that XGboost must decide whether to keep or drop when making predictions.

Principal Component Analysis (PCA)

Binning the SalePrice Variable

```
# Display the structure of the data

hp <- hpc_01


#Handle Target Variable: create 10 bins
bin_breaks <- seq(min(hp$SalePrice, na.rm = TRUE),  #find min price
                  max(hp$SalePrice, na.rm = TRUE),  #find max price
                  length.out = 11)      # 11 values, creating 10 equal intervals (bi
ns)

bin_breaks
```

```
##  [1]  34900 105910 176920 247930 318940 389950 460960 531970 602980 673990
## [11] 745000
```

```
# Create separate column for binned SalePrice
hp <- hp %>%
  mutate(SalePrice_Bin = cut(SalePrice, breaks = bin_breaks, include.lowest = TRUE,
labels = FALSE)) %>%
  mutate(SalePrice_Bin = as.factor(SalePrice_Bin))

# Remove main SalePrice column
hp <- hp %>%
  select(-SalePrice)

# Check the distribution of binned SalePrice:
table(hp$SalePrice_Bin)
```

```
##
##   1   2   3   4   5   6   7   8   9  10
## 140 704 374 140  49  20   5   4   2   1
```

```
#check structure of binned SalePrice
summary(hp$SalePrice_Bin)
```

```
##   1   2   3   4   5   6   7   8   9  10
## 140 704 374 140  49  20   5   4   2   1
```

```
str(hp$SalePrice_Bin)
```

```
##  Factor w/ 10 levels "1","2","3","4",..: 3 3 3 2 4 2 4 3 2 2 ...
```

We do not use any dependent variables in exploratory analysis methods. Remove the dependent variable or treat it as an independent categorical variable. Use bin breaks to categorize the SalePrice values into bins (low, medium, high price houses). The majority of the data points fall into bins 2 and 3, indicating that most houses have a low SalePrice, also indicating right skewed distribution. The higher bins (7 to 10) have very few observations, suggesting that very few houses have too expensive prices.

Standardize the Data

```
# Standardize numeric columns for mean =0 , sd = 1

scales <- build_scales(hp, verbose = TRUE)
```

```
##  [1] "MSSubClass"    "MSZoning"      "Street"        "Alley"
##  [5] "LotShape"      "LandContour"   "Utilities"     "LotConfig"
##  [9] "LandSlope"     "Neighborhood"  "Condition1"    "Condition2"
## [13] "BldgType"      "HouseStyle"    "OverallQual"   "OverallCond"
## [17] "RoofStyle"     "RoofMatl"      "Exterior1st"   "Exterior2nd"
## [21] "MasVnrType"    "ExterQual"     "ExterCond"     "Foundation"
## [25] "BsmtQual"      "BsmtCond"      "BsmtExposure"  "BsmtFinType1"
## [29] "BsmtFinType2"  "Heating"       "HeatingQC"     "CentralAir"
## [33] "Electrical"    "KitchenQual"   "Functional"    "FireplaceQu"
## [37] "GarageType"    "GarageFinish"  "GarageQual"    "GarageCond"
## [41] "PavedDrive"    "PoolQC"        "Fence"         "MiscFeature"
## [45] "MoSold"        "SaleType"      "SaleCondition" "HasGarage"
## [49] "SalePrice_Bin"
## [1] "build_scales: c(\"MSSubClass\", \"MSZoning\", \"Street\", \"Alley\", \"LotSh
ape\", \"LandContour\", \"Utilities\", \"LotConfig\", \"LandSlope\", \"Neighborhood
\", \"Condition1\", \"Condition2\", \"BldgType\", \"HouseStyle\", \"OverallQual\",
\"OverallCond\", \"RoofStyle\", \"RoofMatl\", \"Exterior1st\", \"Exterior2nd\", \"Ma
sVnrType\", \"ExterQual\", \"ExterCond\", \"Foundation\", \"BsmtQual\", \"BsmtCond
\", \"BsmtExposure\", \"BsmtFinType1\", \"BsmtFinType2\", \"Heating\", \"HeatingQC
\", \"CentralAir\", \"Electrical\", \"KitchenQual\", \"Functional\", \"FireplaceQu
\", \"GarageType\", \n\"GarageFinish\", \"GarageQual\", \"GarageCond\", \"PavedDrive
\", \"PoolQC\", \"Fence\", \"MiscFeature\", \"MoSold\", \"SaleType\", \"SaleConditio
n\", \"HasGarage\", \"SalePrice_Bin\") aren't columns of types numeric i do nothing
for those variables."
## [1] "build_scales: I will compute scale on  31 numeric columns."
## [1] "build_scales: it took me: 0s to compute scale for 31 numeric columns."
```

```
hp <- fast_scale(hp, scales = scales, verbose = TRUE)
```

```
## [1] "fast_scale: I will scale 31 numeric columns."
## [1] "fast_scale: it took me: 0.01s to scale 31 numeric columns."
```

Standardization ensures that all features contribute equally, preventing features with larger scales from dominating the analysis. The fastScale function quickly scales the numeric columns.

Covariance Matrix

```
# Handle NA's

library(data.table)

sum(is.na(hp))
```

```
## [1] 0
```

```r
# Impute NA's with column median
hp_num <- lapply(hp, function(x) {
  if (is.numeric(x)) {
    x[is.na(x)] <- median(x, na.rm = TRUE)
  }
  return(x)
})

sum(is.na(hp_num))
```

```
## [1] 0
```

```r
# Convert to data frame

hp_num <- as.data.table(hp_num)


# Filter only numeric columns using .SD
hp_num <- hp_num[, .SD, .SDcols = sapply(hp_num, is.numeric)]

# Check the structure of the cleaned numeric data
str(hp_num)                                          # 32 num cols
```

```
## Classes 'data.table' and 'data.frame':   1439 obs. of  31 variables:
##  $ LotFrontage  : num  -0.2156 0.508 -0.0709 -0.4568 0.701 ...
##  $ LotArea      : num  -0.231 -0.0681 0.1658 -0.0751 0.5923 ...
##  $ YearBuilt    : num  1.049 0.157 0.983 -1.857 0.95 ...
##  $ YearRemodAdd : num  0.876 -0.432 0.827 -0.722 0.731 ...
##  $ MasVnrArea   : num  0.527 -0.573 0.336 -0.573 1.391 ...
##  $ BsmtFinSF1   : num  0.627 1.257 0.117 -0.509 0.509 ...
##  $ BsmtFinSF2   : num  -0.286 -0.286 -0.286 -0.286 -0.286 ...
##  $ BsmtUnfSF    : num  -0.9449 -0.6423 -0.3035 -0.0641 -0.177 ...
##  $ TotalBsmtSF  : num  -0.465 0.509 -0.311 -0.705 0.228 ...
##  $ X1stFlrSF    : num  -0.8042 0.2819 -0.633 -0.5233 -0.0311 ...
##  $ X2ndFlrSF    : num  1.173 -0.797 1.201 0.947 1.632 ...
##  $ LowQualFinSF : num  -0.121 -0.121 -0.121 -0.121 -0.121 ...
##  $ GrLivArea    : num  0.398 -0.484 0.547 0.411 1.358 ...
##  $ BsmtFullBath : num  1.132 -0.822 1.132 1.132 1.132 ...
##  $ BsmtHalfBath : num  -0.24 3.97 -0.24 -0.24 -0.24 ...
##  $ FullBath     : num  0.796 0.796 0.796 -1.034 0.796 ...
##  $ HalfBath     : num  1.24 -0.76 1.24 -0.76 1.24 ...
##  $ BedroomAbvGr : num  0.163 0.163 0.163 0.163 1.399 ...
##  $ KitchenAbvGr : num  -0.211 -0.211 -0.211 -0.211 -0.211 ...
##  $ TotRmsAbvGrd : num  0.93 -0.315 -0.315 0.307 1.552 ...
##  $ Fireplaces   : num  -0.948 0.618 0.618 0.618 0.618 ...
##  $ GarageCars   : num  0.321 0.321 0.321 1.661 1.661 ...
##  $ GarageArea   : num  0.3691 -0.0485 0.6538 0.8152 1.7358 ...
##  $ WoodDeckSF   : num  -0.758 1.684 -0.758 -0.758 0.816 ...
##  $ OpenPorchSF  : num  0.2171 -0.7039 -0.0698 -0.1755 0.5644 ...
##  $ EnclosedPorch: num  -0.36 -0.36 -0.36 4.07 -0.36 ...
##  $ X3SsnPorch   : num  -0.117 -0.117 -0.117 -0.117 -0.117 ...
##  $ ScreenPorch  : num  -0.271 -0.271 -0.271 -0.271 -0.271 ...
##  $ PoolArea     : num  -0.0585 -0.0585 -0.0585 -0.0585 -0.0585 ...
##  $ MiscVal      : num  -0.14 -0.14 -0.14 -0.14 -0.14 ...
##  $ YrSold       : num  0.137 -0.617 0.137 -1.371 0.137 ...
##  - attr(*, ".internal.selfref")=<externalptr>
```

```
#covariance matrix
cov_matrix <- cov(hp_num)

# Display  matrix
print(round(cov_matrix, 3))
```

```
##               LotFrontage LotArea YearBuilt YearRemodAdd MasVnrArea BsmtFinSF1
## LotFrontage         1.000   0.314     0.118        0.091      0.164      0.151
## LotArea             0.314   1.000     0.021        0.050      0.138      0.170
## YearBuilt           0.118   0.021     1.000        0.593      0.311      0.249
## YearRemodAdd        0.091   0.050     0.593        1.000      0.174      0.125
## MasVnrArea          0.164   0.138     0.311        0.174      1.000      0.239
## BsmtFinSF1          0.151   0.170     0.249        0.125      0.239      1.000
## BsmtFinSF2          0.037   0.054    -0.050       -0.065     -0.072     -0.060
## BsmtUnfSF           0.128   0.044     0.154        0.185      0.115     -0.515
## TotalBsmtSF         0.306   0.244     0.402        0.301      0.342      0.465
## X1stFlrSF           0.380   0.310     0.283        0.243      0.321      0.392
## X2ndFlrSF           0.065   0.081     0.003        0.138      0.157     -0.164
## LowQualFinSF        0.042   0.014    -0.185       -0.063     -0.069     -0.066
## GrLivArea           0.339   0.299     0.193        0.290      0.363      0.143
## BsmtFullBath        0.077   0.089     0.185        0.121      0.082      0.653
## BsmtHalfBath       -0.009   0.077    -0.038       -0.010      0.014      0.073
## FullBath            0.190   0.162     0.472        0.441      0.266      0.055
## HalfBath            0.039   0.026     0.240        0.183      0.194     -0.012
## BedroomAbvGr        0.257   0.173    -0.073       -0.043      0.092     -0.111
## KitchenAbvGr       -0.001  -0.016    -0.177       -0.151     -0.037     -0.083
## TotRmsAbvGrd        0.317   0.226     0.091        0.192      0.269      0.015
## Fireplaces          0.214   0.285     0.142        0.114      0.236      0.233
## GarageCars          0.283   0.189     0.543        0.426      0.364      0.224
## GarageArea          0.312   0.204     0.489        0.382      0.367      0.271
## WoodDeckSF          0.087   0.143     0.227        0.214      0.153      0.180
## OpenPorchSF         0.116   0.127     0.185        0.225      0.113      0.090
## EnclosedPorch       0.018  -0.009    -0.388       -0.197     -0.109     -0.101
## X3SsnPorch          0.069   0.036     0.032        0.045      0.020      0.030
## ScreenPorch         0.040   0.073    -0.052       -0.037      0.064      0.069
## PoolArea            0.105   0.049    -0.008       -0.001     -0.027      0.050
## MiscVal            -0.003   0.034    -0.078       -0.046     -0.030     -0.007
## YrSold              0.006  -0.038    -0.010        0.043     -0.006      0.017
##               BsmtFinSF2 BsmtUnfSF TotalBsmtSF X1stFlrSF X2ndFlrSF LowQualFinSF
## LotFrontage        0.037     0.128       0.306     0.380     0.065        0.042
## LotArea            0.054     0.044       0.244     0.310     0.081        0.014
## YearBuilt         -0.050     0.154       0.402     0.283     0.003       -0.185
## YearRemodAdd      -0.065     0.185       0.301     0.243     0.138       -0.063
## MasVnrArea        -0.072     0.115       0.342     0.321     0.157       -0.069
## BsmtFinSF1        -0.060    -0.515       0.465     0.392    -0.164       -0.066
## BsmtFinSF2         1.000    -0.206       0.104     0.095    -0.105        0.015
## BsmtUnfSF         -0.206     1.000       0.450     0.340     0.003        0.028
## TotalBsmtSF        0.104     0.450       1.000     0.804    -0.207       -0.033
## X1stFlrSF          0.095     0.340       0.804     1.000    -0.229       -0.013
## X2ndFlrSF         -0.105     0.003      -0.207    -0.229     1.000        0.065
## LowQualFinSF       0.015     0.028      -0.033    -0.013     0.065        1.000
## GrLivArea         -0.018     0.256       0.412     0.539     0.691        0.142
## BsmtFullBath       0.151    -0.417       0.291     0.224    -0.174       -0.047
## BsmtHalfBath       0.075    -0.103      -0.005    -0.002    -0.035       -0.006
```

```
## FullBath          -0.088     0.294       0.335     0.390     0.415       -0.001
## HalfBath          -0.035    -0.041      -0.070    -0.140     0.608       -0.027
## BedroomAbvGr      -0.025     0.167       0.053     0.132     0.499        0.107
## KitchenAbvGr      -0.044     0.029      -0.072     0.073     0.055        0.008
## TotRmsAbvGrd      -0.048     0.258       0.271     0.403     0.610        0.135
## Fireplaces         0.037     0.058       0.317     0.394     0.184       -0.020
## GarageCars        -0.040     0.222       0.452     0.447     0.186       -0.094
## GarageArea        -0.018     0.193       0.479     0.479     0.138       -0.068
## WoodDeckSF         0.073     0.009       0.224     0.224     0.091       -0.025
## OpenPorchSF        0.004     0.129       0.232     0.198     0.205        0.018
## EnclosedPorch      0.037    -0.005      -0.095    -0.064     0.062        0.061
## X3SsnPorch        -0.030     0.021       0.042     0.060    -0.024       -0.005
## ScreenPorch        0.079    -0.010       0.091     0.094     0.031        0.028
## PoolArea           0.024    -0.028       0.031     0.061     0.056        0.072
## MiscVal           -0.022    -0.049      -0.069    -0.049     0.011        0.002
## YrSold             0.031    -0.042      -0.014    -0.011    -0.023       -0.029
##               GrLivArea BsmtFullBath BsmtHalfBath FullBath HalfBath
## LotFrontage       0.339        0.077       -0.009    0.190    0.039
## LotArea           0.299        0.089        0.077    0.162    0.026
## YearBuilt         0.193        0.185       -0.038    0.472    0.240
## YearRemodAdd      0.290        0.121       -0.010    0.441    0.183
## MasVnrArea        0.363        0.082        0.014    0.266    0.194
## BsmtFinSF1        0.143        0.653        0.073    0.055   -0.012
## BsmtFinSF2       -0.018        0.151        0.075   -0.088   -0.035
## BsmtUnfSF         0.256       -0.417       -0.103    0.294   -0.041
## TotalBsmtSF       0.412        0.291       -0.005    0.335   -0.070
## X1stFlrSF         0.539        0.224       -0.002    0.390   -0.140
## X2ndFlrSF         0.691       -0.174       -0.035    0.415    0.608
## LowQualFinSF      0.142       -0.047       -0.006   -0.001   -0.027
## GrLivArea         1.000        0.012       -0.032    0.641    0.413
## BsmtFullBath      0.012        1.000       -0.146   -0.064   -0.036
## BsmtHalfBath     -0.032       -0.146        1.000   -0.061   -0.013
## FullBath          0.641       -0.064       -0.061    1.000    0.136
## HalfBath          0.413       -0.036       -0.013    0.136    1.000
## BedroomAbvGr      0.534       -0.145        0.043    0.350    0.227
## KitchenAbvGr      0.101       -0.038       -0.037    0.134   -0.079
## TotRmsAbvGrd      0.830       -0.062       -0.028    0.550    0.336
## Fireplaces        0.445        0.119        0.025    0.239    0.196
## GarageCars        0.478        0.125       -0.021    0.480    0.220
## GarageArea        0.463        0.167       -0.025    0.421    0.162
## WoodDeckSF        0.240        0.152        0.036    0.206    0.103
## OpenPorchSF       0.323        0.064       -0.024    0.261    0.195
## EnclosedPorch     0.012       -0.045       -0.007   -0.119   -0.099
## X3SsnPorch        0.023        0.001        0.036    0.036   -0.005
## ScreenPorch       0.098        0.022        0.035   -0.022    0.067
## PoolArea          0.100        0.038        0.028    0.023    0.002
## MiscVal          -0.027       -0.003        0.005   -0.024   -0.059
## YrSold           -0.030        0.069       -0.051   -0.012   -0.003
##               BedroomAbvGr KitchenAbvGr TotRmsAbvGrd Fireplaces GarageCars
```

```
## LotFrontage          0.257       -0.001        0.317      0.214      0.283
## LotArea              0.173       -0.016        0.226      0.285      0.189
## YearBuilt           -0.073       -0.177        0.091      0.142      0.543
## YearRemodAdd        -0.043       -0.151        0.192      0.114      0.426
## MasVnrArea           0.092       -0.037        0.269      0.236      0.364
## BsmtFinSF1          -0.111       -0.083        0.015      0.233      0.224
## BsmtFinSF2          -0.025       -0.044       -0.048      0.037     -0.040
## BsmtUnfSF            0.167        0.029        0.258      0.058      0.222
## TotalBsmtSF          0.053       -0.072        0.271      0.317      0.452
## X1stFlrSF            0.132        0.073        0.403      0.394      0.447
## X2ndFlrSF            0.499        0.055        0.610      0.184      0.186
## LowQualFinSF         0.107        0.008        0.135     -0.020     -0.094
## GrLivArea            0.534        0.101        0.830      0.445      0.478
## BsmtFullBath        -0.145       -0.038       -0.062      0.119      0.125
## BsmtHalfBath         0.043       -0.037       -0.028      0.025     -0.021
## FullBath             0.350        0.134        0.550      0.239      0.480
## HalfBath             0.227       -0.079        0.336      0.196      0.220
## BedroomAbvGr         1.000        0.198        0.676      0.102      0.091
## KitchenAbvGr         0.198        1.000        0.253     -0.126     -0.051
## TotRmsAbvGrd         0.676        0.253        1.000      0.314      0.367
## Fireplaces           0.102       -0.126        0.314      1.000      0.300
## GarageCars           0.091       -0.051        0.367      0.300      1.000
## GarageArea           0.071       -0.065        0.335      0.261      0.888
## WoodDeckSF           0.059       -0.092        0.170      0.190      0.217
## OpenPorchSF          0.092       -0.070        0.228      0.162      0.217
## EnclosedPorch        0.039        0.031        0.001     -0.021     -0.151
## X3SsnPorch          -0.025       -0.025       -0.006      0.013      0.037
## ScreenPorch          0.031       -0.052        0.045      0.180      0.052
## PoolArea             0.053       -0.012        0.031      0.053      0.018
## MiscVal              0.022        0.051       -0.008      0.023     -0.074
## YrSold              -0.032        0.034       -0.026     -0.018     -0.033
##               GarageArea WoodDeckSF OpenPorchSF EnclosedPorch X3SsnPorch
## LotFrontage        0.312      0.087       0.116         0.018      0.069
## LotArea            0.204      0.143       0.127        -0.009      0.036
## YearBuilt          0.489      0.227       0.185        -0.388      0.032
## YearRemodAdd       0.382      0.214       0.225        -0.197      0.045
## MasVnrArea         0.367      0.153       0.113        -0.109      0.020
## BsmtFinSF1         0.271      0.180       0.090        -0.101      0.030
## BsmtFinSF2        -0.018      0.073       0.004         0.037     -0.030
## BsmtUnfSF          0.193      0.009       0.129        -0.005      0.021
## TotalBsmtSF        0.479      0.224       0.232        -0.095      0.042
## X1stFlrSF          0.479      0.224       0.198        -0.064      0.060
## X2ndFlrSF          0.138      0.091       0.205         0.062     -0.024
## LowQualFinSF      -0.068     -0.025       0.018         0.061     -0.005
## GrLivArea          0.463      0.240       0.323         0.012      0.023
## BsmtFullBath       0.167      0.152       0.064        -0.045      0.001
## BsmtHalfBath      -0.025      0.036      -0.024        -0.007      0.036
## FullBath           0.421      0.206       0.261        -0.119      0.036
## HalfBath           0.162      0.103       0.195        -0.099     -0.005
```

```
## BedroomAbvGr       0.071      0.059       0.092         0.039     -0.025
## KitchenAbvGr      -0.065     -0.092      -0.070         0.031     -0.025
## TotRmsAbvGrd       0.335      0.170       0.228         0.001     -0.006
## Fireplaces         0.261      0.190       0.162        -0.021      0.013
## GarageCars         0.888      0.217       0.217        -0.151      0.037
## GarageArea         1.000      0.212       0.239        -0.122      0.037
## WoodDeckSF         0.212      1.000       0.061        -0.125     -0.032
## OpenPorchSF        0.239      0.061       1.000        -0.092     -0.006
## EnclosedPorch     -0.122     -0.125      -0.092         1.000     -0.038
## X3SsnPorch         0.037     -0.032      -0.006        -0.038      1.000
## ScreenPorch        0.055     -0.069       0.069        -0.083     -0.032
## PoolArea           0.025      0.085       0.025         0.068     -0.007
## MiscVal           -0.058      0.000      -0.004         0.012      0.013
## YrSold            -0.022      0.026      -0.054        -0.009      0.019
##               ScreenPorch PoolArea MiscVal YrSold
## LotFrontage         0.040    0.105  -0.003  0.006
## LotArea             0.073    0.049   0.034 -0.038
## YearBuilt          -0.052   -0.008  -0.078 -0.010
## YearRemodAdd       -0.037   -0.001  -0.046  0.043
## MasVnrArea          0.064   -0.027  -0.030 -0.006
## BsmtFinSF1          0.069    0.050  -0.007  0.017
## BsmtFinSF2          0.079    0.024  -0.022  0.031
## BsmtUnfSF          -0.010   -0.028  -0.049 -0.042
## TotalBsmtSF         0.091    0.031  -0.069 -0.014
## X1stFlrSF           0.094    0.061  -0.049 -0.011
## X2ndFlrSF           0.031    0.056   0.011 -0.023
## LowQualFinSF        0.028    0.072   0.002 -0.029
## GrLivArea           0.098    0.100  -0.027 -0.030
## BsmtFullBath        0.022    0.038  -0.003  0.069
## BsmtHalfBath        0.035    0.028   0.005 -0.051
## FullBath           -0.022    0.023  -0.024 -0.012
## HalfBath            0.067    0.002  -0.059 -0.003
## BedroomAbvGr        0.031    0.053   0.022 -0.032
## KitchenAbvGr       -0.052   -0.012   0.051  0.034
## TotRmsAbvGrd        0.045    0.031  -0.008 -0.026
## Fireplaces          0.180    0.053   0.023 -0.018
## GarageCars          0.052    0.018  -0.074 -0.033
## GarageArea          0.055    0.025  -0.058 -0.022
## WoodDeckSF         -0.069    0.085   0.000  0.026
## OpenPorchSF         0.069    0.025  -0.004 -0.054
## EnclosedPorch      -0.083    0.068   0.012 -0.009
## X3SsnPorch         -0.032   -0.007   0.013  0.019
## ScreenPorch         1.000   -0.016   0.058  0.020
## PoolArea           -0.016    1.000  -0.008 -0.055
## MiscVal             0.058   -0.008   1.000  0.080
## YrSold              0.020   -0.055   0.080  1.000
```

The covariance matrix captures correlation between the features. Large positive values indicate strong positive relationships, while large negative values indicate strong negative relationships. Zero correlation indicates no observed relationship.

Perform PCA

```
# Compute Eigen values and vectors to identify PC's

# Perform Eigen decomposition

eigen_result <- eigen(cov_matrix)

eigenvalues <- eigen_result$values

eigenvectors <- eigen_result$vectors

length(eigenvalues)          # 33 PC's
```

```
## [1] 31
```

```
# Variance explained by each PC
variance_explained <- round(eigenvalues / sum(eigenvalues), 3)
cumulative_variance <- cumsum(variance_explained)

# Display the cumulative variance
print(cumulative_variance)
```

```
##  [1] 0.197 0.297 0.363 0.425 0.467 0.505 0.541 0.576 0.610 0.643 0.675 0.705
## [13] 0.735 0.764 0.792 0.819 0.843 0.866 0.887 0.907 0.925 0.943 0.956 0.968
## [25] 0.978 0.985 0.991 0.995 0.998 0.998 0.998
```

The eigenvalues indicate the amount of variance captured by each principal component. The eigenvector indicates the direction of the component. Cumulative variance helps us decide how many PC's to retain. The first 16 PC's explain 82% variance in the data, and hence we retain these and discard the remaining ones.

```
# Transform data using top PC's

# Convert to matrix for multiplication
house_data_matrix <- as.matrix(hp_num)

# Use first 16 PC's
pca_transformed <- house_data_matrix %*% eigenvectors[, 1:16]

# Convert to a data frame for easier handling
pca_df <- as.data.frame(pca_transformed)
colnames(pca_df) <- paste0("PC", 1:16)

# Display the head of the PCA-transformed data
head(pca_df)
```

	PC1 <dbl>	PC2 <dbl>	PC3 <dbl>	PC4 <dbl>	PC5 <dbl>	PC6 <dbl>	
1	-1.0927505	0.4318696	-1.1109834	2.33759342	-1.1309861	0.09449446	0
2	-0.2451810	-1.2151502	0.7267024	-0.13352588	1.2034257	0.88653404	-3
3	-1.2817053	0.2360112	-1.0321840	1.82581525	-0.3026119	-0.09207488	0
4	0.6460449	1.2074951	1.7082256	-0.04891827	0.1566745	1.27025829	1
5	-4.1728922	0.7909943	-0.2777810	1.87774377	-0.5326526	0.12543321	-0
6	0.8720654	-1.7511807	-0.6510089	1.82319651	-0.4008341	-1.91243033	-4

6 rows | 1-8 of 17 columns

The new dataset, pca_df, contains the transformed linear combinations (PC1, PC2,..PC16) in lower-dimensional space, that represents the original data in reduced columns and complexity, while preserving most of the variance in information. The scores are the result of multiplying the standardized data correlation matrix by the 32 eigenvectors or just the first 16 in this case. For instance, the first column(PC1), second column(PC2) and the third column(PC3) have a negative influence or correlation on the first observation. This also means that there are 16 axes(x,y,z..so on) and the first data point's transformed coordinates are (-1.33,-0.16,-1.41,…,0.12).

Check for Correlation

```
# Calculate the correlation matrix

round(cor(pca_df),3)
```

```
##       PC1 PC2 PC3 PC4 PC5 PC6 PC7 PC8 PC9 PC10 PC11 PC12 PC13 PC14 PC15 PC16
## PC1     1   0   0   0   0   0   0   0   0    0    0    0    0    0    0    0
## PC2     0   1   0   0   0   0   0   0   0    0    0    0    0    0    0    0
## PC3     0   0   1   0   0   0   0   0   0    0    0    0    0    0    0    0
## PC4     0   0   0   1   0   0   0   0   0    0    0    0    0    0    0    0
## PC5     0   0   0   0   1   0   0   0   0    0    0    0    0    0    0    0
## PC6     0   0   0   0   0   1   0   0   0    0    0    0    0    0    0    0
## PC7     0   0   0   0   0   0   1   0   0    0    0    0    0    0    0    0
## PC8     0   0   0   0   0   0   0   1   0    0    0    0    0    0    0    0
## PC9     0   0   0   0   0   0   0   0   1    0    0    0    0    0    0    0
## PC10    0   0   0   0   0   0   0   0   0    1    0    0    0    0    0    0
## PC11    0   0   0   0   0   0   0   0   0    0    1    0    0    0    0    0
## PC12    0   0   0   0   0   0   0   0   0    0    0    1    0    0    0    0
## PC13    0   0   0   0   0   0   0   0   0    0    0    0    1    0    0    0
## PC14    0   0   0   0   0   0   0   0   0    0    0    0    0    1    0    0
## PC15    0   0   0   0   0   0   0   0   0    0    0    0    0    0    1    0
## PC16    0   0   0   0   0   0   0   0   0    0    0    0    0    0    0    1
```

Since principal components are orthogonal to each other, they are uncorrelated with any other principal component except itself.

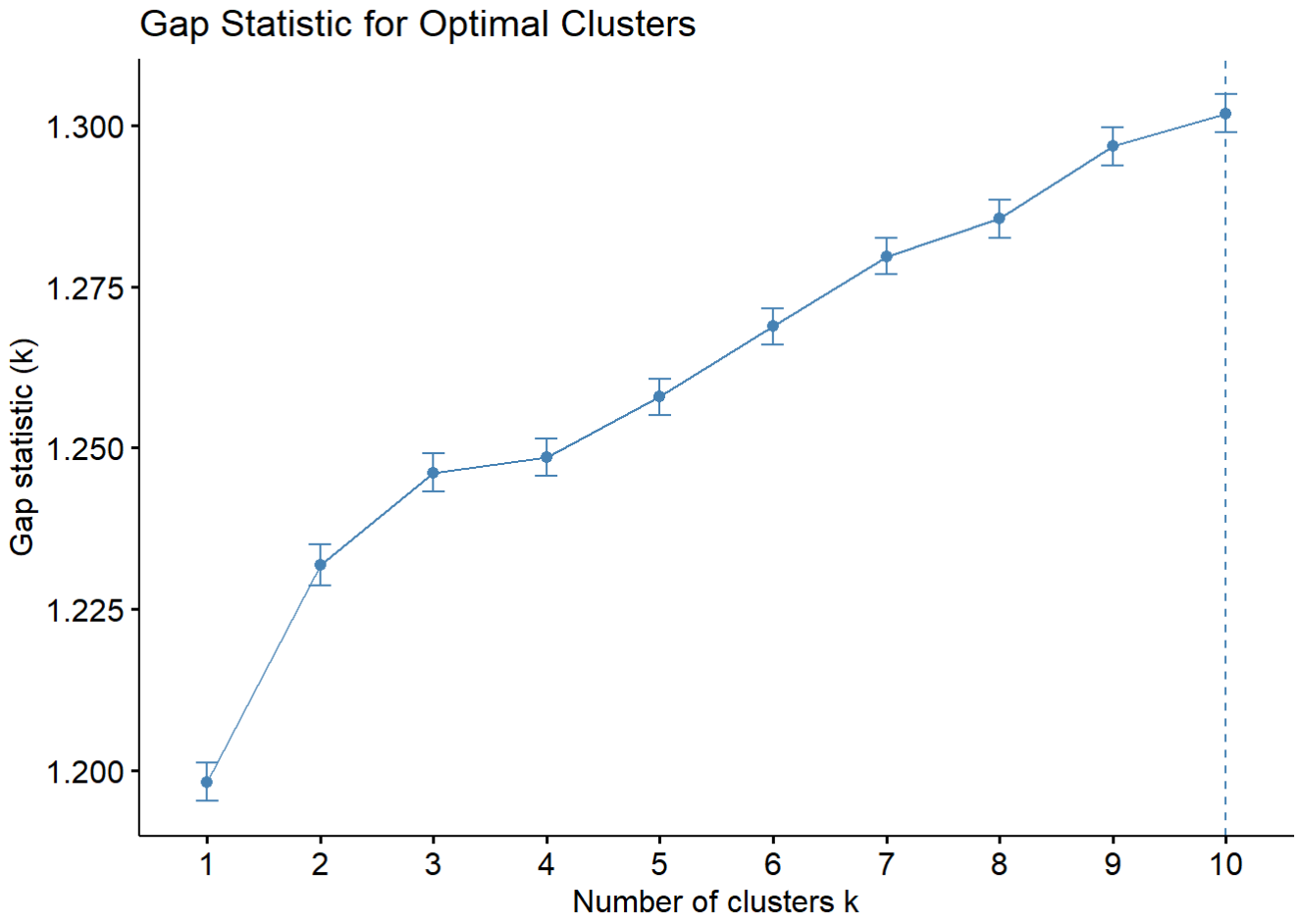
K-Means Clustering

Performing K-Means clustering on the pca dataset and evaluate using Silhouette Score and Elbow Method.

```
scaled_data <- hp_num

# Convert list to data frame
scaled_data <- as.data.frame(scaled_data)

# Checking for missing or infinite values in the dataset
any(is.na(scaled_data))     # Returns TRUE if there are any NAs
```

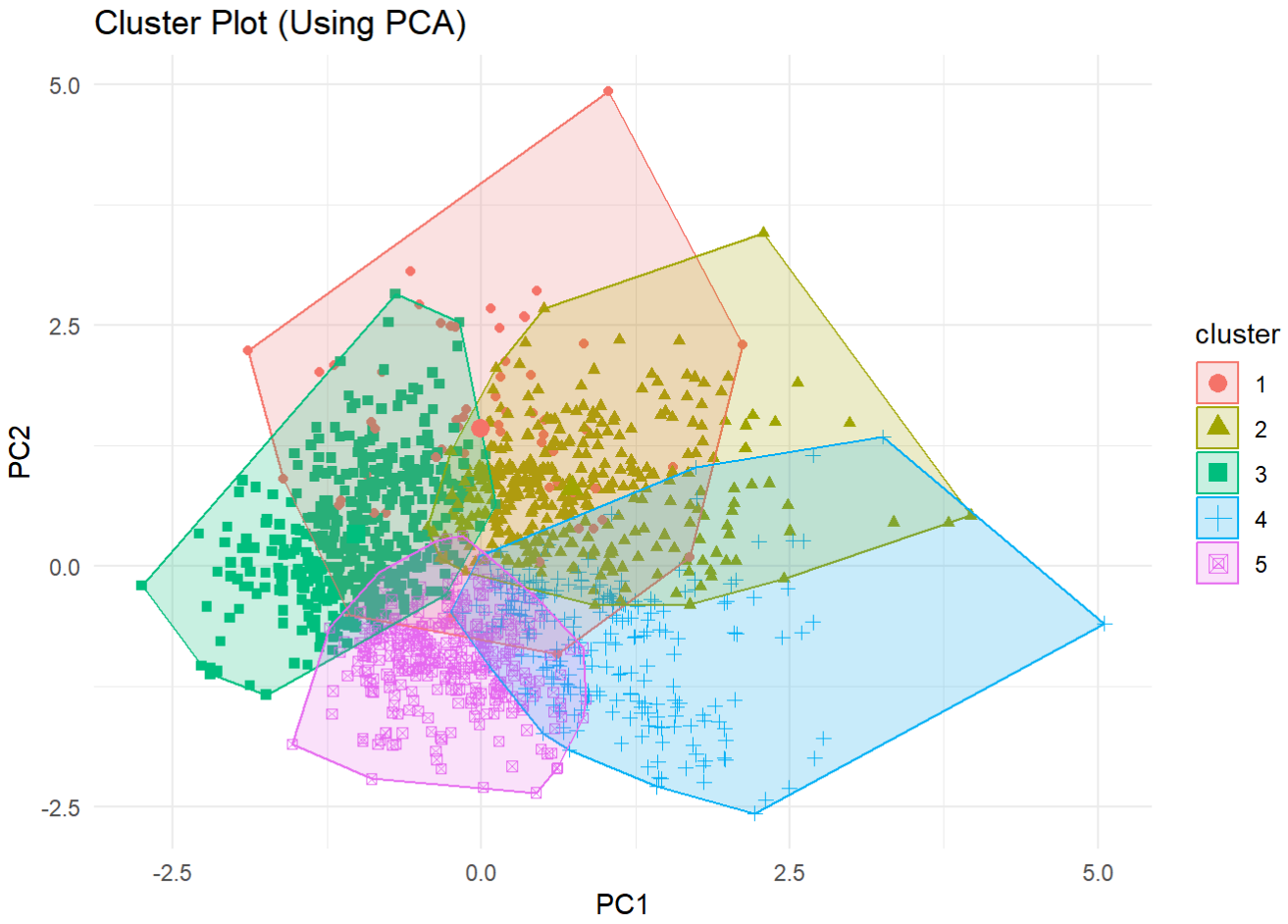
```
## [1] FALSE
```

```
# Removing rows with NA values
scaled_data <- na.omit(scaled_data)  # Removes rows with NA values

library(cluster)

# Compute gap statistic to find the optimal number of clusters
gap_stat <- clusGap(scaled_data, FUN = function(x, k) kmeans(x, centers = k, nstart
= 25, iter.max = 100), K.max = 10, B = 50)
fviz_gap_stat(gap_stat) +
  labs(title = "Gap Statistic for Optimal Clusters")
```



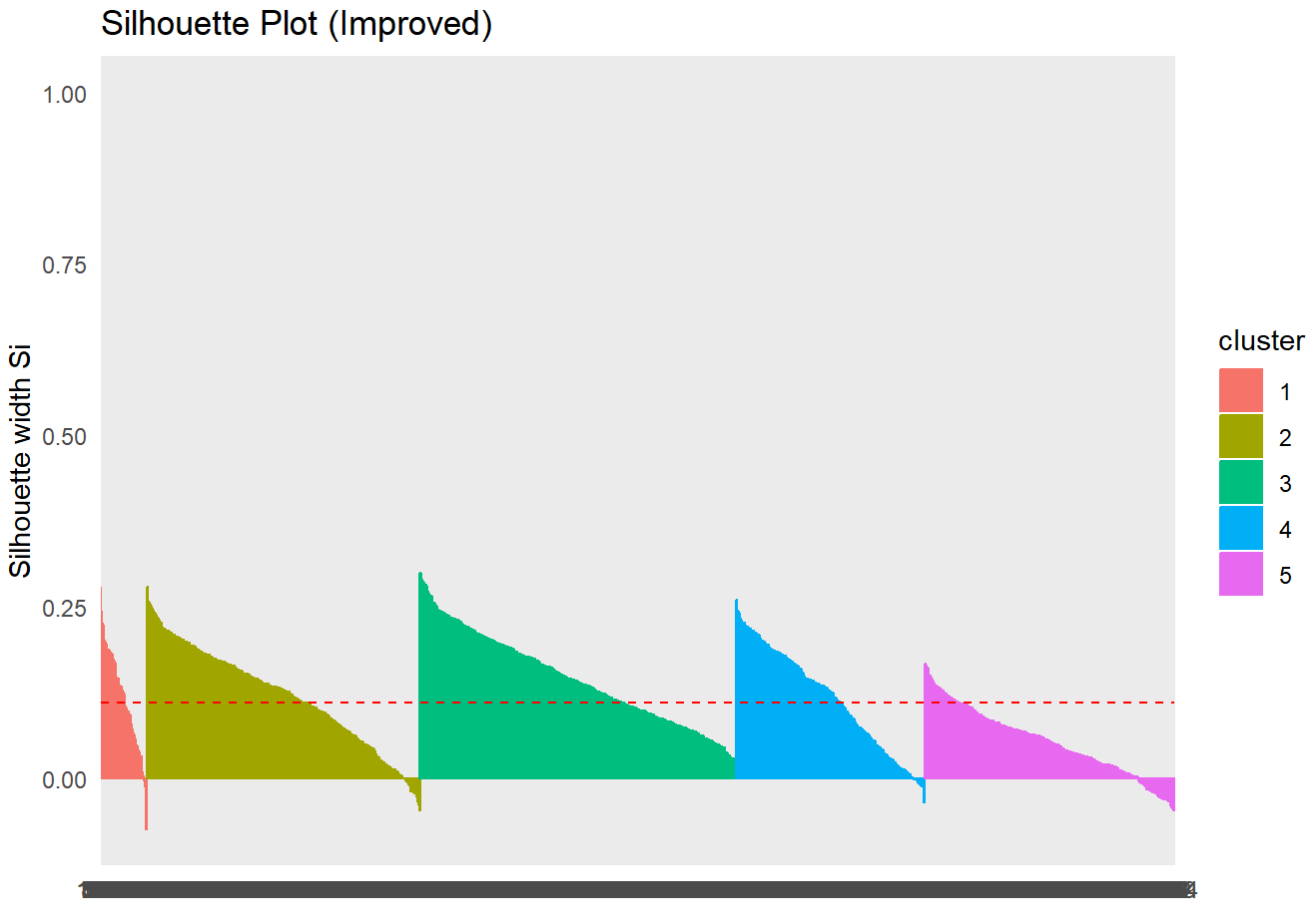
Visualizing Clusters

```
# Perform K-Means Clustering with number of clusters/centers as 5 determined above b
y elbow method
kmeans_model <- kmeans(scaled_data, centers = 5, nstart = 25)

scaled_data <- scale(scaled_data)

library(factoextra)
library(ggplot2)

# Perform PCA on the scaled dataset
pca_result <- prcomp(scaled_data, center = TRUE, scale. = TRUE)

# Visualize clusters using PCA
fviz_cluster(kmeans_model, geom = "point", data = pca_result$x[, 1:2]) +
  labs(title = "Cluster Plot (Using PCA)", x = "PC1", y = "PC2") +
  theme_minimal()
```



```
# Recompute silhouette scores with the optimal clusters
silhouette_score <- silhouette(kmeans_model$cluster, dist(scaled_data))

# Visualize silhouette scores
fviz_silhouette(silhouette_score) +
  labs(title = "Silhouette Plot (Improved)") +
  theme_minimal()
```

```
##   cluster size ave.sil.width
## 1       1   62          0.12
## 2       2  366          0.12
## 3       3  423          0.14
## 4       4  253          0.12
## 5       5  335          0.05
```



In this section we are only using the first two dimensions of the PCA dataset to get a cluster plot for K-Means clustering that is less scattered and more visually interpretative. In particular, the Silhouette Plot (Improved) shows how each cluster's value for the Silhouette Score varies along the dataset/observations while the distances from the centroid for each point can also be seen from the cluster_summary's summary. From the Silhouette Plot (Improved) it can be seen that the width has gone close to 0.30 for some points and it is negative only for the 3rd and 5th cluster at the end.

Extracting Important Features from Clusters

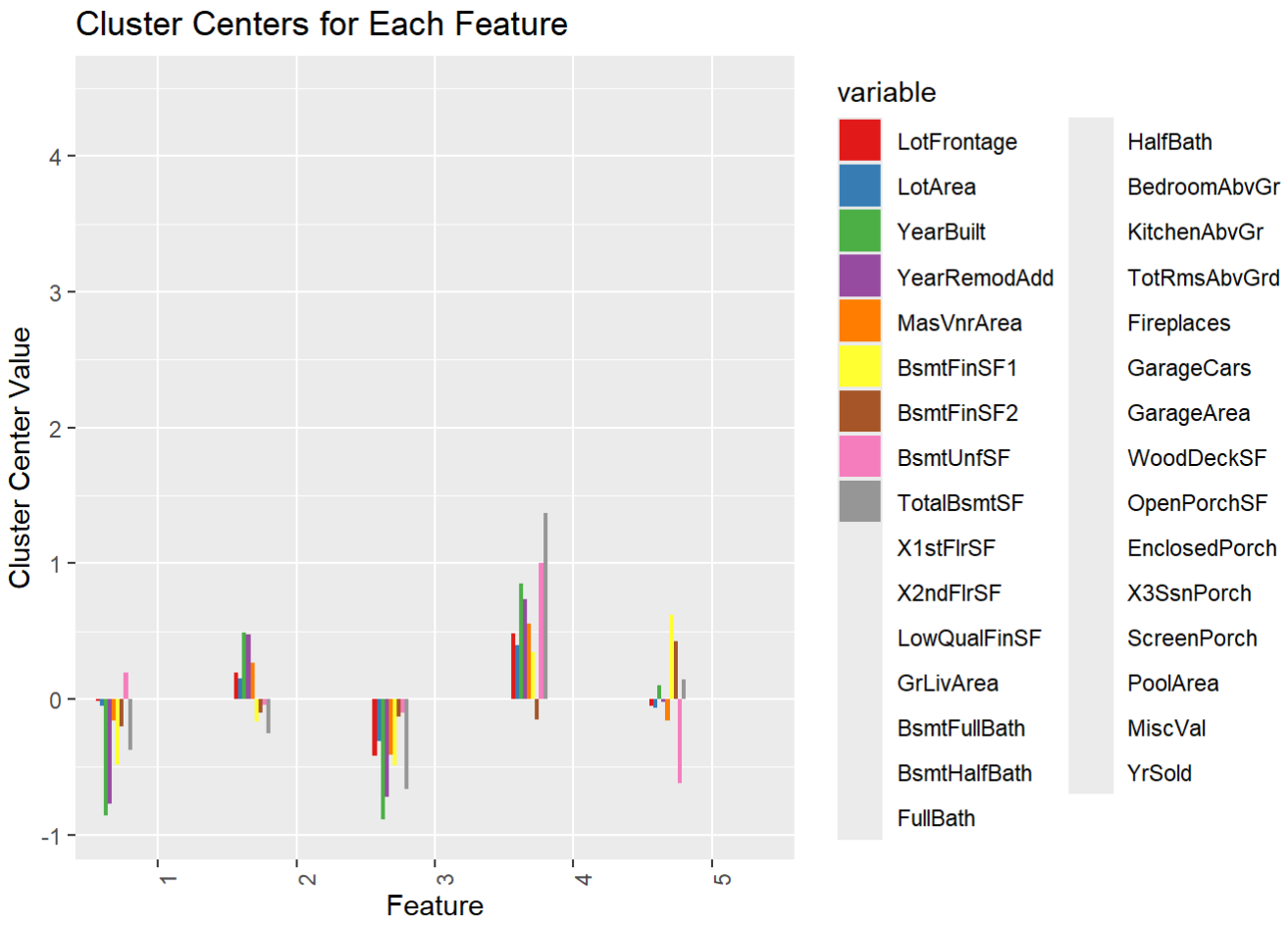
```
centers <- kmeans_model$centers

# Calculate the range (max - min) for each feature
feature_ranges <- apply(centers, 2, function(x) max(x) - min(x))

# Sort the features by their range (descending order)
feature_ranges_sorted <- sort(feature_ranges, decreasing = TRUE)

feature_ranges_sorted
```

```
##  KitchenAbvGr     X1stFlrSF     X2ndFlrSF   TotalBsmtSF  TotRmsAbvGrd
##     4.6933538     2.0745730     2.0617246     2.0332200     1.8576901
##    GarageCars     YearBuilt  BedroomAbvGr    GarageArea     BsmtUnfSF
##     1.7503981     1.7317592     1.6988999     1.6801577     1.6269789
##      FullBath     GrLivArea      HalfBath  YearRemodAdd  BsmtFullBath
##     1.5828882     1.5755928     1.5255127     1.5048435     1.2818629
##    BsmtFinSF1    Fireplaces    MasVnrArea   LotFrontage    WoodDeckSF
##     1.1150137     1.0834335     0.9653702     0.9050892     0.8348706
##   OpenPorchSF       LotArea    BsmtFinSF2 EnclosedPorch   ScreenPorch
##     0.7398410     0.7014345     0.6252513     0.6239674     0.4642166
##  BsmtHalfBath       MiscVal    X3SsnPorch  LowQualFinSF      PoolArea
##     0.4189822     0.2934248     0.2776250     0.2380399     0.1371215
##        YrSold
##     0.1269179
```

The code above shows which features vary the most between clusters. Features that vary the most have the biggest influence in differentating between clusters. In the context of the Ames, Iowa dataset, this means that `KitchenAbvGr`, `XLstFlrSF`, and `TotalBsmtSF` play the biggest influences in differentiating clusters.

Visualization of some of the most important features

```
# Convert the cluster centers to a data frame for easier visualization
centers_df <- as.data.frame(centers)

# Create a bar plot of the feature ranges across clusters
library(ggplot2)

centers_df$Feature <- rownames(centers_df)
centers_df_long <- reshape2::melt(centers_df, id.vars = "Feature")

ggplot(centers_df_long, aes(x = Feature, y = value, fill = variable)) +
  geom_bar(stat = "identity", position = "dodge") +
  labs(title = "Cluster Centers for Each Feature", x = "Feature", y = "Cluster Cente
r Value") +
  theme(axis.text.x = element_text(angle = 90, hjust = 1)) +
  scale_fill_brewer(palette = "Set1")
```



This graph helps to visualize the most important features of the clusters. A larger bar in either direction indicates that the feature plays an very influential role in differentiating the clusters. The bars that are colored represent the largest difference. In the context of Ames, Iowa, the basement variables followed and “Lot” variables have a big influence. An interesting thing is that all these variables describe area in some form or other which suggests that “area” has an influence on the clusters. After that “YearRemodAd” and “Year Built” have an influence which suggests that the age of the home and rennovating the home play another influence in the clusters.

Artificial Neural Networks

Partition Dataset

```
# Designate a shortened name MLP for the MultilayerPercentron ANN method in RWeka
MLP <- make_Weka_classifier("weka/classifiers/functions/MultilayerPerceptron")

# Partititon Dataset via 80/20 split
set.seed(100)
row_indices <- createDataPartition(hpc$SalePrice,p=0.8,list=FALSE)

# Assign rows to train and test set respectively
train_set <- hpc_01[row_indices,] # 80% of data should be in train
test_set <- hpc_01[-row_indices,] # 20 % of data should be in test

nrow(train_set) # Display Number of rows
```

```
## [1] 1153
```

```
nrow(test_set) # Display Number of rows
```

```
## [1] 286
```

```
total_rows <- nrow(train_set) + nrow(test_set)
total_rows
```

```
## [1] 1439
```

The partition is successful, the number of rows from the train set (1153) and the test set(286) is equal to 1439 rows.

Set up Cores

```
num_cores <- detectCores()
num_cores <- num_cores - 1
```

Cores will be setup to help with processing models faster.

First Model

```
tic()
doParallel::registerDoParallel(cores = num_cores)
ANN_00 <- MLP(SalePrice ~ . , data = train_set)
toc()
```

```
## 443.53 sec elapsed
```

Generate Predictions and Metrics

```
# Generate Predictions
#tr = train
ANN_00_train <- predict(ANN_00,newdata = train_set)
# te = test
ANN_00_test <- predict(ANN_00, test_set)

# In the Weka model, there are default paramters which are list down below
#l <- 0.3 - learn rate
#m <- 0.2 - momentum
#n <-500 - number of epochs/passes
#h <- 'a' - #The hidden nodes to be created on each layer:
  # an integer, or the letters 'a' = (# of attribs + # of classes) / 2,
  #'i' = # of attribs, 'o' = # of classes, 't' = (# of attribs + # of classes)
  # default of H is 'a'.
```

Train Metrics

```
ANN_00_tr_met <- mmetric(train_set$SalePrice,ANN_00_train,metric = metrics_list)
round_metrics(ANN_00_tr_met)
```

```
## $MAE
## [1] 15404.57
##
## $RMSE
## [1] 18087.08
##
## $R2
## [1] 0.97
##
## $COR
## [1] 0.99
##
## $MAPE
## [1] 9.41
##
## $RMSPE
## [1] 1.11
##
## $RAE
## [1] 27.21
##
## $RRSE
## [1] 23.41
```

The model seems to be doing good on the train set with an R^2 of 96%. This means that the model is able to explain 96% of the variation within the data.

The RMSE also seems fairly decent at 20666.28, dollars. This means that the model's predictions are of on average by 20666.28 dollars.

Test Metrics

```
ANN_00_te_met <- mmetric(test_set$SalePrice,ANN_00_test,metric = metrics_list)
round_metrics(ANN_00_te_met)
```

```
## $MAE
## [1] 27321.19
##
## $RMSE
## [1] 37500.45
##
## $R2
## [1] 0.8
##
## $COR
## [1] 0.9
##
## $MAPE
## [1] 15.1
##
## $RMSPE
## [1] 1.93
##
## $RAE
## [1] 48.91
##
## $RRSE
## [1] 47.73
```

It appears that the model is over-fitting when it's cross-validated with the test set. For instance, R^2 sees a large decrease of 14% from the train to the test set. (train_set R^2: 96%, test_set R^2: 82%)

The RMSE however sees a large increase of 15,288 dollars from the train to the test sets:

- train_set RMSE: 20666.28
- test_set RMSE: 35954.44

This means that the model is picking up on noise in the data and is very complicated. A potential reason could be that the model has 171 hidden nodes on 1 layer.

Second Model

```
# These are the default values for MLP, except for h
l <- 0.3
m <- 0.2
n <-500
h <- '86' # Split the number of hidden nodes by half (171/2)
# 86 means 1 layer with 86 nodes

tic()
doParallel::registerDoParallel(cores = num_cores)
ANN_02 <- MLP(SalePrice ~ . , data = train_set,control= Weka_control(L = l, M = m, N
= n, H = h))
toc()
```

```
## 259.18 sec elapsed
```

```
summary(ANN_02)
```

```
##
## === Summary ===
##
## Correlation coefficient                  0.9897
## Mean absolute error                   8734.2005
## Root mean squared error              11225.2304
## Relative absolute error                 15.4286 %
## Root relative squared error             14.5264 %
## Total Number of Instances             1153
```

Since the first model moderately overfit, we'll attempt to simplify this model by reducing the number of nodes on the hidden layer. We can do this by cutting the number of nodes in half. If one looks at the first model there were 171 nodes, so we'll cut this in half and train the model.

Generate Predictions and Train Metrics

```
ANN_02_train <- predict(ANN_02,newdata = train_set)
ANN_02_test <- predict(ANN_02,newdata= test_set)

ANN_02_tr_met <- mmetric(train_set$SalePrice,ANN_02_train,metric = metrics_list)
round_metrics(ANN_02_tr_met)
```

```
## $MAE
## [1] 7959.22
##
## $RMSE
## [1] 10197.32
##
## $R2
## [1] 0.98
##
## $COR
## [1] 0.99
##
## $MAPE
## [1] 5.01
##
## $RMSPE
## [1] 0.68
##
## $RAE
## [1] 14.06
##
## $RRSE
## [1] 13.2
```

Test Metrics

```
ANN_02_te_met <- mmetric(test_set$SalePrice,ANN_02_test,metric = metrics_list)
round_metrics(ANN_02_te_met)
```

```
## $MAE
## [1] 25167.33
##
## $RMSE
## [1] 37369.47
##
## $R2
## [1] 0.78
##
## $COR
## [1] 0.88
##
## $MAPE
## [1] 14.19
##
## $RMSPE
## [1] 2.05
##
## $RAE
## [1] 45.06
##
## $RRSE
## [1] 47.57
```

Reducing the nodes by half on the hidden layer has caused the model to overfit even more severely.

This model's RMSE is:

- train_set RMSE: 14404.63
- test_set RMSE: 43047.93
- difference: 28,643 dollars

The RMSE values are much higher than the first model's RMSE Values which are listed down below for reference.

First model's RMSE:

- train_set RMSE: 20666.28
- test_set RMSE: 35954.44
- difference: 15,288 dollars

Additionally the R^2 on the first model was:

- train R^2: 96%
- test R^2: 82%
- difference: 14%

R^2 on second model for train and test:

- train R^2: 98%
- test R^2: 75%
- difference: 23%

These metrics indicate that the number of nodes should not be reduced so drastically since it causes a large decline in model performance. The model actually overfits more.

Grid Search

It's quite apparent based of the previous neural networks that there are several different hyper-parameters from the Learn and Momentum Rates to the number of hidden nodes. It would take an incredibly long time to experiment with all of these hyper-parameters to find a model that is generalizable.

Some of the hyper-parameters cause the model to severely underfit while other hyper-parameters cause the model to overfit.

It's also very computationally expensive to experiment and select the model with the best hyper-parameters. Thus a more efficient method is required to find a model that generalizes well to new data and does not or underfit.

This method is Grid Search which we can use to find a model that generalizes well to new data. Grid-Search will search through various hyperparameter combinations, till it finds the best model based on a metric that we specify.

AVNET

We'll try AVNET with the decay parameters to see if the model performance can be improved. The decay parameters act as regularization a parameter to prevent overfitting.

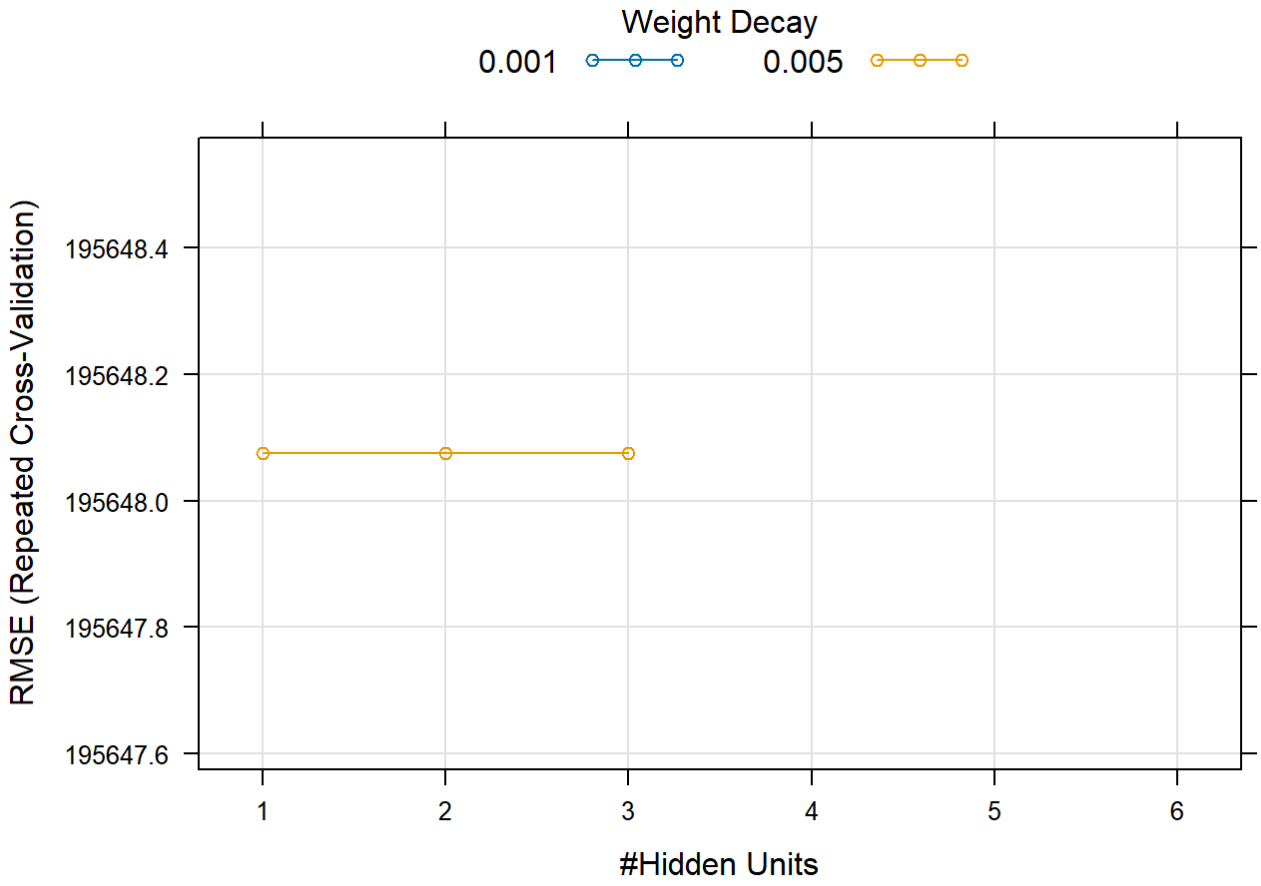
The implementation of AVNET also involves grid search.

```
train_predictor <- train_set[,-79] # SalePrice is col 80 in data-set
train_outcome <- train_set [,79]

test_predictor <- test_set[,-79]
test_outcome <- test_set[,79]


ctrl <- trainControl(method = "repeatedcv",   # cross-validation
                     number = 5,               # 5 folds
                     repeats = 5,              # 5 repetitions
                     savePredictions = "all"   # Save all predictions to help with d
iagnostics
)

avnnetGrid <- expand.grid(decay = c(.001,.005),
size = c(1:6),
bag = FALSE
)

tic()
doParallel::registerDoParallel(cores = num_cores)
avnnet_fit <- train(train_predictor, train_outcome,
method = "avNNet",
tuneGrid = avnnetGrid,
trControl = ctrl,
#preProc = c("center", "scale"),
trace = FALSE,
metric= "RMSE",
allowParallel=TRUE,
maxit = 100)
toc()
```

```
## 32.54 sec elapsed
```

This grid search was alright however the model is unable to perform a more complex grid search with more nodes. (If we try a size of 10-20, the algorithm starts to crash)

```
plot(avnnet_fit)
```




```
predicted_outcome <- predict(avnnet_fit, newdata = test_predictor)
# Evaluate performance
metrics <- postResample(pred = predicted_outcome, obs = test_outcome)

# Print the metrics
print(metrics)
```

```
##     RMSE Rsquared      MAE
## 196813.9       NA 180454.6
```

The algorithm was only able to build out 3 models that all had the same RMSE.

- This is probably because AVnet is a lightweight neural net implementation. Thus, a more robust implementation is needed.

RWeka Grid Search

RWeka is a more robust implementation, that can handle more complex models. Thus, this implementation will be used in combination with Grid Search to find the best hyper-parameters.

Implement Grid Search

```r
train_predictor <- train_set[,-79] # SalePrice is col 80 in data-set
train_outcome <- train_set [,79]

test_predictor <- test_set[,-79]
test_outcome <- test_set[,79]
# Define parameter grid
learning_rates <- c(0.1, 0.01, 0.001)
momentums <- c(0.2, 0.5, 0.9)
hidden_layers <- c("5", "10,5", "20","32,64,83")  # Different layer configurations
epochs <- c(100, 200)

# Placeholder for results
results <- data.frame(LearningRate = numeric(),
                      Momentum = numeric(),
                      HiddenLayers = character(),
                      Epochs = numeric(),
                      RMSE = numeric(),
                      stringsAsFactors = TRUE)

# Grid search
tic()
set.seed(100)  # Use this seed for consistency in future work
for (lr in learning_rates) {
  for (mom in momentums) {
    for (hl in hidden_layers) {
      for (epoch in epochs) {

        # Train model with current parameters
        model <- MLP(train_outcome ~ .,
                     data = train_predictor,
                     control = Weka_control(L = lr, M = mom, H = hl, N = epoch))

        # Evaluate performance (e.g., RMSE)
        predictions <- predict(model, test_set)
        rmse <- sqrt(mean((predictions - test_outcome)^2))

        # Store results
        results <- rbind(results, data.frame(
          LearningRate = lr,
          Momentum = mom,
          HiddenLayers = hl,
          Epochs = epoch,
          RMSE = rmse
        ))
      }
    }
  }
```

```
}
toc()
```

```
## 1519.11 sec elapsed
```

```
# Find best parameters
best_model <- results[which.min(results$RMSE), ]
print(best_model)
```

```
##    LearningRate Momentum HiddenLayers Epochs     RMSE
## 69        0.001      0.9           20    100 26198.78
```

Based on the Grid Search Results, the best model is number 58. This model is doing a great job at generalizing with a train R^2 of 93% and an RMSE of 23447.3 dollars. Test metrics must be computed before we can make an assessment on this model's generalizability.

Additionally, this is a relatively single ANN with a single layer that has 5 hidden neurons and a learn rate of .001. (The learn rate is a little higher, so the model will take a little longer to converge in other words.)

Calculate Test Metrics

```
# Extract best parameters
best_lr <- best_model$LearningRate
best_mom <- best_model$Momentum
best_hl <- best_model$HiddenLayers
best_epochs <- best_model$Epochs

# Train the model with the best parameters
final_model <- MLP(train_outcome ~ .,
                   data = train_set,
                   control = Weka_control(L = best_lr, M = best_mom, H = best_hl, N
= best_epochs))

final_predictions <- predict(final_model, test_set)

# Calculate RMSE
rmse <- sqrt(mean((final_predictions - test_outcome)^2))

# Calculate R-squared
sse <- sum((final_predictions - test_outcome)^2)  # Sum of Squared Errors
sst <- sum((test_outcome - mean(test_outcome))^2) # Total Sum of Squares
r_squared <- 1 - (sse / sst)

# Calculate MAE
mae <- mean(abs(final_predictions - test_outcome))

# Print metrics
cat("RMSE:", rmse, "\n")
```

```
## RMSE: 21464.91
```

```
cat("R-squared:", r_squared, "\n")
```

```
## R-squared: 0.9253478
```

```
cat("MAE:", mae, "\n")
```

```
## MAE: 13322.84
```

Calculate Train Set Metrics

```
train_predictions <- predict(final_model, train_set)

# Calculate RMSE
train_rmse <- sqrt(mean((train_predictions - train_outcome)^2))

# Calculate R-squared
train_sse <- sum((train_predictions - train_outcome)^2)  # Sum of Squared Errors
train_sst <- sum((train_outcome - mean(train_outcome))^2) # Total Sum of Squares
train_r_squared <- 1 - (train_sse / train_sst)

# Calculate MAE
train_mae <- mean(abs(train_predictions - train_outcome))

# Print metrics
cat("Training Set Metrics:\n")
```

```
## Training Set Metrics:
```

```
cat("RMSE:", train_rmse, "\n")
```

```
## RMSE: 14709.4
```

```
cat("R-squared:", train_r_squared, "\n")
```

```
## R-squared: 0.9637662
```

```
cat("MAE:", train_mae, "\n")
```

```
## MAE: 10196.26
```

Train vs Test Set Metrics

```
comparison_metrics <- data.frame(
  Dataset = c("Train", "Test"),
  RMSE = c(train_rmse, rmse),
  Rsquared = c(train_r_squared, r_squared),
  MAE = c(train_mae, mae)
)

print(comparison_metrics)
```

```
##   Dataset     RMSE  Rsquared      MAE
## 1   Train 14709.40 0.9637662 10196.26
## 2    Test 21464.91 0.9253478 13322.84
```

The table shows that this neural network's train and test metrics are very close. There is only a difference of 2% between the train and test sets.

The RMSE is also quite small at 799.01 dollars. This is very impressive considering that the house prices range from 34,900 dollars to 745,000 dollars.

5-fold valuation

```
cv_function_MLP <- function(df, target, nFolds, seedVal, metrics_list, l, m, n, h)
{
# create folds using the assigned values

set.seed(seedVal)
folds = createFolds(df[,target],nFolds)

# The lapply loop

cv_results <- lapply(folds, function(x)
{
# data preparation:

  test_target <- df[x,target]
  test_input <- df[x,-target]

  train_target <- df[-x,target]
  train_input <- df[-x,-target]
  pred_model <- MLP(train_target ~ .,data = train_input,control = Weka_control(L=l,M
=m, N=n,H=h))
  pred <- predict(pred_model, test_input)
  return(mmetric(test_target,pred,metrics_list))
})

cv_results_m <- as.matrix(as.data.frame(cv_results))
cv_mean<- as.matrix(rowMeans(cv_results_m))
cv_sd <- as.matrix(rowSds(cv_results_m))
colnames(cv_mean) <- "Mean"
colnames(cv_sd) <- "Sd"
cv_all <- cbind(cv_results_m, cv_mean, cv_sd)
kable(t(cbind(cv_all)),digits=2)
}
```

```
cv_function_MLP(hpc_01,79,5,100,metrics_list = c("R2","RMSE"),0.001,0.9,100,"20")
```

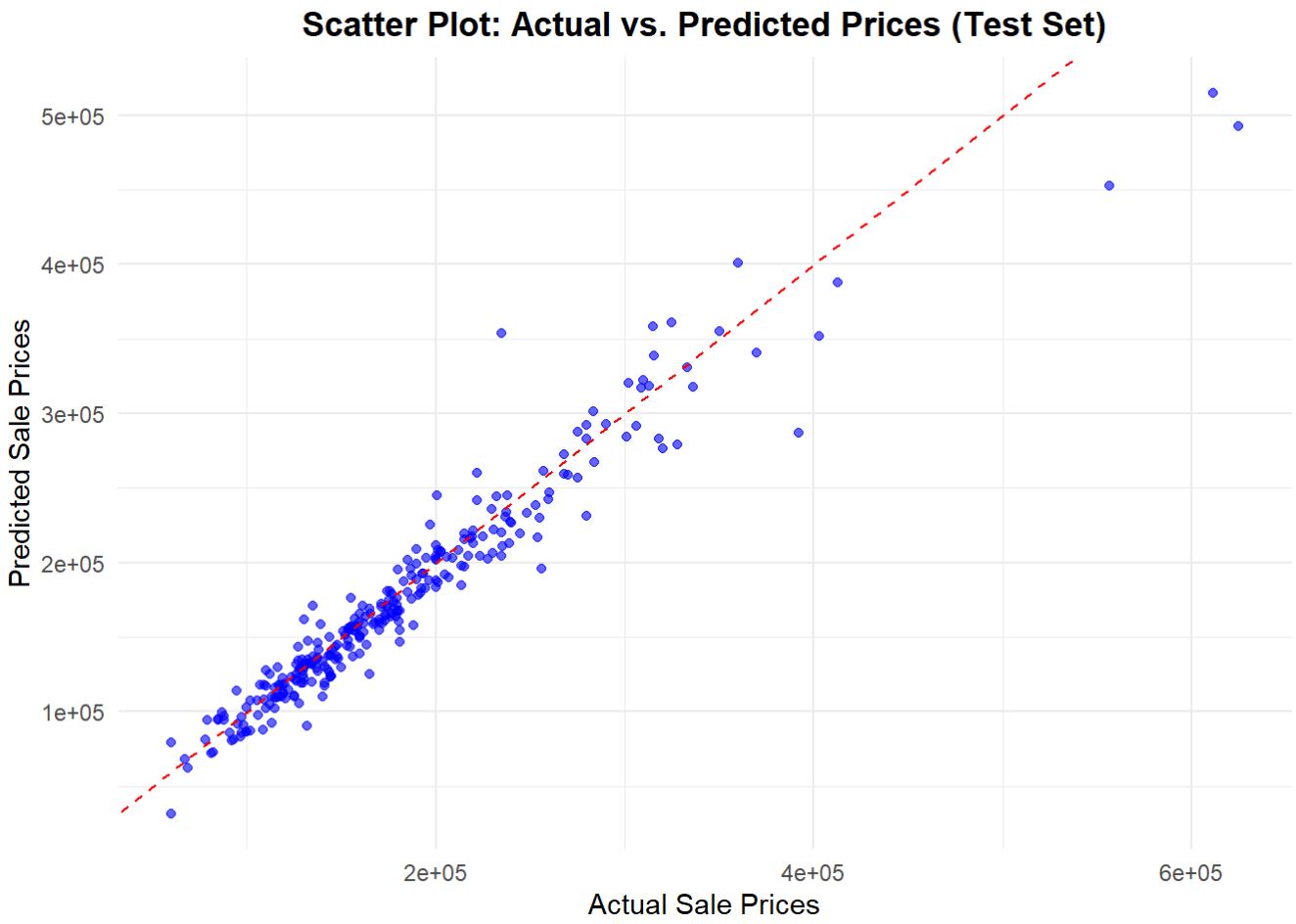
	R2	RMSE
Fold1	0.90	24178.27
Fold2	0.91	24245.88
Fold3	0.79	38569.63
Fold4	0.92	21699.27
Fold5	0.89	27003.66
Mean	0.88	27139.34
Sd	0.05	6659.66

```
ridge_best_lambda <- cv_ridge$lambda.min
ridge_model <- glmnet(X, y, alpha = 0, lambda = ridge_best_lambda)
```

Visualizations

Scatter Plot: Actual vs. Predicted Prices (Test Set)

```
# Scatter Plot: Actual vs. Predicted Prices (Test Set)
scatter_plot <- ggplot(data = data.frame(Actual = test_outcome, Predicted = final_pr
edictions), aes(x = Actual, y = Predicted)) +
  geom_point(color = "blue", alpha = 0.6) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  labs(title = "Scatter Plot: Actual vs. Predicted Prices (Test Set)",
       x = "Actual Sale Prices",
       y = "Predicted Sale Prices") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold"))

plot(scatter_plot)
```

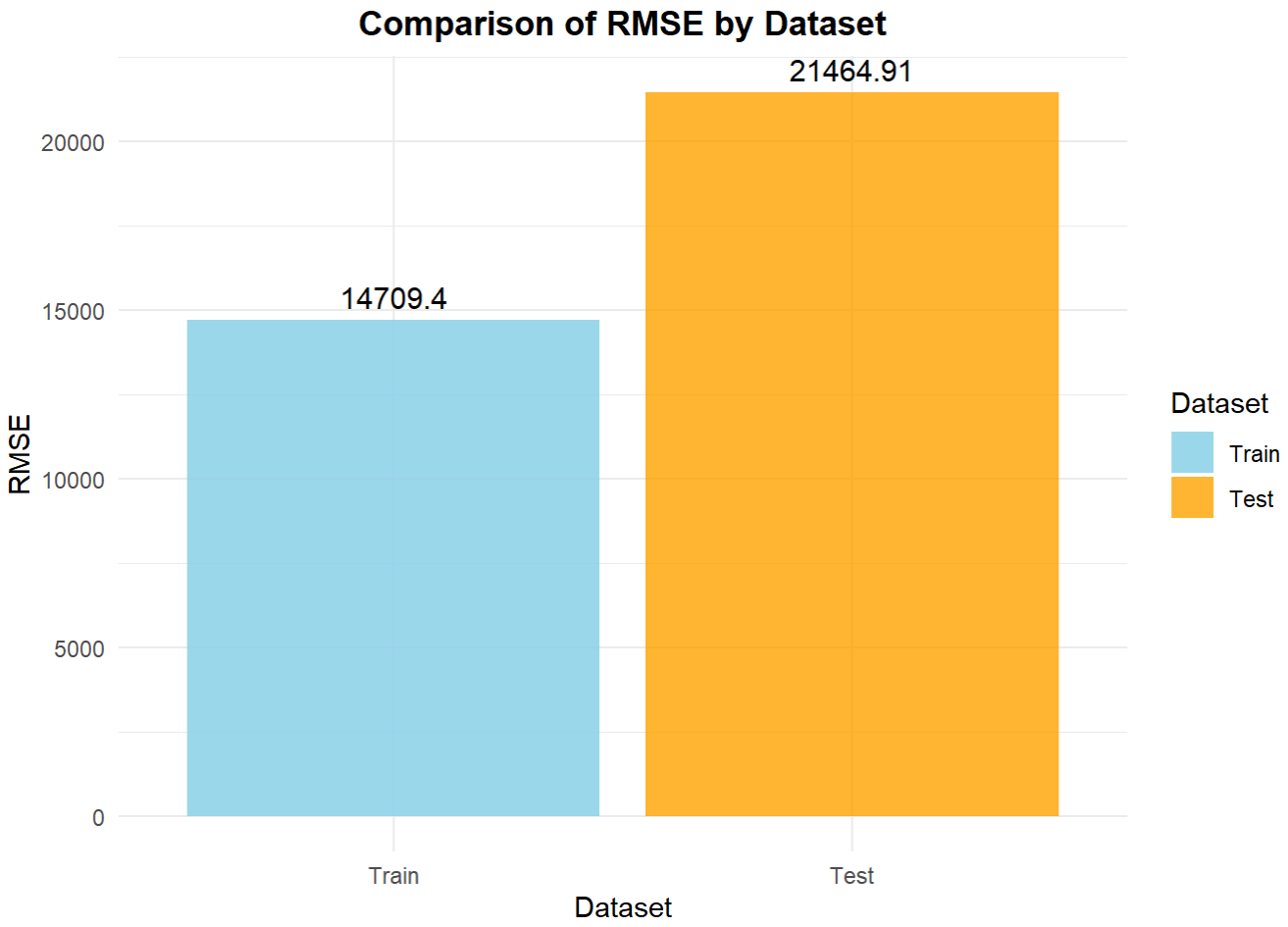


Scatter Plot:

It Plots test_outcome (actual values) vs. final_predictions (predicted values). Visual feedback on how well the model's predictions align with actual values. A tight clustering around the diagonal line indicates a good fit. The diagonal red dashed line indicates perfect predictions. Points close to this line signify accurate predictions.

Comparison Table

```
# Comparison Table Visualization

comparison_metrics$Dataset <- factor(comparison_metrics$Dataset, levels = c("Train",
"Test"))
comparison_plot <- ggplot(comparison_metrics, aes(x = Dataset)) +
  geom_col(aes(y = RMSE, fill = Dataset), position = position_dodge(width = 0.8), al
pha = 0.8) +
  geom_text(aes(y = RMSE, label = round(RMSE, 2)), vjust = -0.5, color = "black", si
ze = 4) +
  labs(title = "Comparison of RMSE by Dataset",
       x = "Dataset",
       y = "RMSE") +
  theme_minimal() +
  theme(plot.title = element_text(hjust = 0.5, face = "bold")) +
  scale_fill_manual(values = c("Train" = "skyblue", "Test" = "orange"))

plot(comparison_plot)
```



Comparison Table Visualization:

Displays the RMSE values for the train and test datasets as bar heights.

It is an intuitive comparison of RMSE for training and testing datasets to check for overfitting or underfitting. It also annotates the bar heights with their corresponding RMSE values for clarity.

Assumptions, Limitations and Robustness Checks for Artifical Neural Networks

Assumptions

Linear Separability in Transformed Space: Although neural networks can capture non-linear patterns, the dataset must contain enough information for meaningful separation of price categories.

Feature Importance: All selected variables are assumed to have predictive relevance. Features like GarageCars and GrLivArea were identified based on prior knowledge and exploratory data analysis.

No Multicollinearity Among Predictors: Preprocessing removed or combined redundant variables to prevent multicollinearity from impacting model performance.

Data Consistency: The relationships between predictors and the target variable are assumed to be consistent over time and across samples.

Limitations

Model Interpretability: Neural networks are inherently less interpretable compared to simpler models (e.g., linear regression). Understanding feature contributions requires additional tools like SHAP or LIME.

Computational Cost: avNNET's lightweight architecture is suitable for small datasets but may struggle with scalability for larger datasets.

Imputation Bias: Imputation methods for missing values (e.g., median for LotFrontage) could introduce bias if missingness is not random.

Limited Generalizability: Results are specific to the Ames, Iowa dataset and may not generalize to other housing markets.

Robustness Checks

Cleaning: We chose not to remove any columns except for the ID Column. We additionally added a binary column to more explicitly record whether a house has a garage or not. The model may have trained differently had we decided to remove some columns.

Outlier Cleaning was also relatively conservative, however MLP's are fairly robust to outliers, thus this shouldn't be a huge issue.

Cross-Validation: Performed 5-fold cross-validation to evaluate the model's performance across different subsets of the data.

Grid Search for Hyperparameters: Conducted a thorough grid search to optimize parameters like learning rate, hidden neurons, and activation function.

Alternative Architectures: Compared avNNET results with RWeka's neural network implementation to validate findings.

Sensitivity Analysis: Assessed model stability by removing less important variables and retraining.

Infinite Hyperparamter Combinations: There are multiple different hyperparameter combinations which could be applied. It would be computationally intensive and time consuming to evaluate all these combinations.

Random Forest

```
set.seed(100)

# Remove missing values as previous workflows can sometimes reintroduce them
hpc <- hpc %>%
  filter(!is.na(GarageYrBlt))  #removes NA rows

sum(is.na(hpc$GarageYrBlt))
```

```
## [1] 0
```

```
# Split the data into training and testing sets
hpc_split <- initial_split(hpc, prop = 0.8)
hpc_train <- training(hpc_split)
hpc_test <- testing(hpc_split)


# Create a recipe (add preprocessing steps as needed)
hpc_recipe <- recipe(SalePrice ~ ., data = hpc_train) %>%
  step_normalize(all_numeric_predictors()) %>%  # Normalize numeric predictors
  step_nzv(all_predictors())  # Remove all near-zero variance predictors
```

Define Model

```
# Define the Random Forest model
model_hpc <- rand_forest(
  mtry = tune(),   #no .of features
  trees = 100,    #no. of trees
  min_n = 10   #minimum samples
) %>%
  set_mode("regression") %>%  # Regression mode
  set_engine("ranger")
```

Tuning using cross validation and grid search

```
# Create cross-validation folds
hpc_folds <- vfold_cv(hpc_train, v = 5)

# Define a workflow
hpc_wf <- workflow() %>%
  add_model(model_hpc) %>%
  add_recipe(hpc_recipe)


# Tune the model using grid search
hpc_tune <- hpc_wf %>%
  tune_grid(
    resamples = hpc_folds,
    grid = 10,
    metrics = yardstick::metric_set(yardstick::rmse, yardstick::rsq)  #Use rmse and
rsq
  )
```

Test set Performance

```
# Select the best model based on RMSE
best_model_hpc <- hpc_tune %>%
  select_best(metric = "rmse")


# Finalize the workflow with the best model
final_workflow_hpc <- hpc_wf %>%
  finalize_workflow(best_model_hpc)

# Fit the final model on the training data and evaluate on the test set
final_fit_hpc <- final_workflow_hpc %>%
  last_fit(split = hpc_split)


# Collect metrics for the test set
test_metrics <- final_fit_hpc %>%
  collect_metrics()
print("Test Set Metrics:")
```

```
## [1] "Test Set Metrics:"
```

```
print(test_metrics)
```

```
## # A tibble: 2 × 4
##   .metric .estimator .estimate .config
##   <chr>   <chr>          <dbl> <chr>
## 1 rmse    standard   33348.    Preprocessor1_Model1
## 2 rsq     standard       0.787 Preprocessor1_Model1
```

```
# Collect predictions for the test set
test_predictions <- final_fit_hpc %>%
  collect_predictions()
```

Train set Performance

```
# Extract the finalized model again to evaluate on train set
final_model <- final_workflow_hpc %>%
  fit(data = training(hpc_split))

# Collect predictions for the training set
train_predictions <- predict(final_model, new_data = training(hpc_split)) %>%
  bind_cols(training(hpc_split))

# Calculate metrics for the training set
train_metrics <- train_predictions %>%
  yardstick::metrics(truth = SalePrice, estimate = .pred)
print("Training Set Metrics:")
```

```
## [1] "Training Set Metrics:"
```

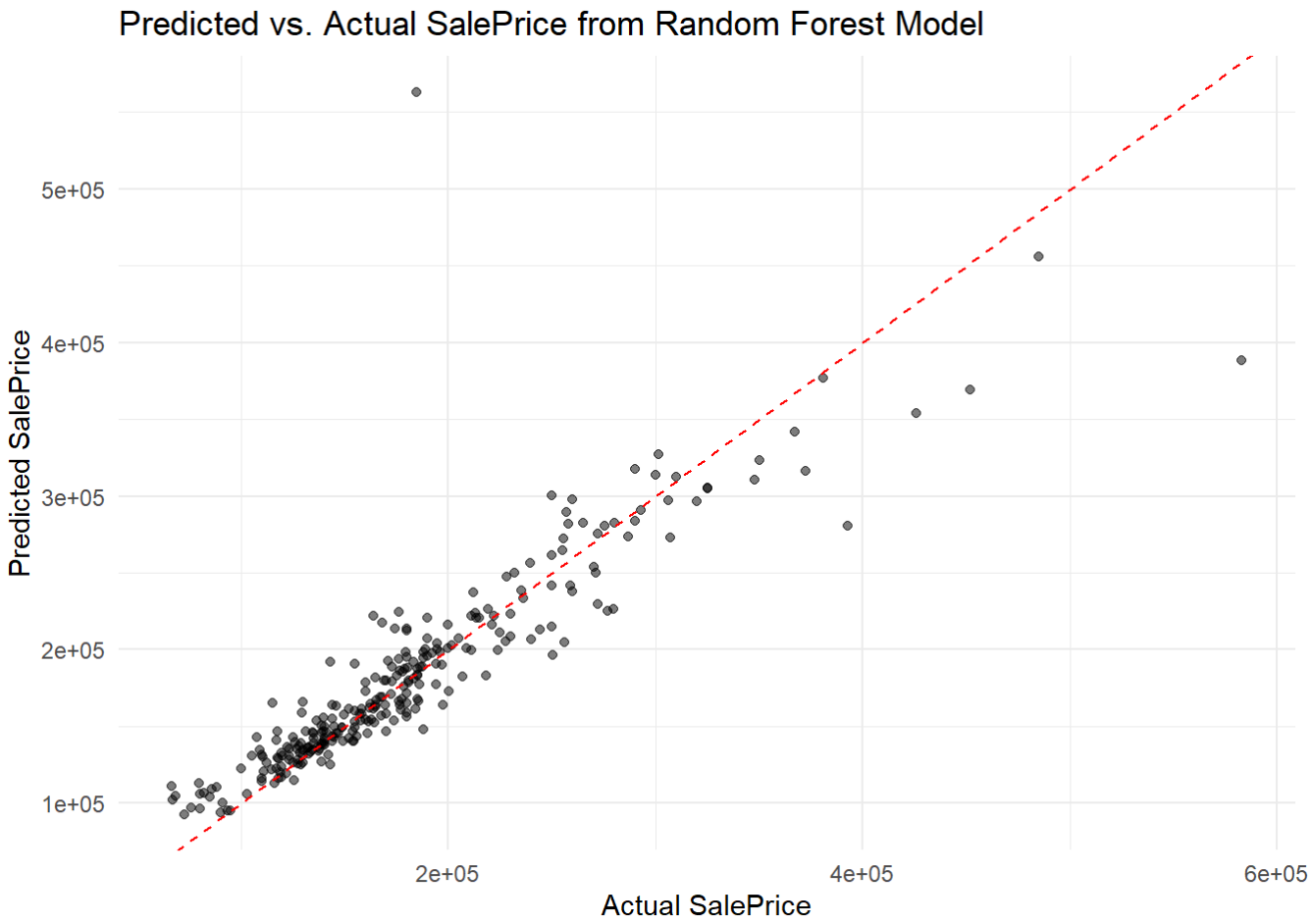
```
print(train_metrics)
```

```
## # A tibble: 3 × 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard   13588.
## 2 rsq     standard       0.975
## 3 mae     standard    8227.
```

The Random Forest model demonstrates strong performance on the training set, with RMSE of 13,587.71, MAE of 8,227.26 and a high R^2 of 0.974, indicating it fits the training data very well. However, its performance on the test set significantly declines, with a much higher RMSE of 33,347.92 and a lower R^2 of 0.786. The model is overfitting the training data. While further tuning could be considered, the current results from multiple models reflect a comprehensive evaluation.

Visualize Predictions for Test Data

```
ggplot(test_predictions, aes(x = SalePrice, y = .pred)) +
  geom_point(alpha = 0.5) +
  geom_abline(slope = 1, intercept = 0, color = "red", linetype = "dashed") +
  theme_minimal() +
  labs(title = "Predicted vs. Actual SalePrice from Random Forest Model",
       x = "Actual SalePrice",
       y = "Predicted SalePrice")
```



Most points cluster around the line, indicating that the model performs well for a majority of predictions. However, there are visible deviations and outliers at higher SalePrices, where the model over predicts these values.

Limitations, Assumptions, and Robustness Checks for Random Forest Model

Limitations:

1. Random Missing: Removing rows assumes that missingness is random, but if the missing values are systematic (e.g. more frequent in higher or lower SalePrice observations), the results could be biased.

2. Bias-Variance Tradeoff: Reducing training time and computation cost by decreasing the number of trees or folds may increase bias and overlook complex patterns, leading to lower predictive accuracy.
3. Lack of Interpretability: While the model performs decently well on training data, it is difficult to interpret important predictors and decision boundaries from the results.

Assumptions:

1. Sufficient Tuning: The chosen hyperparameter ranges assume they sufficiently capture the optimal space for model performance.
2. Non-Linearity: The results from the model are based on a non-linear assumption between predictors and the target variable.
3. No Multicollinearity: As it selects features and observations randomly during tree construction, it is assumed to be robust to multicollinearity.

Robustness Checks:

1. Cross-Validation: The model's performance was validated using 5-fold cross-validation, ensuring generalization across different subsets of the data.
2. Hyperparameter Tuning: Grid search tuning was performed for key parameters such as mtry and trees to optimize model performance.
3. Outlier Detection: Visual representation of predictions vs. actual values highlights areas where the model struggles and needs attention, particularly for extreme values.

Linear Regression Comparison

```
set.seed(123)

train_rows <- createDataPartition(hpc_01$SalePrice,p = 0.8, list = FALSE)

train_set_01 <- hpc_01[train_rows,]
test_set_01 <- hpc_01[-train_rows,]

vars_to_drop <- c("Condition2","MiscFeature","PoolQC","Exterior1st","Exterior2nd")
# drop variables with different levels

# Drop the variables from the train set
train_set_01 <- train_set_01[, !(colnames(train_set_01) %in% vars_to_drop)]

# Drop the variables from the test set
test_set_01 <- test_set_01[, !(colnames(test_set_01) %in% vars_to_drop)]

m2 <- lm(SalePrice ~ ., data = train_set_01)

m1_te_pred <- predict(m2,newdata=  train_set_01)
m2_te_pred <- predict(m2,newdata = test_set_01)

mmetric(m1_te_pred,train_set_01$SalePrice,metric = c("R2","RMSE"))
```

```
##           R2         RMSE
## 9.284101e-01 2.106820e+04
```

```
mmetric(m2_te_pred,test_set_01$SalePrice,metric = c("R2","RMSE"))
```

```
##           R2         RMSE
## 8.864826e-01 2.512570e+04
```

This is just a simple linear regression model to estimate a baseline for comparison. Some of the variables had to be dropped like `Exterior1st` which had different levels between the train and test sets. (In other words one of `Exterior1st` levels was in the train set but not the test set. This caused linear regression to have some issues.

However, despite this the model generalizes well to the data with a 4% drop between train and test in R^2. The drop in RMSE however is moderate at 4,057 dollars.

The model also does really well, but not quite as well as some of its other counterparts like Elastic Regression or Neural Networks. It however needs variables with a lot of variance, not as robust as the other models.